令和 7 年度 修士論文

実態調査に基づくパスキー認証の実装アンチパターンの抽出と分析

Empirical Analysis of Passkey Implementation Anti-Patterns

川原　悠佑

早稲田大学　基幹理工学研究科

情報理工・情報通信専攻

5124F031-2

提出日　　：令和 8 年 1 月 26 日

研究指導名：暗号プロトコル研究

指導教員　：佐古 和恵 教授

概要

パスキー認証は、FIDO2 仕様群（Web Authentication API / Client to Authenticator Protocol）に基づく公開鍵暗号技術を用いた認証方式であり、フィッシング耐性のある認証手段として普及が進んでいる。一方で、導入時の Web Authentication API のパラメータ設定や周辺設計が不適切である場合、仕様が意図する安全性や利便性が損なわれる恐れがある。本研究では、仕様、公開ガイドライン、先行研究を参照し、サービスがパスキー認証を実装する際に判断すべき事項を外部から観測可能な評価観点として整理した。次に、国内事業者が提供する Web サービス 10 件および国外事業者が提供する Web サービス 20 件の計 30 件を対象に、統一した手順で Web Authentication API の主要パラメータ設定と周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー）を観測し、評価観点に照らして実装状況を整理した。併せて、国内と国外を区別して集計し、実装傾向の違いも確認した。調査の結果、同じ仕様に基づく実装であっても、仕様や公開ガイドラインが示す推奨と異なるパラメータ設計および周辺設計が多く確認され、その一部には安全性や利便性を損なう可能性がある実装が含まれていた。これらをアンチパターンとして抽出し、同種の悪影響を及ぼすものを、 (i) パスキー認証の利用開始に向けた案内の不足、(ii) ユーザー操作支援の不足、(iii) パスキー管理のための情報提示の不足、(iv) 安全性を損なう周辺設計、の四つの類型として整理した。以上により、実装者がユーザーの利用場面に沿って設計課題を把握し、改修方針を検討するための観点を提示した。

目次

1 はじめに 8
1.1 研究背景 8
1.2 目的 9
1.3 本論文の構成 10

2 パスキー認証の仕様・関連研究の整理 11
2.1 パスキー認証の概要と登場主体 11
2.1.1 登録・認証セレモニー 12
2.1.2 役割分担と RP の実装範囲 14
2.1.3 Web Authentication API の詳細 15
2.1.4 Attestation 17
2.1.5 ユーザーインターフェースとユーザー体験 18
2.1.6 用語と基本概念の補足 19
2.2 実装指針および関連資料の参照源 21
2.2.1 標準化団体およびコミュニティによるガイドライン 21
2.2.2 ベンダーおよび開発者向けドキュメント 22
2.2.3 公的基準および書籍・技術ブログ 22
2.3 先行研究によって指摘された実装課題 23
2.3.1 実装におけるセキュリティ脆弱性の検証 23
2.3.2 ユーザーの理解と心理的障壁 24
2.3.3 本研究の立ち位置 24

3 設計上の論点と評価観点 25
3.1 Web Authentication API パラメータ設定 25
3.1.1 navigator.credentials.create() における考慮事項 25
3.1.2 navigator.credentials.get() における考慮事項 28
3.2 UI/UX（登録・認証手順）の設計 29
3.2.1 アカウント作成とパスキー登録への誘導 29
3.2.2 用語の統一と誤解の防止 30
3.2.3 登録セレモニーのセキュリティと UX 31
3.2.4 ログイン手順の設計 32
3.2.5 パスキー管理機能の設計 32
3.3 多様な認証環境への対応 34

3.4 アカウントリカバリー戦略 35
3.4.1 リカバリー問題の複雑性 35
3.4.2 アカウントリカバリー設計 36

4 調査設計 38
4.1 調査対象の選定 38
4.1.1 調査要件 38
4.1.2 国内サービスの抽出 38
4.1.3 国外サービスの抽出 39
4.1.4 除外事例とその理由 39
4.1.5 調査対象サービス一覧 39
4.2 調査手法 41
4.2.1 調査の目的と評価観点 41
4.2.2 調査環境の構築 41
4.2.3 記録項目と記録方法 41
4.2.4 調査項目と実行手順 42
4.2.5 倫理面および安全性への配慮 43

5 調査結果 45
5.1 Web Authentication API パラメータの実装状況 45
5.1.1 navigator.credentials.create() 45
5.1.2 navigator.credentials.get() 52
5.2 周辺設計の実装状況 57
5.2.1 アカウント作成とパスキー登録への誘導 57
5.2.2 用語の統一と誤解の防止 58
5.2.3 登録セレモニーのセキュリティと UX 60
5.2.4 ログイン手順と Conditional UI 68
5.2.5 パスキー管理機能の実装状況 69
5.2.6 仮想認証器と多様な認証環境への対応 74
5.2.7 アカウントリカバリーの実装状況 75

6 考察 79
6.1 アンチパターンの四類型 79
6.1.1 アンチパターンの類型 1：パスキー認証の利用開始に向けた案内の不足 79
6.1.2 アンチパターンの類型 2：ユーザー操作支援の不足 80
6.1.3 アンチパターンの類型 3：パスキー管理のための情報提示の不足 82
6.1.4 アンチパターンの類型 4：安全性を損なう周辺設計 83

6.1.5 まとめ 84
6.2 パスキーに関連する新たな仕様が与える影響 84
6.2.1 クレデンシャルエクスポート／インポート仕様（Credential Exchange Specifications） 84

7 結論 **87**
7.1 貢献と示唆 87
7.2 今後の展望 87

謝辞 **89**

参考文献 **90**

図目次

1 登録セレモニーの概念図 12
2 認証セレモニーの概念図 13
3 Conditional UI の例。Passkeys Demo [1] を利用。 19
4 パスキー管理画面の例 [2]。 33

表目次

1 調査対象 Web サイト一覧 40
2 navigator.credentials.create() パラメータ：国内 46
3 navigator.credentials.create() パラメータ：国外 47
4 navigator.credentials.get() パラメータ：国内 52
5 navigator.credentials.get() パラメータ：国外 54
6 パスキーに関する言葉遣い：国内 58
7 パスキーに関する言葉遣い：国外 59
8 登録セレモニーのセキュリティと UX：国内 61
9 登録セレモニーのセキュリティと UX：国外 63
10 パスキー管理画面調査結果：国内 69
11 パスキー管理画面調査結果：国外 70
12 パスキー以外のログイン方法：国内 75
13 パスキー以外のログイン方法：国外 76

1　はじめに

1.1　研究背景

インターネットサービスが社会基盤として定着した現在、オンライン認証は、多様なユーザーを安全に受け入れるための根幹技術である。しかし実運用では、長年にわたりパスワード認証が中心であった。パスワード認証は実装が容易で普及しやすい一方で、ユーザーが秘密情報を記憶し、入力し、そしてサービス側がそれを照合する、という共有秘密モデルに依存する。このモデルは、フィッシングやマルウェア、情報漏えいといった現実の脅威と相性が悪く、少なくともパスワードそのものはフィッシング耐性[*1]を持たないことが明確に指摘されている [3]。さらに、パスワードの複雑さの要件の追加や定期変更といった運用ルールを課しても、ユーザー負担を増やす一方で、現実の安全性向上を一律に保証しにくいことが、実証的に指摘されている [4, 5]。この点について、NIST は過度な構成規則を課さないことや、任意の周期での変更を要求しないことを推奨している [3]。

こうした状況を受けて、多要素認証（Multi-Factor Authentication、以下 MFA）が広く導入されてきた。MFA は単一要素の漏えいに対する耐性を高めるための有効な手段である一方で、追加の操作がユーザーの負担となり、導入と運用の複雑性も増す。また、方式によっては新たな攻撃ベクトルが生まれ得るため、「MFA を導入すれば十分に安全である」と単純化して結論づけることは難しい。例えば、ワンタイムパスワード（OTP）のような手動入力を伴う方式は、フィッシング耐性を満たさないことが指摘されている [3]。したがって、セキュリティと利便性の両立を、より根本から支える認証方式が求められている。

この文脈で注目されているのが、FIDO2 仕様に基づく認証方式、**パスキー認証**である。パスキー認証とは、公開鍵暗号技術を用いた認証方式であり、パスキーと呼ばれる公開鍵暗号鍵ペアを利用してフィッシング耐性のある認証を提供する [6]。FIDO2 は、パスワード認証への依存を減らすことを目的とする業界団体 FIDO Alliance [7] と、Web 技術の標準化団体である W3C [8] により標準化が進められている仕様群であり、Web Authentication API（WebAuthn とも呼ばれる）[9, 10] と、Client to Authenticator Protocol（以下、CTAP）[6, 11] から構成される。Web Authentication API は、サービス提供者（Relying Party、以下 RP）がブラウザ上の Web アプリケーションからパスキーの登録や認証を行うための API を定義する。そして、CTAP は Web Authentication API を補完するものであり、パスキー認証においてブラウザ/OS が外部のデバイスと USB/NFC/BLE などを介して通信するためのプロトコルを定義する [6, 11]。

パスキー認証は公開鍵暗号に基づいた認証方式であり、RP と秘密情報を共有しない [9]。また、クレデンシャルスタッフィング[*2]やフィッシング攻撃に耐性がある [12]。さらに、ユーザー

[*1] 本研究における「フィッシング耐性」とは、正規のサーバーを装う攻撃者に対し、ユーザーの注意に依存せず、認証に使う秘密情報や認証に用いる情報が攻撃者へ開示・転用されることを防ぐ認証プロトコルの性質を指す [3]

[*2] あるサイトで侵害されたパスワードなどの認証に使う秘密情報を使用して他のサイトでユーザーのアカウントに

はパスワード等の共有秘密をサーバー側に送信して照合する代わりに、デバイス（スマートフォン、パソコン、セキュリティキー）のロック解除に用いるのと同様の本人確認手段（例：生体認証、PIN 入力）を実行するだけで認証を完了できる [12, 13]。

このように、パスキー認証は安全性と利便性の両面で有望であり、現在国内外の主要な Web サービスでも導入が進んでおり、今後のオンライン認証における重要な選択肢になりつつある。実際に、FIDO Alliance は、2025 年 5 月 1 日に公表した複数国（日本を含む）における消費者調査において、パスキーの認知が 74%、少なくとも 1 つのアカウントで有効化した人が 69% に達したと報告している [14]。また、世界の上位 100 の Web サイトの 48% でパスキー認証が導入されていると述べており、パスキーの可用性が増加していることを示している [14]。

しかし、パスキー認証は「導入すれば自動的に安全で使いやすくなる」わけではない。パスキー認証を導入する際、RP は Web Authentication API のパラメータ設定や様々な周辺設計を行う必要がある。周辺設計には、ユーザーがパスキー認証を利用する際の UI/UX、そのサービスに登録したパスキーをユーザーが管理できるようにする機能、どのような認証器を受け入れるかの方針、そしてパスキー認証が利用できない場合の代替手段（アカウントリカバリー）などが含まれる [9]。これらのパラメータ設定や周辺設計が不適切である場合、パスキー認証の仕様が意図する安全性や利便性が損なわれる恐れがある。また、ユーザーが仕組みを理解できず、利用が進まない可能性がある。

このようなパラメータ設定や周辺設計に関して、仕様および公開ガイドライン（詳細は第 2 章で述べる）は、実装における推奨事項を RP に示している。

また、先行研究でも（詳細は第 2 章で述べる）、RP の実装・運用上の不備に起因するセキュリティ上の課題が指摘されてきた [15, 16]。加えて、パスキー認証に対するユーザーの誤解や心理的障壁も報告されている [17]。しかし、パスキー認証への対応が進む中で [14]、主要な Web サービスが仕様・公開ガイドライン等が示す推奨をどの程度遵守しているのか、推奨と異なる実装としてどのようなものがあるのかを、網羅的に観測・比較した整理は不足している。

1.2　目的

本研究の目的は、RP が導入・運用時に陥りやすい実装上の落とし穴となり得る点を、主要 Web サービスの実装実態に基づいて整理して提示することである。そのために、アンチパターンとその類型を示す。ここで、本論文における**アンチパターン**とは、仕様や公開ガイドライン等が示す推奨と異なる実装のうち、安全性や利便性を損なう実装を指す。

アンチパターンは、仕様や公開ガイドライン等が示す推奨事項を、導入・運用の点検に利用しやすい形で補完する。そして、複数のアンチパターンを影響が現れる場面に沿って類型化して示すことで、RP は一つのアンチパターンを手掛かりとして、同種の実装（アンチパターン）が他にも存在しないかを横断的に点検できる。

アクセスする攻撃 [3]

この目的を達成するために、以下の点に取り組む。

1. Web Authentication API のパラメータ設定と周辺設計について、既存仕様・公開ガイドライン・先行研究を基に、調査した実装状況を評価するための項目（以下、評価観点）を整理する。
2. 主要 Web サービスを対象に実装の実態調査を行い、各評価観点に対する実装状況を整理する。そして、推奨と異なる実装を抽出し、アンチパターンを明らかにする。併せて、国内サービスと国外サービスを区別して集計し、傾向の違いも確認する。
3. 調査結果を踏まえ、明らかにしたアンチパターンを類型化して整理し、その影響を考察する。そして、パスキーに関連する仕様の動向も踏まえて、RP が導入と運用を検討する際の示唆を述べる。

1.3　本論文の構成

本論文は、上記の目的に沿って以下のように構成する。

第 2 章では、パスキー認証の基盤となる Web Authentication API を中心に、まず登場主体と情報の流れ、登録・認証セレモニーを整理する。そして、RP が実装しなくてはならない処理範囲を明確にし、その範囲の中で主要な設定項目の意味、ならびにブラウザ／ OS が提示する UI と RP が補完する UI/UX の関係を順に整理する。併せて、設計・実装判断の参照源となるガイドライン類と先行研究を整理し、本研究の前提と立ち位置を明確化する。

第 3 章では、第 2 章で整理した仕様、ガイドライン、先行研究を踏まえ、RP が設計上判断すべき論点を、Web Authentication API のパラメータ設定と周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー）について整理し、以降の章で用いる評価観点を整理する。

第 4 章では、第 3 章で整理した評価観点に基づき、調査対象の選定方法、データ収集手順、調査項目、および倫理面・安全性への配慮を述べる。

第 5 章では、第 4 章の設計に基づく調査結果を示し、第 3 章で整理した評価観点と照合しながら、Web Authentication API のパラメータ設定と周辺設計の実装状況を整理・分析する。併せて、国内サービスと国外サービスを区別して集計し、傾向の違いも確認する。

第 6 章では、第 5 章の結果を総合して、アンチパターンとその類型を整理し、影響を考察する。また、パスキーに関連する仕様の動向を踏まえ、RP が導入と運用を将来にわたり維持するための示唆を述べる。

第 7 章では、本研究の貢献をまとめ、今後の課題と展望を述べる。

2　パスキー認証の仕様・関連研究の整理

本章では、以降の章で用いる前提として、パスキー認証を仕様・指針・先行研究の観点から整理する。まず、Web Authentication API の仕様に基づき、登場主体と情報の流れを示し、登録・認証セレモニーの概要を整理する。ここで、セレモニーとはネットワークプロトコルの拡張であり、コンピュータノードだけでなく人間も含めた一連のプロセスを指す [9]。

次に、セレモニーのうちサービス提供者（Relying Party; RP）が実装する処理範囲を明確にし、その範囲で設計・実装判断が生じる Web Authentication API の主要なパラメータの意味や、ブラウザ／ OS が提示する UI と RP が補完する UI/UX の関係を概観する。そのうえで、実装上の判断に影響を与えるガイドライン類の参照源を整理し、導入実態・脆弱性・ユーザーの理解や心理的障壁に関する先行研究を概観して、本研究の立ち位置を明確にする。

2.1　パスキー認証の概要と登場主体

本節では、パスキー認証に登場する主体を、RP・ブラウザ/OS・認証器・ユーザーの 4 者で整理する。

パスキー認証では、RP が実装するサーバーがユーザーのブラウザ／ OS に対してパスキーの登録・認証を要求し、ブラウザ／ OS は暗号関連の操作を担当する主体である認証器に対して鍵生成や署名生成を依頼する。

サーバーは、登録時に得た公開鍵や、認証時に得た署名等を検証し、登録の成立やログインの成否を決定する [9]。このとき秘密鍵は認証器に保持され、サーバーは秘密鍵を取得しない。

登録・認証の要求は、RP が提供するブラウザ上で動作する Web アプリケーション（JavaScript）が、ブラウザが提供する Web Authentication API を呼び出すことで開始される [9]。ブラウザ／ OS が外付けデバイス等の認証器と通信する際には、Client to Authenticator Protocol（CTAP）という通信プロトコルが用いられる [6, 11]。

以上を踏まえ、本論文では主要な登場主体として以下の語を用いる。

Relying Party（RP）　サービス側の主体であり、ブラウザ上で動作する Web アプリケーション（JavaScript）とサーバー側の処理を実装・運用する。サーバーではチャレンジ生成や署名検証等の処理を行い、Web アプリケーションからの API 呼び出しを通じて登録・認証を開始し、受信した応答を検証して成否を決定する [9]。なお、後述する図 1、2 では、RP の実装するサーバーを「RP サーバー」、Web アプリケーションを「RP Web アプリケーション」と表記する。

ブラウザ／ OS　ユーザー側の実行環境であり、Web アプリケーション（ブラウザ上で動作する JavaScript）が利用する Web Authentication API を提供する。登録・認証の要求を受け取り、認証器との対話や UI 提示を仲介して処理を進行する [9, 10]。

認証器（Authenticator）　パスキー（鍵ペア）の生成、保持、及び署名生成などの暗号処理を行う、ハードウェアまたはソフトウェアである。必要に応じてユーザーの同意やユーザー検証を伴って暗号処理を許可する（詳細は後述）[9]。認証器は、ブラウザ/OS が動作するデバイスに内蔵されるプラットフォーム認証器と、そのデバイスに対して外付けで利用するローミング認証器に大別される（詳細は 2.1.6 を参照）[9]。

ユーザー　サービスの利用者であり、ブラウザ／ OS 上で操作を行い、認証器の暗号処理の実行に対して同意し、必要に応じてユーザー検証を行う主体である [9, 10]。

次に、登録・認証セレモニーの流れをより具体的に示す。

2.1.1　登録・認証セレモニー

パスキー認証は、大きく「登録（Registration）」と「認証（Authentication）」の 2 つのセレモニーに分けられる [9]。いずれのセレモニーにおいても、RP がランダムなチャレンジを生成してブラウザ/OS に渡し、RP がブラウザ/OS からの応答に含まれるチャレンジを照合することで、当該要求に対する応答であることを確認する [9]。このとき、秘密鍵は認証器に保持され、RP は秘密鍵を取得しない。

図 1、2 は、登録・認証セレモニーにおける主要な主体とそれらの間でのやり取りの流れを示す。

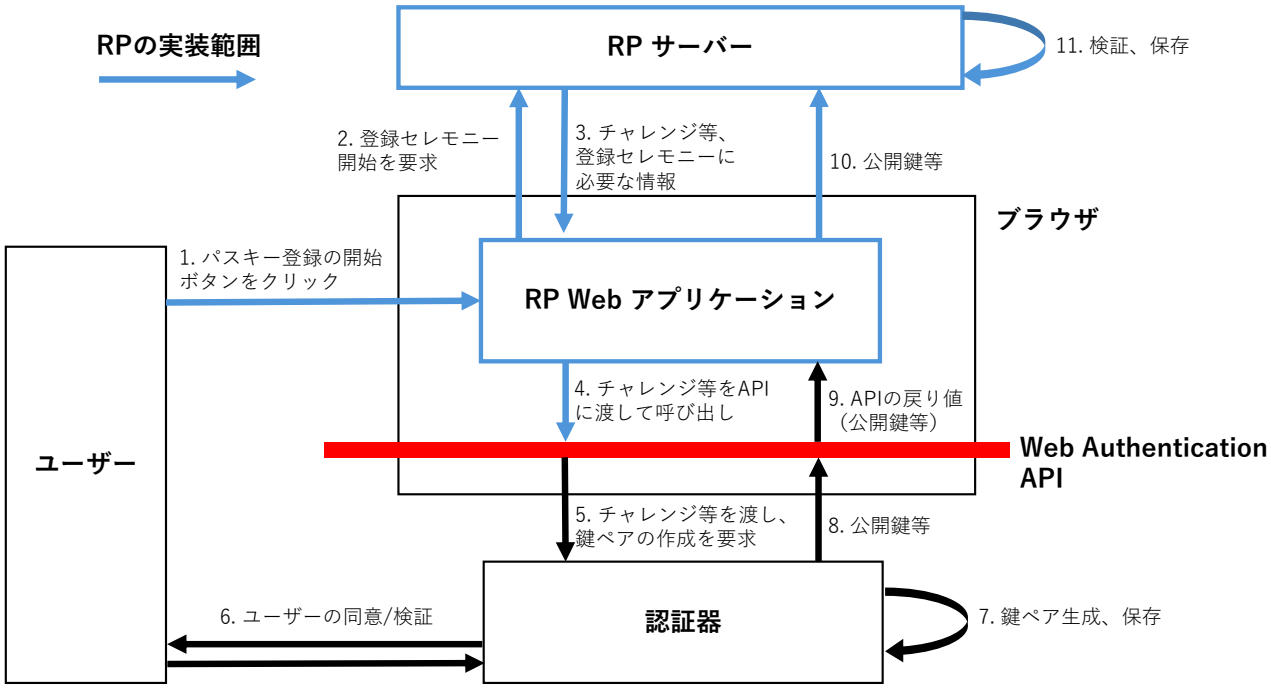

図 1　登録セレモニーの概念図

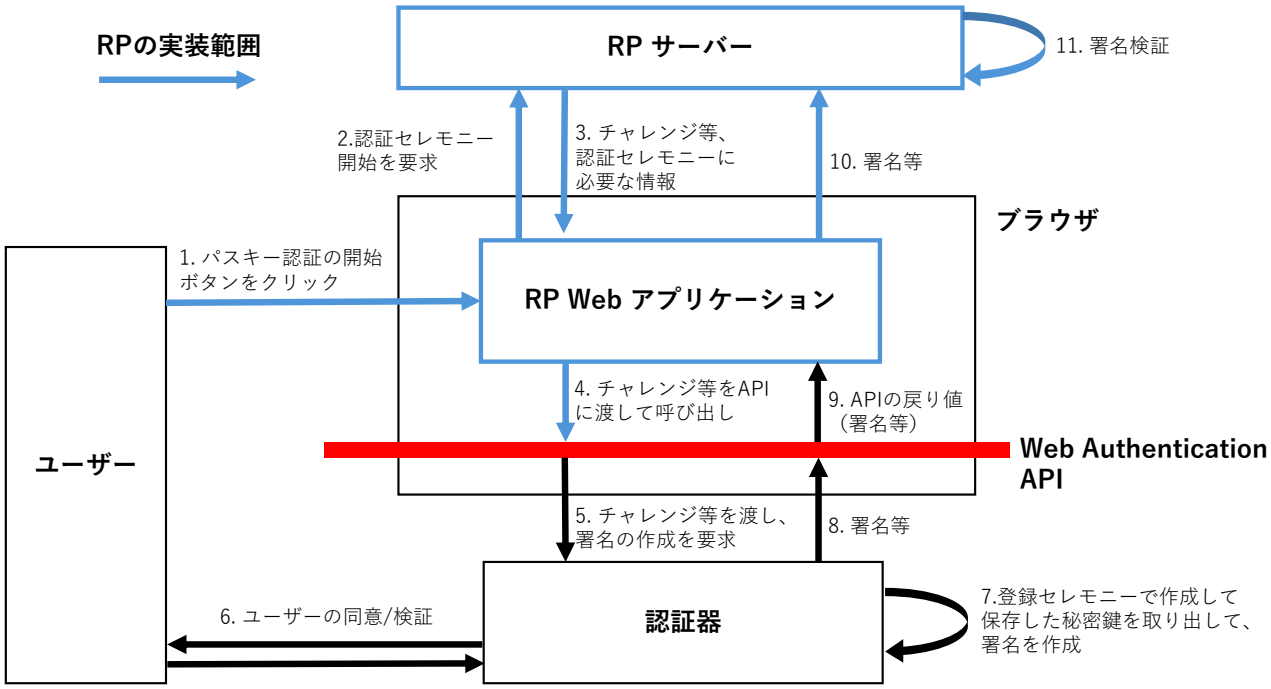

図 2　認証セレモニーの概念図

1. **登録セレモニー（Registration ceremony）**[9]: RP と、ユーザー、認証器、ブラウザ/OS が協調して、パスキーを作成し、その公開鍵をユーザーのアカウントに紐づける手続きである。RP はチャレンジを含む登録の要求を提示し、認証器は鍵ペアを生成して秘密鍵を保持し、公開鍵等を RP へ返す。RP は応答に含まれるチャレンジを照合し、当該登録要求に対する応答であることを確認したうえで、公開鍵等を保存する。応答に、RP が提示したチャレンジ等の登録の要求に対する、認証器による署名が付随する場合がある。
2. **認証セレモニー（Authentication ceremony）**[9]: ユーザーと認証器、ブラウザ/OS が協調して、RP に登録済みの公開鍵に対応する秘密鍵をユーザーが制御していることを、RP に対して暗号学的に証明する手続きである。RP が提示するチャレンジに対して、認証器が秘密鍵で署名した応答を生成し、RP がそれを検証してログインを成立させる。

セレモニーの最中、RP の要求に応じて認証器はユーザー検証を行う（図 1、2 の手順 6）。

■ユーザーの存在確認とユーザー検証　[9] 登録・認証セレモニーは、認証器が処理を実行するにあたりユーザーの同意（user consent）を前提として進行する。ユーザーの同意とは、ユーザーが求められている内容に同意したこと（提示された内容を読み、理解し、了承したこと）を指す。

ユーザーの同意は、セレモニー中の authorization gesture（ユーザーが認証器に対して行う操作）によって示されることが多い。authorization gesture は、登録・認証セレモニーの一部とし

て実施され、これが完了することでユーザーは当該セレモニーの進行を許可（ユーザーの同意を提供）したものとして扱われる。具体的には、単純に認証器に触れる等の操作のみで成立する場合（ユーザーの存在確認）もあれば、PIN 入力や生体認証伴う場合（ユーザー検証）もある。

ユーザーの存在確認（test of user presence; UP）は、authorization gesture の一つであり、ユーザーが、典型的には認証器に触れる等の操作を行ったことにより真偽値を得る技術的プロセスである。ただし、ユーザーの存在確認は生体認証や PIN の入力等を伴わないため、ユーザー検証とは区別される。

ユーザー検証（user verification; UV）は、PIN の入力や生体認証等により、認証器がユーザーを検証したことを示す技術的プロセスである。RP はブラウザ/OS を介して返される署名付きの応答に含まれるフラグを通じて、ユーザー検証が実施されたかどうかを確認できる。ユーザー検証は RP に対して「そのユーザーが具体的に誰であるか」を特定させるものではないが、ユーザー検証を伴うセレモニーが 2 回以上実行された場合、それら全てを同一のユーザーが実行したことが示される。

RP は、ブラウザから返されるデータに含まれるフラグにより、UP が実施されたか（UP フラグ）および UV が実施されたか（UV フラグ）を検証できる。

なお、ユーザーに対するプロンプト（PIN 入力や生体認証の要求画面等）は、認証器が表示する場合もあれば、認証器が表示能力を持たない場合にはブラウザ／ OS 側が提示する場合もある。

2.1.2　役割分担と RP の実装範囲

図 1、2 に示した登録・認証セレモニーにおいて、RP が実装する処理は、ブラウザ上で動作する Web アプリケーション（JavaScript）とサーバー側の双方にまたがる。主な項目を以下に示す。

- ブラウザ上で動作する Web アプリケーション（JavaScript）から Web Authentication API を呼び出してセレモニーを開始すること（登録セレモニーでは `navigator.credentials.create()`、認証セレモニーでは `navigator.credentials.get()`）
- 登録・認証セレモニーにおいて、サーバー側でチャレンジ（Challenge）を生成し、その他必要なパラメータとともにブラウザへ渡すこと
- 登録セレモニーで得た公開鍵等を、ユーザーのアカウントに関連付けて保存すること
- 認証セレモニーで得た署名等を、ユーザーのアカウントに紐付けられた公開鍵で検証し、ログインの成否を決定すること

上記に加え、UI/UX も RP が実装する必要がある。ブラウザ／ OS は、認証器へのアクセスを仲介し、ユーザーの同意やユーザー検証に関する UI を提示してセレモニーを進行する [9]。一方で RP は、ブラウザ／ OS が提示するこれらの UI の表示内容を制御できない。そのため、ユーザーが操作の意味を理解できるように、ページ内の説明、画面遷移の設計、失敗時の案内

や代替手段提示等を Web アプリケーション側の UI で補完しなくてはならない。

さらに、登録・認証の実行に際して「どの認証器が用いられるか」は、ユーザーが所有する認証器の状況や、ブラウザ／ OS が提示する選択 UI を通じたユーザーの選択にも依存する。一方で RP は、サービスの要件（例：法的な制限、許容する利用環境・デバイス等）に応じて、どのような認証器を受け入れるかの方針を定めたい場合がある。そこで、`navigator.credentials.create()` や `navigator.credentials.get()` 実行時に渡す引数を通じて、利用したい認証器の要件を示したり、認証器に関する情報をどの程度受け取って検証に用いるかの方針を示すことで、RP は受け入れ範囲を間接的に制御できる [9]。

加えて、登録したパスキーをユーザーが管理できるようにする機能や、パスキーが利用できなくなった場合に備えたアカウントリカバリー手段といった周辺設計も、RP に委ねられる。

以降では、これら RP が処理を担当する項目について整理する。まず Web Authentication API の詳細（主要パラメータ）を整理し、次にブラウザ／ OS が提示する UI と、RP が補完すべき UI/UX の関係を述べる。

2.1.3　Web Authentication API の詳細

ここでは、Web Authentication API Level 2 仕様 [9] に基づき、パスキーの登録・認証において RP が Web Authentication API に渡すオプション（要求に含める情報）と、その主要項目の意味を概観する。ここで、「Level」とは、Web Authentication API 仕様の世代（主要改訂）を指す呼称である。Level 1 は 2019 年に W3C 勧告（Recommendation）として公開され [18]、Level 2 はその保守的更新（maintenance update）として Level 1 を置換する勧告として公開された [9, 19]。また、Level 3 は策定中の草案（Working Draft）であり、現時点では勧告には至っていない [10]。

■Registration：navigator.credentials.create()　登録セレモニーで新しい鍵ペアを作成するために、ブラウザ上で動作する Web アプリケーション（JavaScript）は、`navigator.credentials.create()` を呼び出す [9]。

呼び出しの際、Web アプリケーションは、RP が用意した登録用のオプションを引数として渡す [9]。このオプション（`PublicKeyCredentialCreationOptions`）には、チャレンジや RP の識別情報、ユーザー情報、ならびに認証器の振る舞いを制御するための各種パラメータが含まれる [9]。ブラウザ／ OS はこのオプションを解釈し、必要に応じて認証器との対話やユーザーの操作（同意やユーザー検証）を仲介したうえで、登録結果を返す [9]。登録結果には、作成されたパスキーの識別子や公開鍵、チャレンジやオリジン[*3]等を含むデータ、ならびに（場合により）認証器に関する付加情報が含まれる [9]。

RP は、(i) 受信したチャレンジやオリジン、RP ID（以下で説明）が RP の期待するものかどうか、(ii) ユーザーの存在確認/ユーザー検証が行われたかどうか (iii) 必要に応じて後述する Attestation の検証 といった確認を行ったうえで、公開鍵等の必要な情報をサーバー側に保存す

[*3] オリジン（origin）は、スキーム・ホスト・ポートからなる（例：`https://example.com:443`）[20]。

る [9]。

以下では、この登録用オプションに含まれる主なパラメータを説明する。

- **rp**: RP の表示名（`rp.name`）と ID（`rp.id`）を指定する。`rp.id` はスキームやポートを含まないドメイン名そのものか、またはその上位のドメインに限られる [9]。例えば、`https://login.example.com:1337` からの呼び出しでは、`login.example.com` は有効かつこれが規定値となり、 `example.com` も有効となる。一方で、`m.login.example.com` や `com` は無効である [9]。
  Web 上では、呼出元オリジンのスキームは `https` であることが要求され（ポートは制約されない）、これらの制約によりフィッシング等で別オリジンからパスキーを不正に利用することを防止する [9]。
- **user**: ユーザーを一意に識別する ID、表示名 (`name`,`displayName`)。
- **challenge**: リプレイ攻撃を防ぐための、ランダムなバイナリデータ。必ず RP 側が生成し、少なくとも 16 バイトの長さを持たなくてはならない [9]。
- **pubKeyCredParams**: RP が許容する公開鍵暗号アルゴリズム（例: ES256、 RS256）のリスト。
- **authenticatorSelection**: 要求する認証器の要件を、以下の複数のプロパティにより設定する。
  - `authenticatorAttachment`: 利用する認証器の接続形態を指定する。`platform` を指定した場合、ブラウザ／ OS はデバイス内蔵のプラットフォーム認証器を用いてパスキーを登録することを意図できる。一方で `cross-platform` を指定した場合、USB/NFC/BLE 等で接続されるローミング認証器を用いてパスキーを作成することを意図できる [9]（用語の補足は 2.1.6 節）。
  - `requireResidentKey`: Discoverable Credential（詳細は 2.1.6 節を参照）として扱うかどうかを指定する（Level 2）[9]。これにより、RP が `allowCredentials` で Credential ID を提示しない場合でも、認証器側で候補を見つけて認証できる。Level 3 では、この挙動をより明確にするために `residentKey` パラメータ（`discouraged`、 `preferred`、 `required`）の使用が推奨されている [10]。
  - `userVerification`: 認証器が、ローカルでユーザーを識別するために生体認証や PIN の入力等によって行う UserVerification を要求するかどうか。（`required`：必須、 `preferred`：User Verification が行えなくても失敗しない、 `discouraged`：User Verification を行わない）。
- **excludeCredentials**：重複登録の抑止のために、パスキーを識別するためのバイト列である Credential ID（詳細は 2.1.6 を参照）を提示する。

■Authentication：navigator.credentials.get()　認証セレモニーで認証器に署名を作成させるために、ブラウザ上で動作する Web アプリケーション（JavaScript）は、ブラウザ上で `navigator.credentials.get()` を呼び出す [9]。呼び出しの際、Web アプリケーションは、RP が用意した認証用のオプションを引数として渡す [9]。このオプション（`PublicKeyCredentialRequestOptions`）には、チャレンジや RP の識別情報、ならびに利用可能なパスキーの絞り込み等の各種パラメータが含まれる [9]。ブラウザ／ OS はこのオプションを解釈し、必要に応じて認証器との対話やユーザーの操作（同意やユーザー検証）を仲介したうえで、認証結果を返す [9]。認証結果には、チャレンジ等に対する署名や、署名に付随するデータ（認証器データ等)、ならびに（場合により）ユーザー識別子が含まれる [9]。

RP は登録セレモニーで説明した (i)・(ii) の確認に加え、保存済みの公開鍵と照合して署名を検証し、ログインの可否を決定する [9]。

以下では、この認証用オプションに含まれる主なパラメータを説明する。

- **rpId**: RP の ID を指定する。登録時と同様。
- **challenge**: リプレイ攻撃防止のためのランダムデータ。
- **allowCredentials**: 認証に使用可能な Credential ID のリスト。Discoverable Credential（詳細は 2.1.6 節を参照）を使用する場合、このリストは空の状態でリクエストされることが許容される。
- **userVerification**: 登録時と同様、ユーザー検証の要件を指定する。

2.1.4　Attestation

Web Authentication API における Attestation とは、認証器によって提供されるデータであり、RP はこの Attestation を登録セレモニーにおいて利用することで、認証器の製造元/モデルや認証器の特性、認証器が出力するデータ（Credential ID、公開鍵など）に対する署名といった追加情報を得ることができる [9]。RP は登録時に `attestation` というパラメータに `"none"` / `"indirect"` / `"direct"` / `"enterprise"` を指定することで、Attestation を利用するかどうかを指定することができる [9]。それぞれの値の意味は次の通りである [9]。

- **None（`attestation="none"`）**: RP が Attestation を要求しない。プライバシーおよび実装簡素化の観点から、一般消費者向けサービスではこの指定が採られやすい。
- **Indirect**: 検証可能な Attestation を希望するが、ブラウザ／ OS が取得方法を裁量で選ぶため、RP が期待する形で検証できない場合がある。
- **Direct**: 認証器が生成した attestation statement をそのまま受け取ることを希望する。
- **Enterprise**: 一意識別子を含む Attestation を要求する。特定認証器への紐付けを要する統制環境（企業内）を想定し、ブラウザ／ OS や認証器の設定が許可しない限り提供されない。

"none" は既定値であり、認証器のモデルを識別する情報が送信されることに伴うプライバシー上の懸念や、Attestation を利用することへ同意するかの UI がブラウザに表示されること、証明書チェーン検証等の実装負荷を避けたい場合に選択される。

認証器が Attestation を提供する場合[*4]、登録セレモニーの応答には、RP が保存する公開鍵等に加えて、認証器に関する情報（例：AAGUID[*5]）と、それらを含むデータに対して付与された署名付きデータ（attestation statement）が含まれる [22]。この署名は、Attestation 用の鍵により生成される場合があるほか、認証器が専用の鍵を持たない場合には、登録セレモニーで生成された秘密鍵（当該パスキーの秘密鍵）で署名される場合もある [9]。ただし本論文では鍵の種別や構成の詳細には立ち入らず、RP が認証器に関する署名付き情報を受け取って検証に利用できる点に着目する。

RP は、登録セレモニーでブラウザ/OS から返されるデータ（および必要に応じて証明書チェーン）を検証し、受け入れる認証器の方針（例：特定モデルに限定するか）に照らして登録を許可するかを判断できる [22]。また、例えば FIDO Metadata Service（MDS）[23] というデータベースを参照することで、AAGUID をキーとして当該モデルのメタデータ（モデル識別、対応アルゴリズム、認証・ステータス情報等）を取得して評価に利用できる [9]。

2.1.5　ユーザーインターフェースとユーザー体験

パスキー認証では、RP が認証器を直接操作するのではなく、ブラウザ／ OS が認証器アクセスを「仲介」し、ユーザーの同意に基づいてセレモニーを進行するという構造を取る。RP は API 呼び出しにより「登録・認証を要求する」ことはできるが、一部の UI（生体認証をユーザーに要求するダイアログ、PIN 入力、利用するパスキーを選択する画面等）の表示や遷移を直接指定・制御することはできない。例えば、登録・認証セレモニー中にブラウザ／ OS がユーザーを案内するための UI を表示する場合があり、ユーザーは一連の操作を拒否・キャンセルできる [9]。

RP 側が負う責務は、そのようなブラウザ／ OS によるダイアログの外側にある、RP の Web ページ側の画面において、用語の説明や案内文言、失敗時の再試行や代替手段の提示、プライバシーに関する説明等を補完することである。

■Conditional UI　Web Authentication API Level 3 では、認証手順の制御の拡張として *Conditional Mediation* が導入されている [10]。これは、`navigator.credentials.get()` 呼び出しにおいて `mediation: 'conditional'` を指定することで、入力フォーム（ユーザー名入力欄など）に、当該 RP に登録済みのパスキーの候補を表示できるようにする仕組みである [10]。図 3 は、この Conditional UI で表示される UI の例である。ユーザー名を入力するフィールドをクリックして選択すると、図のように利用できるパスキーが表示される。ここで

[*4] 同期パスキー等（詳細は 2.1.6 節を参照）では提供されないことがある [21]

[*5] 認証器の型を表す識別子。詳細は 2.1.6 節を参照

ログインしたいアカウントをクリックすると、認証セレモニーが開始される。

Conditional UI を用いる場合、RP は「ページ表示時」等に `mediation: 'conditional'` を指定した `navigator.credentials.get()` を開始し、ブラウザはその要求を待機状態として保持する。ユーザーがユーザー名入力欄にフォーカスしたタイミングで、ブラウザ／パスワードマネージャは当該 RP に紐づく登録済みのパスキーを候補として提示し、ユーザーは候補からアカウントを選択して（必要に応じてユーザー検証を伴い）シームレスに認証を完了できる [10, 24]。

注意点として、Conditional UI を利用するには、認証器が Discoverable Credential（詳細は 2.1.6 節を参照）を利用できることが前提となる [10]。

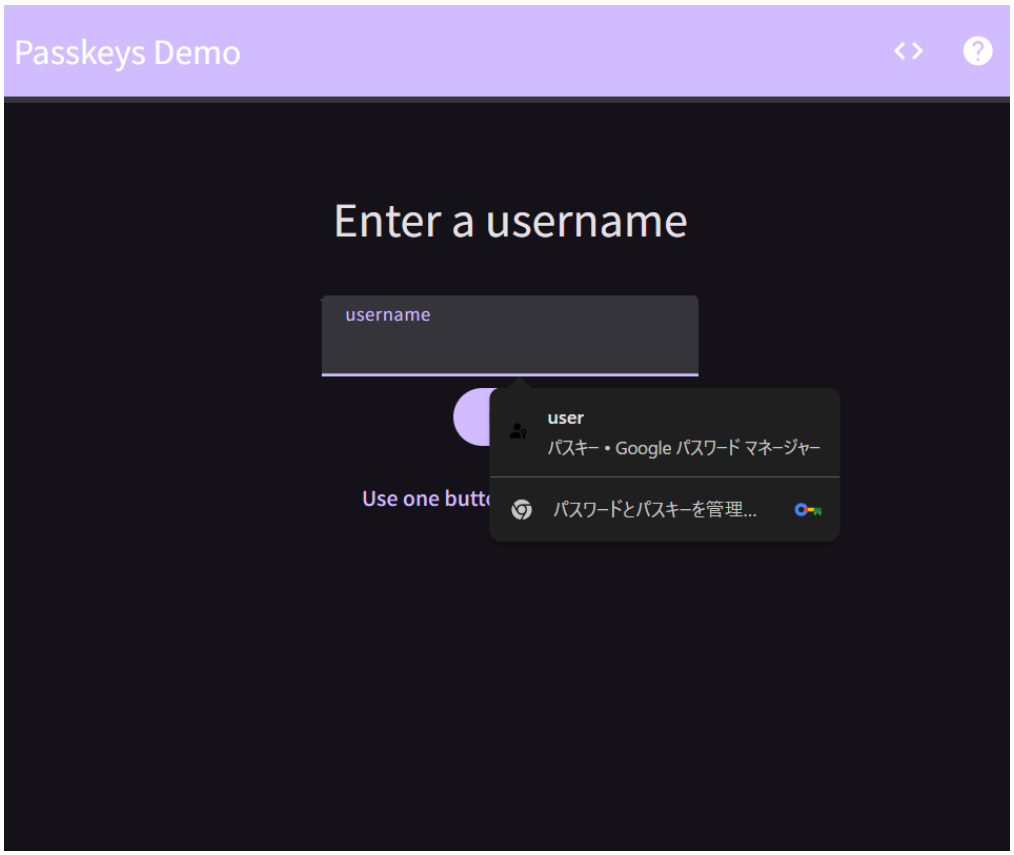

図 3　Conditional UI の例。Passkeys Demo [1] を利用。

2.1.6　用語と基本概念の補足

本節では、本章以降で用いる主要な用語について、仕様上の定義、略語、類似語との違い等をまとめる。

■プラットフォーム認証器とローミング認証器　認証器は、実装がブラウザ／ OS が動作するデバイス上に存在し、当該デバイス固有の手段でアクセスされ、通常は取り外しできない プラットフォーム認証器（*platform authenticator*） と、実装が当該デバイス外に存在し、USB/BLE/NFC

等の標準的な通信方式でアクセスされる ローミング認証器（*roaming authenticator*） に大別される [9]。

■パスキープロバイダ　Web Authentication API 仕様は**認証器**という言葉で暗号処理の主体を定義する一方で、近年では、パスキーの保存・同期・管理（および利用時の提示・選択 UI）を担う主体を指して「パスキープロバイダ」という語が用いられる [12]。この語は FIDO Alliance [12] や主要ベンダの開発者向け資料 [25] で用いられており、ブラウザや OS に組み込まれたもの（iCloud Keychain や Google Password Manager など）と、サードパーティのアプリやブラウザ拡張機能（1Password や Dashlane など）等を含む [12]。本論文では、認証器（Authenticator）を Web Authentication API 上の暗号処理の主体として扱い、パスキープロバイダは保存・同期・管理のレイヤとして区別して記述する。

■AAGUID　AAGUID（Authenticator Attestation GUID）は、認証器の型（製造元・モデル等）を表す識別子である [9]。RP は AAGUID を参照して認証器情報を表示したり、受け入れるかどうかの判断に用いることができるが、`attestation` を伴わずに得られる AAGUID 等の情報は真正性が保証されない場合がある点に留意する [22]。

■Credential ID　Credential ID は、認証器内のパスキーを識別するためのバイト列であり、RP はこれを用いて対応する公開鍵等を参照できる他、認証器もサーバーからこれを受け取り、自身が保存するパスキーを探すことがある [9, 10]。登録セレモニーで呼び出される `navigator.credentials.create()` のパラメータの一つである、`excludeCredentials` や、認証セレモニーで呼び出される `navigator.credentials.get()` のパラメータの一つである `allowCredentials` において用いられる [9, 10]。

■Discoverable Credential/Server-Side Credential　Discoverable Credential は、RP が `navigator.credentials.get()` の `allowCredentials` で Credential ID を提示しなくても、認証器側でパスキーを発見して利用できる（ユーザー名レス認証を可能にする）Public Key Credential Source である [9]。すなわち、認証セレモニーにおいて、`allowCredentials` が非空の場合にも利用可能であるパスキーのことを指す [9]。ここで、Public Key Credential Source とは、認証器が内部に保持するデータで、Credential ID・秘密鍵・RP ID を含む構造体を指す [9]。この場合、認証器は、RP ID のみから Public Key Credential Source を特定して署名を生成できる必要があるため、Public Key Credential Source は認証器またはパスキープロバイダ側に保存されることが要請される [9]。

これに対し Server-side Credential は、RP が `allowCredentials` で Credential ID を提示した場合にのみ利用できる Public Key Credential Source である [9]。したがって RP は、(i) ユーザーを先に特定した上で、(ii) 当該ユーザーに紐づく Credential ID を発見・管理し、認証セレモニーでユーザー側（Web アプリケーション）へ渡す責務を負う [9]。この方式では、Public Key Credential Source のクライアント側での保存は必須ではない。また仕様は、Credential ID

が「認証器のみが復号できる形で Public Key Credential Source を暗号化したもの」を取る場合があるとしており、この場合、認証器はほぼステートレスに近い形で動作できる一方、必要な状態は RP 側に保持される [9]。

■同期パスキー（Synced Passkey）とデバイス固定パスキー（Device-Bound Passkey）　パスキーは、ユーザーの複数デバイスで利用可能な「同期パスキー（Synced Passkey）」と、生成元の認証器（デバイス）から外部へ移動しない「デバイス固定パスキー（Device-Bound Passkey）」に大別される [26]。ただし、これらは Web Authentication API の仕様によって規定されるものではなく、認証器（パスキープロバイダ）の実装差として現れる概念である。RP が実装・運用上の判断のためにバックアップ（同期）の可能性を把握したい場合は、Web Authentication API Level 3 で規定されている、認証器から返されるデータに含まれるフラグ BE（Backup Eligible）および BS（Backup State）により、秘密鍵が他の認証器に移動（バックアップ）可能か、および現にバックアップされた状態かを知ることができる [10]。

同期パスキーは、パスキープロバイダ（例：iCloud キーチェーン等）により E2E で暗号化された形で複数デバイスへ同期され、ユーザーが新デバイスへ移行しても再登録コストを抑えられる一方、プロバイダのアカウントの回復・保護強度が可用性と安全性の重要な前提となる [12]。デバイス固定パスキーは、秘密鍵の移動性は低いが、「鍵が複製されない」前提を取りやすく、NIST の AAL3 のエクスポート不可要件に違反する同期パスキー [3] と異なり、高保証用途では好まれる場合がある。ただし、Web Authentication API 仕様はバックアップや同期のためのプロトコル自体は規定せず、例えば Web Authentication API Level 3 では、必要に応じて複数の認証器でパスキーを登録することやアカウントリカバリー手段を設計することが推奨されている [10]。

2.2　実装指針および関連資料の参照源

パスキーの実装においては、W3C による公式仕様や FIDO Alliance が提供するドキュメントだけでなく、OS ベンダーやセキュリティ専門家が提供するガイドライン、およびコミュニティによる技術情報が重要な判断材料となる。本研究では、第 4 章における観測項目（評価指標）の策定にあたり、以下の媒体によって提供される情報を参照する。ただし、ここでの「参照」とは参照基準を整理するためのものであり、個別サービスが必ず従うべきと断定する意図ではない。

2.2.1　標準化団体およびコミュニティによるガイドライン

FIDO Alliance および W3C を中心とした標準化団体は、仕様書に加えて実装者向けのガイドラインを提供している。

- **W3C 公式仕様書**: 前節で述べた Web Authentication API の仕様書である [9, 10]。API の正確な挙動やパラメータの定義、およびセキュリティモデルの基礎となる一次情報で

ある。

- **Passkey Central**: FIDO Alliance が運営する、パスキー導入のための総合情報サイトである [27]。UX/UI デザインガイドラインとして、アイコンの使用法やユーザーへの案内文言、アカウントリカバリーの設計指針などが整理されている [28, 29]。また、経営層向けの導入意義や、開発者向けの各 OS・ブラウザ等の実装リソースへのリンク集も提供されている。
- **passkeys.dev**: W3C WebAuthn Community Adoption Group [30] のメンバーおよび FIDO Alliance によって運営される、ベンダー中立な開発者向けリソースである [31]。各 OS やブラウザのパスキー対応状況や、デバッグツール、ライブラリ情報が集約されており、特定の実装に依存しない汎用的な知識が得られる。

2.2.2　ベンダーおよび開発者向けドキュメント

主要なブラウザベンダー、OS ベンダー、ならびに認証器メーカーは、自社のエコシステムに基づいた実装詳細や推奨事項を公開している。

- **Google Identity (web.dev / developers.google.com)**: Google は、Android および Chrome におけるパスキー実装の公式ドキュメントを提供している [32, 33]。特に Android の Credential Manager API の仕様や、パスワードからの移行ガイド、クロスデバイス認証における UI/UX の推奨デザインについて詳述されている。
- **MDN Web Docs**: Mozilla による Web 標準技術のリファレンスである [34]。Web Authentication API のブラウザ互換性、各メソッド（`create`、`get`）のパラメータ詳細、および Attestation Object の構造など、実装レベルの微細な仕様確認において参照される。
- **Yubico Developers**: セキュリティキー製造大手の Yubico は、RP 向けの実装ガイダンスを公開している [35]。特に Attestation を用いた認証器のモデル検証や、高セキュリティ要件を満たすためのメタデータの扱いについて専門的な指針を示している。

2.2.3　公的基準および書籍・技術ブログ

仕様書やベンダードキュメントを補完するものとして、以下の基準や解説書、専門家による技術記事を参照する。

- **NIST SP 800-63B**: 米国国立標準技術研究所（NIST）によるデジタル本人認証ガイドラインである [3]。本ガイドラインでは認証保証レベル（AAL）が定義されており、同期可能なパスキー（Synced Passkey）が AAL2（多要素認証相当）に適合するという解釈基準を提供している。
- **SimpleWebAuthn**: Web Authentication API の実装を支援する TypeScript ライブラリおよびそのドキュメントである [36]。複雑なバイナリパース処理や署名検証ロジックの実

装例として、多くの開発者に参照されている。

- **書籍「パスキーのすべて」**: FIDO 関係者らによって執筆された、パスキーに関する包括的な解説書である [21]。導入戦略から UI/UX 設計、セキュリティに関する誤解の解消まで、実装・運用上の論点が網羅されている。
- **技術コミュニティによる知見**: 仕様書には記載されていない実装上の落とし穴や、運用上の注意点が共有されることがある。例えば Brown [37] は、Discoverable Credential の無制限な作成が物理セキュリティキーのストレージ容量を圧迫するリスク（"Junk"化）を指摘している。また、Ackermann [38] は、一般のコンシューマー向けサービスにおいて Attestation の要求は不要であり、プライバシーリスクや実装の複雑さを避けるべきであると論じている。これらの指摘は、仕様書には明記されない運用上の考慮事項として参照する。

2.3　先行研究によって指摘された実装課題

前節では、パスキー実装において参照される指針やガイドラインの参照源を整理した。一方で、先行研究では、プロトコルの性質だけでは説明しきれない「実装・運用上の課題」や「ユーザーの理解・心理的障壁に関する課題」が複数報告されている。本節では、パスキー実装のセキュリティ上の課題と、ユーザーの理解や心理的障壁に関する課題を中心に先行研究を概観し、最後に本研究が扱う問題設定と未解決点を整理する。

2.3.1　実装におけるセキュリティ脆弱性の検証

パスキー認証は、高い暗号学的強度を持つユーザー認証や、フィッシングに対する耐性等をセキュリティ目標としている [39]。ただし先行研究では、RP やブラウザ／ OS の実装・運用に起因して、そのセキュリティ目標が十分に満たされない事例が報告されている。

Kuchhal ら [15] は、Tranco Top 1,000 に含まれる Web サイトのうち、パスキー認証を導入しているサービスを対象にセキュリティ対策の評価を行った。彼らは、RP がどの認証器を登録対象として受け入れるかの方針や、Attestation の検証状況を分析し、特にマルウェア耐性の欠如に着目している。その調査結果によると、96% の認証器がマルウェア耐性（TEE： Trusted Execution Environment を備え、マルウェアがデバイス内に存在していても秘密鍵が漏洩せず、暗号処理が不正に利用されない）を持たない状態で運用されており、多くのサイトがそのような認証器の登録を許可していることが報告されている。Kuchhal らは、ブラウザ/OS 等のユーザー側がマルウェアによって侵害された状況では、認証器が保持する秘密鍵がユーザーの意図しない署名生成に利用されたり、攻撃者による不正なアカウントアクセスや不正なパスキー登録に繋がる可能性があると整理している。これらの結果を踏まえ、同論文では RP は Attestation を要求し、信頼できる認証器のみを登録許可すべきであると論じている。

また、RP の実装上の不備による脆弱性も指摘されている。Chen [16] は、Web Authentication

API の実装に対する脆弱性テストを行い、特に User Verification（ユーザー検証）フラグや Origin（オリジン）の検証漏れが商用サイトで実際に発生していることを報告している。UserVerification の検証が不適切だと、攻撃者が物理的に被害者の認証器を手に入れるだけでアカウントにアクセスできるようになる。また、Origin を検証しないことで、攻撃者は不正にアカウントにアクセスできるようになる恐れがある。Chen は Burp Suite を用いたブラックボックステスト手法を提案し、既存の自動診断ツールでは検出が困難であった実装上の不備を明らかにしている。これらの研究は、プロトコルが安全であっても、実装上の不備によりセキュリティが損なわれる可能性を実証している。

2.3.2　ユーザーの理解と心理的障壁

パスキーの普及においては、技術的な安全性だけでなく、ユーザーがその仕組みを正しく理解し、安心して利用できるかどうかが重要となる。

Lassak ら [17] は、パスキー認証に対するユーザーの障壁を調査した（2024 年発表、調査期間は 2020-2021 年頃）。彼らの調査によると、67% のユーザーが「生体情報が Web サイトに送信される」という誤解を抱いていることが明らかになった。「生体情報は Web サイトや第三者と共有されない」という通知を提示することで、82% のユーザーが「登録は安全である」と回答するようになったものの、「生体情報や PIN がデバイス内にのみ保存される」という仕組みを正しく理解できたユーザーは 55% にとどまっている。この結果は、サービス側の画面上の説明や用語の選択が、ユーザーの理解と心理的障壁に影響することを示唆している。

2.3.3　本研究の立ち位置

以上の先行研究は、プロトコルの安全性だけでなく、RP 側の受け入れ方針や検証・運用、ならびにユーザー向けの説明や運用設計が、安全性と利用体験の双方に影響することを示している。以上の先行研究により、個別のセキュリティ上の実装不備やユーザーの誤解など、パスキー導入における複数の課題が指摘されてきた。しかし、これらは特定の技術的脆弱性に焦点を当てたものや、数年前の調査に基づくものが多い。現在、主要ベンダによるパスキー対応が急速に進む中で、第 2.2 節で挙げた仕様・ガイドライン等が示す設計上の論点に照らし、主要な Web サービスが各論点に対してどのような設計・実装上の判断をしているかを網羅的に調査・比較した研究は不足している。本研究はこの不足を補うため、主要な Web サービスの設計・実装判断を実証的に調査し、設計上の示唆を提供する。

3　設計上の論点と評価観点

第 2 章では、パスキー認証の登場主体と役割分担、登録・認証セレモニーの流れ、および RP が実装・設計する範囲を整理した（図 1、2）。

本章では、この整理を前提として、パスキー認証の実装に際し RP が判断を要する点について、Web Authentication API 仕様、関連ガイドライン、および先行研究に基づき整理する。具体的には、各項目について、推奨とされている実装がどのようなものか、また推奨と異なる判断をした場合にどのような影響が生じる可能性があるかを示す。本研究では、RP の責務の範囲内にあり外部から観測可能であることを条件として、(i) Web Authentication API のパラメータ設定と、(ii) 周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー等）の 2 領域を扱う。各節では、まず対象となる場面（登録／認証／管理／アカウントリカバリー等）と RP の判断点を示し、続いて根拠と、推奨と異なる判断をした場合に生じる可能性のある影響（誰がどの場面で何に困るか）を述べる。第 4 章では、本章での整理をもとに調査項目を定め、データ収集手順を設計する。

3.1　Web Authentication API パラメータ設定

Web Authentication API では、登録セレモニーで `navigator.credentials.create()`、認証セレモニーで `navigator.credentials.get()` をブラウザ上の JavaScript から呼び出し、それぞれに各種オプション（`PublicKeyCredentialCreationOptions`、`PublicKeyCredentialRequestOptions`）を渡すことで処理を開始する [9]。RP が指定するオプション値は、利用できる認証器の範囲、ユーザーに求めるユーザー検証の強さ、ログイン時のアカウント選択 UI、ならびに本章で後述するパスキーの管理画面といった、セキュリティとユーザビリティの双方に影響する。本節では、主要なパラメータについて「何を制御する項目か」「推奨が置かれる理由」「逸脱した場合の影響」を整理し、第 5 章での観測項目に接続する。第 5 章では、本節で述べた主要パラメータの設定値を観測し、実サービスにおける実装状況を整理する。

3.1.1　navigator.credentials.create() における考慮事項

登録セレモニーでは、新しい鍵ペアの作成と公開鍵の登録を行う。RP は登録時に渡すオプションにより、利用可能な認証器の範囲やユーザー検証の扱い等を指定し、登録の成立性とユーザー体験に影響する。

■Attestation の扱い　第 2 章で述べた通り、RP は `attestation` パラメータを通じて、認証器の構成証明（製造元やモデル等を示す情報）を要求可能である [9]。一方で、一般消費者向けの広く公開されるサービスでは、`attestation` を要求しない（`none`）設計が推奨され

ることが多い [21, 35, 38]。NIST SP 800-63B でも、広く一般公開されるアプリケーションでは attestation の検証可否のみを理由として認証器の利用を妨げるべきでないとされている [3]。

attestation を要求する設計では、許可リストに含まれない認証器や、証明書を持たない認証器を使用しているユーザーが排除されるリスクがある。その結果、ユーザーがパスキー認証を利用できず、より脆弱でフィッシング耐性を持たない代替手段（例：パスワード認証）へ誘導され、全体の安全性を低下させる可能性がある。ただし、要求しても常に attestation が得られるとは限らない点にも留意が必要である [21, 22]。

なお、attestation: "none" を指定した場合の AAGUID（Authenticator Attestation GUID）の取り扱いについては、WebAuthn 仕様のバージョン間で重要な変更が行われている。

Web Authentication API Level 2（2021 年 4 月勧告）[9] においては、プライバシー保護の観点から、attestation が "none" の場合、ブラウザは AAGUID を 16 バイトのゼロ（0x00）で上書きし、attestationStatement を "none" に差し替えることが必須とされていた [9]。したがって、このバージョンに準拠する環境では、attestation を要求しない限り、RP は認証器の種類を特定するために使われる AAGUID を取得できない場合があった。

Web Authentication API Level 3 [10] ではこの挙動が変更された。Level 3 では、attestation: "none" を指定した場合であっても、AAGUID を改変せずに保持できるようアルゴリズムが修正されている [10]。ただし、実際にどの情報が得られるかはブラウザ／ OS の実装にも依存し得る。取得された AAGUID は暗号学的な構成証明を持たない自己申告の値ではあるが、後述するパスキーの管理画面において「iCloud Keychain」や「YubiKey」といった認証器の名称を表示し、ユーザビリティを向上させるための参考情報として利用する分には有用である。

■User Verification　ユーザー検証の要求を設定する userVerification については、特別なセキュリティ要件がない限り preferred を設定することが推奨されている [21, 31]。

required を設定した場合、指紋の読み取り不良や PIN 忘れなどでローカル認証に失敗すると、登録セレモニー自体がエラーとなり中断されてしまうリスクがある [21]。preferred であれば、ローカルユーザー検証が実行できない場合でも、エラーにならない [21]。discouraged という、ユーザー検証を実施しないことを期待する設定もあるが、認証器の実装によってはユーザー検証が求められる場合がある点 [21] に留意が必要である。

ユーザビリティだけでなくセキュリティの観点からも、多くの一般向けサービスにおいて required は必ずしも必須ではないという議論がある。SimpleWebAuthn のドキュメントが指摘するように、パスキー認証はフィッシング耐性を有している時点で、従来の「ユーザー名＋パスワード＋二要素認証」よりも高い保護レベルを実現している [36]。したがって、ユーザビリティを阻害してまで厳密な本人確認（UV）を強制する必要性は低いと考えられている。

■AuthenticatorAttachment　第 2 章で述べた認証器の接続形態を指定する `authenticatorAttachment` については、初回登録時においては、`platform` を指定し、デバイス内蔵の認証器（Touch ID, Face ID, Windows Hello 等）への登録を促すことが望ましいとされている [21]。そうすることで、USB セキュリティキーや NFC を含めた外部デバイスのローミング認証器が選択肢として表示されず、ユーザーの混乱を避けることができる。しかし、パスキー認証をあくまで「複数の認証方式の一つ」あるいは「二要素認証の一環」として位置づける場合、あえて指定を行わず、セキュリティキーなどの幅広い認証器を受け入れる方針も有効である。

また、`authenticatorAttachment` は「同期パスキーかデバイス固定パスキーか」[*6]を直接制御するものではない点に注意が必要である。プラットフォーム認証器であっても、常にデバイス間同期されるとは限らない。そのため、同期可否が重要であれば、登録応答で得られる `backupEligible` ／ `backupState` 等の情報を保持し、同期されないパスキーのみが登録された場合に追加登録を促す等、UI/UX 側での補完が必要となる [10]。

■ExcludeCredentials（重複登録の防止）　`excludeCredentials` は、登録済みの Credential ID のリストをブラウザ/OS 側に伝え、同一の認証器から同じアカウントに複数のパスキーを登録することを防ぐための仕組みである [9]。特に、2 つ目以降のパスキーを追加登録する際には、既に登録済みの Credential ID を必ず設定することが推奨される [21]。

この設定を行わずに同じ認証器で登録を試みた場合、ブラウザ/OS 側で特にエラーが発生せず重複登録が行われてしまう場合がある。また、Web Authentication API Level 3 では、Discoverable Credential の場合、`user.id`（user handle）が同一 `rp.id` 内の既存 Discoverable Credential を上書きするために用いられることがある [10]。このため、`excludeCredentials` を空のまま追加登録を許容すると、重複登録に留まらず、既存のパスキーが意図せず置き換わる等の状態変化が生じる恐れがある。その結果、RP 側で参照すべき公開鍵が意図せず変化し、以後の認証が失敗する可能性がある。重複登録が直ちにアカウント侵害を引き起こすわけではないものの、後から「どのパスキーを削除すべきか」が判別しづらくなり、管理画面での誤操作や運用上の混乱につながる可能性がある。

■PubKeyCredParams（暗号アルゴリズム）　`pubKeyCredParams` には、許容する公開鍵暗号アルゴリズムを指定する。Web Authentication API Level 3 の仕様に基づき、少なくとも以下の 3 つを含めることが推奨される [10]。

- **-8**: Ed25519
- **-7**: ES256
- **-257**: RS256

空配列を指定した場合、ユーザーエージェントが内部的に ES256(-7) と RS256(-257) を補完

[*6] 用語の意味は 2.1.6 節参照

する [9]。

一部の報告では、RS256 および ES256 がサポートされていれば実用上は十分であるとの見解もあるが [40]、法規制や特定のハードウェア制約がない限り、署名アルゴリズムに対応していないことが理由で認証器を除外することがないよう、上記 3 つのアルゴリズムはすべて対応しておくことが望ましい。

■ResidentKey（Discoverable Credential） Discoverable Credential は、ユーザーがユーザー名を入力せずにアカウントを選択できるログイン体験を実現するための前提となる。特に、Conditional UI [*7]は Discoverable Credential を前提とする。

Discoverable Credential を要求するには、`authenticatorSelection` において `requireResidentKey` を `true`、かつ `residentKey` を `required` に設定する必要がある。

一方で、Discoverable Credential を必須化すると、当該機能を提供しない認証器を排除することになり得る。そのため、サービスのユーザー環境やログイン手順（ユーザー名入力を前提とするか等）に応じて、必須化する範囲や導入手順を検討する必要がある。また、物理セキュリティキー等のストレージ容量が限られる認証器では、Discoverable Credential を無制限に作成すると容量枯渇につながるという指摘もある [37]。

注意点として、登録されたパスキーが実際に Discoverable Credential であるか否かを Credential ID のみから RP が判断することは、拡張機能なしには不可能である。Web Authentication API の拡張機能である `credProps` (`credProps: true`) を利用すると、登録されたパスキーが RP で Discoverable かどうかのフラグを受け取ることが可能となる [10]。

3.1.2 navigator.credentials.get() における考慮事項

認証セレモニーでは、登録済み公開鍵に対応する秘密鍵による署名を生成し、RP が署名を検証してログインを成立させる。RP は認証時オプションにより、利用するパスキーの特定やユーザー検証の扱いを指定し、誤選択や再認証[*8]時の混乱を防ぐ必要がある。

■AllowCredentials `allowCredentials` は、認証に使用できる Credential ID のリストを指定する [9]。RP は、ログインを試みているユーザーの登録済みパスキーの Credential ID をこのリストに含め、認証器はこのリストの中に一致するパスキーを探し出し、ユーザーに使用を促す [9]。空で指定された場合、認証器の中に保存されている Discoverable Credential が利用される。

ログイン手順において、パスキー認証の開始前にユーザー ID を入力させる場合（Server-Side Credential の場合これが必須）や、Conditional UI を使用しない場合は、`allowCredentials` に対象ユーザーの登録済み Credential ID を設定する必要がある [9]。これを適切に設定しない

[*7] 2.1.5 節参照

[*8] 既存セッションが存在する場合であっても、改めて認証を要求することを指す

場合、ユーザーは認証器側のアカウント選択画面で、意図しないアカウント（あるいは無関係なアカウント）を選択してしまう可能性がある。

特に、決済処理や重要情報の変更直前に行う「再認証」では注意が必要である。この場合、`allowCredentials` を空の配列あるいは未指定にしてしまうと、ユーザーが複数のアカウントを保持している際、意図せず別のアカウントで認証を通すこととなり、想定外の状態遷移がなされる可能性がある。再認証時には、必ず現在ログイン中のユーザーに紐付いた Credential ID を指定しなくてはならない [21,41]。

■User Verification　認証時においても、登録時と同様の理由により、基本的には `preferred` が推奨される。ユーザー体験を阻害せず、かつ可能な限り高いセキュリティレベルを維持するための現実的な解であるといえる。

3.2　UI/UX（登録・認証手順）の設計

パスキー認証の導入は、API の呼び出しだけで完結するものではなく、ユーザーに対して「何が起きているか」を伝える UI/UX を伴う。技術的な実装が正しく行われていたとしても、UI/UX の設計が不適切であれば、ユーザーが登録やログインを躊躇したり、誤解に基づく不安を抱いたりする恐れがある。本節では、FIDO Alliance や Google 等のガイドライン、および先行研究に基づき、登録・認証・管理の各場面での設計上の論点を整理する。2.1.5 節で述べた通り、ブラウザ／ OS は認証器の選択やユーザー検証等の UI を提示してセレモニーを進行するが、RP はそれらの UI を直接制御できない（図 1、2）。このため RP は、ブラウザ／ OS の UI の前後で、ページ内の説明、失敗時の案内、代替手段提示等を補完し、ユーザーが操作の意味を理解できるように設計する必要がある。

第 5 章では、本節の整理を基に、登録手順、画面文言、登録前後のメッセージングと通知、ログインフロー（Conditional UI 等）、および管理画面の実装状況を観測する。

3.2.1　アカウント作成とパスキー登録への誘導

Passkey Central のガイドライン [28] では、アカウントの作成、アカウントリカバリー、設定といった主要な操作手順の中に、自然な形でパスキー登録を促す説明文言やボタン等を組み込むことが推奨されている。特に、パスキーを「二要素認証のオプション」といった深い階層に隠すのではなく、アカウント設定の主要な選択肢として提示することが重要とされている。深い階層に隠れている場合、ユーザーは「設定できること」自体に気づきにくく、結果として登録が進まず、パスワード等の代替手段に依存し続ける可能性がある。

また、登録手順を開始する直前に、ページ上で登録の目的（何が変わるか、何が楽になるか）を短く説明した上で手続きを開始させることが推奨されている [28]。

第 5 章では、(i) アカウント作成直後・初回ログイン後・設定画面等における登録導入点の有無、(ii) パスキーが主要な選択肢として提示されているか、(iii) 登録前後の説明文言、を観測

する。

3.2.2　用語の統一と誤解の防止

パスキーは、多くのユーザーにとって新しい概念であるため、ユーザーが何をすべきかを明確にすることが重要であるとされている [28]。ユーザーにとって分かりやすく、誤解を招かないような言葉遣いを使い、理解を促すことが求められる。

■「パスキー」の呼称　パスキー認証は、「FIDO2 認証」「WebAuthn」といった様々な用語で呼ばれることがある。しかしながら、このような不統一な用語はユーザーの混乱を招く恐れがある。FIDO Alliance の資料では、一般消費者向けの用語として「FIDO クレデンシャル」や「セキュリティキー」ではなく、「パスキー（Passkeys）」という名称が広く用いられている [12]。また、主要なパスキープロバイダのエコシステムを提供する Google や Apple の開発者向け資料でも、「passkeys / パスキー」という語が採用されている [32,42]。このように実際のエコシステムで「パスキー」が中心的な呼称として用いられていることを踏まえると、ユーザー向けの説明文言では、この呼称に統一して示すことが理解の助けになると考えられる。

FIDO Alliance のホワイトペーパー [26] では、同期されるパスキーのことを「同期パスキー（Synced Passkey）」、単一のデバイスに紐づくパスキーのことを「デバイス固定パスキー（Device-Bound Passkey）」と呼び分けている。サービスの中でそれらを呼び分ける必要があれば、このような表現の利用も選択肢に上がる。

■「生体認証」表記　パスキー認証を説明する際、「生体認証」という言葉を使う際には注意が必要である。認証器（Authenticator）の種類によっては、生体認証ではなく PIN コードやパターン入力が用いられる場合があり、またそもそも生体認証を利用する場合も 2 章で述べた通り RP はその結果のフラグを得るのみである。また、「生体認証」という表現は、ユーザーに対して「自分の指紋や顔データがサーバーに送信されるのではないか」というプライバシー上の懸念を抱かせる要因となる可能性がある。Google Developers のガイドラインでも、生体認証はあくまでデバイスのロック解除（ローカル認証）に用いられる手段に過ぎないことが述べられている [32]。

■操作を表す動詞　パスキーに関する操作を表す動詞も、統一することが望ましい [43]。

ユーザー向けの UI 文言としては、Registration（登録）を「作成（Create）」と表現することが推奨されている[*9] [43]。パスキーはパスワードと異なり、一つのサービスの一つのアカウントに複数のパスキーを設定できる。そのため、「作成」という言葉を用いることが、一意の新しいパスキーを生成するプロセスを適切に説明することにつながるとされている [43]。

また、公開鍵を RP から削除する際は、「削除（Delete）」という言葉を使うことが推奨されて

[*9] ただし、仕様・実装上の手続きとしては、「登録セレモニー」、つまり「登録」と呼ぶのが正式な説明であるため、本論文では「登録」という表現を用いる

いる [43]。Web Authentication API の仕様上では、"deregister" や "decommissioning" といった語が用いられているが、こちらも仕様上の表現よりもユーザー向けの言葉として伝わりやすい言葉遣いとして推奨されているものと考えられる。

これらの用語はユーザー向けの表現であり、技術的な状態遷移（例：「作成」は鍵ペア生成だけでなくアカウントへの紐付けも伴う。「削除」と表示されても、当該デバイス側のパスキーが同時に消えるとは限らない。）を一意に保証するものではない。したがって、「どこから何が消えるのか（RP 側の登録情報か、デバイス側のパスキーか）」を補足し、誤解に基づく操作やロックアウトを避けることが求められる。

3.2.3　登録セレモニーのセキュリティと UX

本節は、登録セレモニーの前後に RP が補完すべき手続き（再認証、メッセージング、通知）を扱い、不正な追加登録や誤解・不安を抑える観点を整理する。

■登録前の再認証　パスキーは一度登録されれば強固な認証を可能にするが、その登録プロセス自体が攻撃の標的となるリスクがある。したがって、パスキーの登録を許可する直前には、既にログイン中であっても、パスワードだけでなく E-mail や SMS によるワンタイムパスワード等、その時点で利用可能な強固な手段により再認証を行うべきである [44]。適切な再認証なしに登録を許可した場合、セッションハイジャック攻撃によって攻撃者のデバイス（公開鍵）が正規のアカウントに登録されてしまう恐れがある。

特に、既存のパスキーが存在する状態で新たなパスキーを追加登録する際には注意が必要である。可能であれば既存のパスキー認証を求めるべきであるが、異なるデバイス/OS など、既存のパスキーが利用できない場合もある。そのような場合でも、可能な限り高いレベルの認証を要求し、攻撃者による不正な追加を防ぐ設計が求められる。

■登録時のメッセージングとハンドシェイク　OS が提供するネイティブの登録ダイアログが表示される前後に、アプリケーション側で事前の説明と完了後のフィードバックを行う「ハンドシェイク」のプロセスが重要である [28]。具体的には、OS のダイアログが表示される直前に「パスキーを作成します」といった明確なプロンプトを表示し、登録完了後には「作成が完了しました」というフィードバックを行う。これにより、ユーザーはアプリケーションと OS が連携して動作していることを認識し、安心感を得ることができる [28]。

また、ユーザーを安心させるため、「生体認証データがユーザーのデバイスから外部に送信されることはない」というメッセージを伝えることも重要である [32]。実際、Lassak らの調査 [17] によれば、「生体情報はサーバーに送信されず、デバイス内に安全に保存される」という旨の注意書きを表示することが、ユーザーの心理的なハードルを下げる上で効果的であることが示されている。

■登録完了後の通知　パスキーの登録は、パスワードの変更と同様に、アカウントのセキュリティに関わる重要な変更である。したがって、登録が完了した際には、アカウントに紐付けられたメールアドレス等に対して通知を送ることが推奨されている [44]。これにより、万が一攻撃者がユーザーの許可なくパスキーを登録したとしても、ユーザーが即座に異変に気づき、対処することが可能となる。

3.2.4　ログイン手順の設計

本節では、Conditional UI について整理する。

■Conditional UI と自動入力　Conditional UI[*10]を利用すると、ユーザー ID やメールアドレスなどの入力欄を起点として利用するパスキーの候補が提示されるため、パスワードのオートフィルに近い操作感を提供しやすく、特にパスワード認証からパスキー認証へ認証方式を移行途中のサービスでの活用が期待される。Passkey Central では、専用の「パスキーでログイン」ボタンを起点とする手順よりも、自動入力を起点とする手順の方がログイン成功率が高いことが示されている [29]。ただし、Conditional UI は Discoverable Credential を前提とする点に留意が必要である。

3.2.5　パスキー管理機能の設計

本節は、登録後のライフサイクル（識別、編集、削除等）をユーザーが扱えるようにするための管理画面設計を扱う。

ユーザーが自身の認証器を効果的に管理できるよう、一覧表示と詳細情報の提供が重要である。特に、複数のパスキーを登録している場合、それらを識別できなければ、意図しない削除などのセキュリティ事故につながる恐れがある。

上述した管理画面設計の一例を、図 4 に示す [2]。

[*10] 詳細は 2.1.5 節を参照

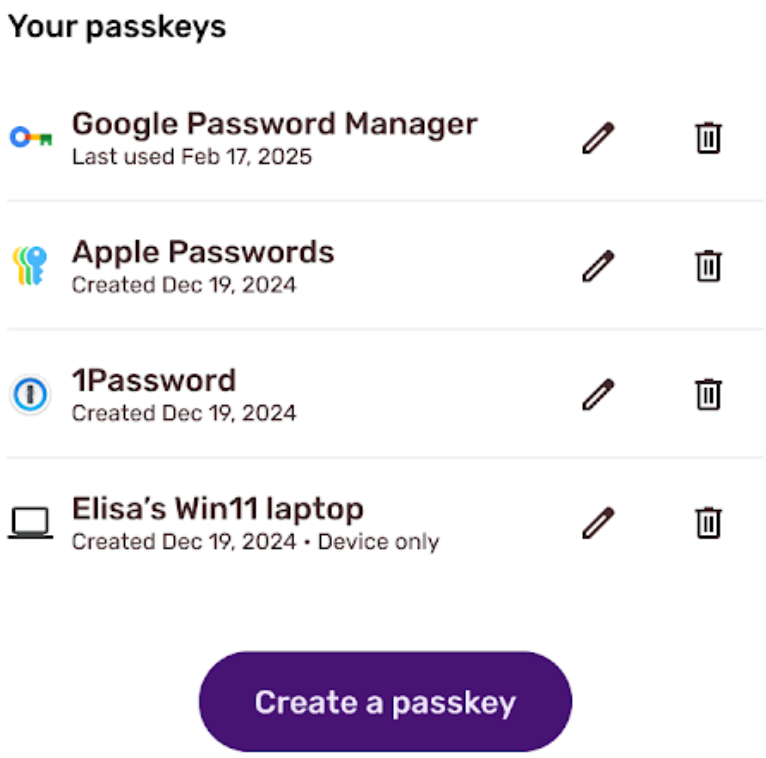

図 4　パスキー管理画面の例 [2]。

■パスキー名の表示と編集機能　管理画面においては、可能な限り認証器やパスキープロバイダの種類を特定し、登録されたパスキーをユーザーが識別できるように、適切な名称を表示することが推奨されている [2, 21]。そうすることで、ユーザーはどこに自分のパスキーがあるのか、どれを削除すればいいかといった判断が容易になる。

認証器の種類の特定には、`AAGUID`[*11]が利用できる [45]。`attestation` がない場合は `AAGUID` の真正性は保証されないが、管理画面で名称を表示する目的で参照することは有用である。ただし、`attestation: "none"` の場合に AAGUID が取得できるかは、仕様バージョンや実装に依存する（3.1.1 節）。`AAGUID` と表示名の対応は、例えばコミュニティ主導の一覧[*12]を参照して実現できる [45]。これを利用すると、「iCloud Keychain」や「Google Password Manager」といった名前を AAGUID を元に特定することができる。

AAGUID が利用できない、あるいは認証器の種類を特定できない場合は、OS やブラウザ名を HTTP リクエストのヘッダーから推定して「Chrome on Windows」のようなデフォルト名を設定するフォールバック策が必要である。

さらに、ユーザー自身による「名前の編集」機能の実装も推奨される [2, 21]。自動生成され

[*11] 詳細は 2.1.6 節を参照
[*12] passkey-authenticator-aaguids: https://github.com/passkeydeveloper/passkey-authenticator-aaguids

る名称（例：「Chrome on Windows」）のみに頼ると、同一環境で複数のパスキーを登録した場合に区別がつかなくなり、誤って有効なパスキーを削除してしまうリスクが高まるためである。

■メタデータ（登録日・最終使用日）の可視化　各パスキーには、「登録日時」および「最終使用日時」を表示することが推奨されている。これらの情報は、ユーザーが管理するパスキーの特定に利用される。[2,21]。また、最後に利用された際のブラウザや OS 等も提供することで、ユーザーは不審なログインを特定することができる [2,21]。

■同期ステータスの表示とバックアップへの配慮　そのパスキーがデバイス間で「同期される」ものか、特定のデバイスに「固定される（非同期）」ものかを明示することも重要である [2,21]。RP 側では、登録時のレスポンス（`backupEligible` や `backupState` 等）を確認するまで同期の可否は判断できないが、この情報を保持し、管理画面で「同期済み」や「デバイス固有」といったラベルを表示することで、ユーザーが自分が登録したパスキーがどのようなものかの理解に繋がる。特に、バックアップ可能な（同期される）パスキーが一つも登録されていない場合には、紛失リスクを警告し、同期パスキーの追加を促すなどの工夫が求められる。

■削除機能と確認ダイアログ　パスキーの管理画面では、パスキーの削除機能の実装も求められている [2,21]。ただし、削除機能を活かすためには、そもそもとして適切にパスキー名を設定し、メタデータ（登録日・最終使用日）の可視化がなされている必要がある。

3.3　多様な認証環境への対応

本節では、仮想認証器（ブラウザの開発者ツール等が提供する、WebAuthn 認証器のテスト用エミュレータ。例：Chrome DevTools の WebAuthn Emulator 機能 [46]）を題材として、RP が「どの認証環境・認証器を受け入れるか」をどのように設計するかという論点を整理する。これは、第 2 章で整理した RP の設計範囲のうち、受け入れる認証環境（認証器）の範囲を定めるという設計判断に関係する。受け入れ範囲を狭めることは、不正利用のリスクを抑える意図で選択される場合がある一方で、特定の環境ではパスキー登録やパスキー認証によるログインができない、といった形でユーザーに不利益が生じる場合がある。したがって、受け入れ範囲をどのように定めるかは、その根拠と想定される影響を明確にした上で設計判断を行う必要がある。

Kuchhal らの研究 [15] では、実際の FIDO2 導入事例におけるセキュリティ態勢を評価し、テスト用途で利用される仮想認証器の無制限な追加許可がセキュリティリスクになり得ると指摘している。彼らの主張では、このような認証器は、秘密鍵を含むクレデンシャルを平文で保存し、それらをワンクリックでエクスポート可能にしている。また、マルウェアを介した攻撃シナリオで悪用されると指摘している。攻撃者は、悪意のある仮想認証器をユーザーに登録させ、平文で保存されている FIDO2 のクレデンシャルを盗み出し、盗んだクレデンシャルを持つ仮想認証器を複製して、リモートからユーザーアカウントを侵害することが可能になる恐れがあると主張する。

この観点からは、attestation を利用して特定の認定されたハードウェアのみを許可リストに加え、それ以外を排除する運用は選択肢となり得る。特に、金融機関や企業の内部システム等の高いリスク管理要件がある領域では、この制限が有効な場合がある。

しかし、一般消費者向けサービスにおけるパスキーの普及という文脈においては、前提条件が大きく異なる。もし RP が「セキュリティ上の懸念」を理由に仮想認証器や未知の AAGUID を持つデバイスをブロックすれば、ユーザーは手元のデバイスでサービスを利用できなくなり、結果としてより脆弱なパスワード認証への回帰を招くことになりかねない。

したがって、広く公開される一般消費者向けサービスにおいては、仮想認証器を含む多様な認証環境を一律に制限する設計は慎重に検討すべきである。制限は安全性向上を意図したものである場合がある一方で、正当なユーザーの利用可能性を損ない、結果としてより脆弱な代替手段への回帰を招く恐れがある。「完璧なセキュリティ」を追求して利用のハードルを上げるよりも、「十分なセキュリティ」を維持しつつ「高い可用性」を提供することで、エコシステム全体としての安全性を底上げするアプローチが求められている。

第 5 章では、この整理を基に、仮想認証器を用いた登録・認証の可否を観測する。

3.4　アカウントリカバリー戦略

本章の最後に、パスキー認証の運用における主要な課題の一つであるアカウントリカバリーについて論じる。現状、すべての環境やユーザーが常にパスキーを利用できるわけではない。さらに、パスキーを登録済みのユーザーであっても、別デバイスからは当該パスキーを利用できない場合がある。したがって、ロックアウトを回避するためには、一定のフォールバック（アカウントリカバリー）手順を設計として用意しておく必要がある。本節では、アカウントリカバリー手段の選択に関わる前提と、公開文書・先行研究で言及される論点を整理し、第 5 章の観測項目へ接続する。

結論から言えば、アカウントリカバリーに万能な解は存在せず、サービスの想定リスクと運用制約に応じた設計が求められる。本節では、その設計判断を行うための前提（複雑性）を整理したうえで、広く公開される一般消費者向け Web サービスを主対象として、公開文書・先行研究で言及される要点をアカウントリカバリー経路の下限強度と冗長性の観点から整理する。

3.4.1　リカバリー問題の複雑性

本節では、アカウントリカバリー設計が複雑化する要因を整理し、後述するアカウントリカバリー手段の比較と評価の前提とする。

Kunke らの研究 [47] は、FIDO2 ベースのパスワードレス認証におけるアカウントリカバリーメカニズムを体系的に評価し、多くの既存手法がセキュリティ上のトレードオフやユーザビリティ上の欠陥を抱えていることを指摘している。とりわけ、パスキーの紛失・移行シナリオは以下の要因により複雑化する。

- **同期の有無**: クラウド同期されるパスキー（Synced Passkey）であれば、デバイス紛失時もクラウド経由で復元が可能である。一方、デバイス固定のパスキー（Device-bound Passkey）の場合、物理的な紛失はアクセス権の喪失を意味する場合がある。
- **複数デバイスでの登録状況**: 複数のデバイス（「スマートフォンと PC」 等）の各認証器（またはパスキープロバイダ）でパスキーを登録し、同一サービスの同一アカウントに複数のパスキーが紐付けられている場合、一方を紛失しても他方でログインできる可能性がある。しかし、単一の認証器のみに依存している場合、アカウントリカバリーは困難となる。
- **失効モデルの未標準化**: FIDO の仕様上、パスキーの失効（Revocation）に関する標準的な仕組みが確立されていないため、紛失した認証器を無効化するプロセスは RP の実装に委ねられている。

3.4.2　アカウントリカバリー設計

アカウントリカバリーの設計は、サービスのリスク許容度とパスワードレス化の程度によって異なる [21]。もっとも単純なアカウントリカバリー手段はパスワード認証によるログインである。しかし、これはパスワード認証の脆弱性を抱え続けることになり、パスワードレス認証の導入によるセキュリティ向上効果を相殺する可能性がある [47]。

Kunke ら [47] は、FIDO2 ベースのパスワードレス認証における 12 種類のアカウントリカバリーメカニズムを、ユーザビリティ・デプロイ容易性・セキュリティの 3 観点で比較評価し、秘密の質問やパスワード認証でのリカバリーは総合的に最も低い評価であり、パスワードレス環境では避けるべきだと結論付けている。一方で、FIDO アライアンスが推奨する「複数のパスキーを登録しておく」アプローチは、スケーラビリティの課題はあるものの、全体として望ましいバランスを示しているとした。また、NIST SP 800-63B [3] はアカウントリカバリーを「保存済み／発行済みリカバリコード」「リカバリ連絡先」「再本人確認」といった複数のクラスに整理し、認証保証レベルに応じてこれらを組み合わせることを要求している。えーじら [21] は、どのサービスにも共通する単一の正解は存在しないこと、そして「アカウントリカバリー手段のセキュリティレベルが、アカウント全体のセキュリティレベルを決めてしまう」ことを強調している。

以上を踏まえると、設計上の要点は次の二つに集約できる。第一に、アカウントリカバリー経路が全体の最弱点にならないよう、アカウントリカバリー手段の下限強度（保証レベル）を確保することである。第二に、同期・複数デバイス・補助手段といった複数チャネルにより冗長性を確保し、ロックアウトリスクを現実的に管理することである。本研究では主に広く公開される一般消費者向け Web サービスを想定し、公開文書・先行研究で言及される設計上の要点を次のように整理する。

- 同期パスキー、または複数のパスキー登録を前提にする。同期可能なパスキーを登録さ

せるか、または複数のパスキーの登録をユーザーに促す。これにより、「紛失したらパスワード認証にフォールバックする」という設計を避けつつ、実用的な冗長性を確保できる。

- リカバリコードを補完的に用意する。パスキー登録時に高エントロピーなランダム値から生成したリカバリコードを発行し、ユーザーに安全なオフライン保管を求める [21]。NIST の Saved Recovery Codes [3] に倣い、コードは一回限りの利用とし、有効期限と利用時通知（メール等）を組み合わせることで、不正利用の検知可能性を高める。ただし、リカバリコードはユーザーの負担が高いことに留意が必要である。
- カスタマーサポート経由の高強度リカバリを「最後の手段」として設計する。書留郵便や店舗での対面確認、公的身分証・デジタルアイデンティティを用いた再本人確認など、コストは高いがなりすまし耐性の高いチャネルを用意し、高額資産や高リスク操作を伴うアカウントのみを対象とする。これは頻度は低いが、同期パスキーとバックアップキーがすべて失われたケースに対応するためのセーフティネットとなる。
- 脆弱なリカバリ経路を廃止し、全体の保証レベルを整合させる。秘密の質問や、平文に近いメールリンクのみといった手法はパスキーの強度を損なうため避けるべきである [21, 47]。リカバリにパスキー以外の経路を残す場合でも、「リカバリコード＋既存デバイス」や「リカバリコード＋再本人確認」など複数要素の組み合わせにより、可能な限り認証保証レベルを維持することが望ましい [3]。

以上の整理は、「アカウントリカバリー経路の下限強度を確保して最弱点化を避けること」と、「複数チャネルで冗長性を確保してロックアウトを管理すること」を同時に満たすための具体策である。第 4 章以降では、これらの観点が実サービスにおいてどの程度満たされているかを評価軸として、調査設計と結果整理を行う。

4　調査設計

本章では、第 3 章で整理した評価観点に基づき、実サービスにおける実装状況を観測するための、調査対象・手順・調査項目、および倫理面・安全性への配慮を述べる。

4.1　調査対象の選定

本研究では、国内事業者が提供する Web サービス 10 件、および国外事業者が提供する Web サービス 20 件の計 30 件を調査対象として選定した。本節では、その選定基準および選定プロセスについて述べる。

4.1.1　調査要件

本研究では、第 2 章で整理した RP の実装範囲のうち外部から観測可能な要素に着目し、実サービスにおける実装状況を明らかにする。具体的には、登録・認証における Web Authentication API のパラメータ設定と、周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、ならびにアカウントリカバリー等）を観測対象とする。そのため、調査対象サービスは少なくとも次の要件を満たす必要がある。

- **観測可能性**: Web ブラウザ経由でログインが可能であり、ブラウザの開発者ツール等を用いて Web Authentication API の呼び出しにおけるパラメータを観測できること。
  - モバイルアプリケーション等、ネイティブアプリケーションのみでパスキー認証を提供するサービスは除外した。これは、本研究では Web ブラウザ上での UI 挙動と Web Authentication API のパラメータを観測対象とするのに対し、ネイティブアプリケーションでは同等の観測が困難であるためである。
- **アカウント作成の容易性**: 調査用アカウントを作成するにあたり、過度な個人情報の提供や金銭的コストを要しないこと。
  - 具体的には、本人確認書類（身分証明書）の提出や、詳細な住所情報の登録が必須であるサービスは除外する。
  - また、クレジットカード情報の登録が必須であるサービスについても、調査の再現性と安全性の観点から対象外とする。

4.1.2　国内サービスの抽出

国内の Web サービスにおけるパスキー導入状況を網羅的に示したデータベースは、調査時点において存在しなかった。そのため、検索エンジンを用いた手動検索を実施した。「パスキー」、「FIDO 認証」、「WebAuthn」等のキーワードを用いて検索を行い、要件を満たす国内主要サービス 10 件を選定した。

検索および抽出の過程において、国内では特に金融分野（銀行および証券会社）におけるパスキー導入が急速に進展していることが確認された [48]。しかしながら、これら金融機関のサービス利用にあたっては、厳格な本人確認（eKYC、マイナンバーカードの読み取り、あるいは転送不要郵便による住所確認）が口座開設時に必須となる。これらの手続きは、4.1.1 節「調査要件」で述べた「アカウント作成の容易性（過度な個人情報の提供を要しないこと）」の要件を満たさない。したがって、調査対象からは除外した。

4.1.3　国外サービスの抽出

国外サービスの抽出にあたっては、Kuchhal らによる先行研究 [15] を参照した。同研究では、Web トラフィックランキング上位（Tranco Top 1000）のサイトにおける FIDO2 導入状況を調査している。その中でパスキー利用が確認されたサービスリストを基礎データとして利用し、このリストから前述の調査要件を満たす 20 サービスを抽出した。

4.1.4　除外事例とその理由

前述した抽出プロセスにおいて、候補として抽出されたサービスであっても、本研究の調査要件に照らして以下の理由により対象から除外した事例が存在する。

- **技術的な制約による除外**:
  - **AoL**: アカウント作成プロセスにおいて、SMS 等で受信する確認コードの入力（電話番号による追加の本人確認）を求められたが、試行回数制限によりシステムエラー（"Too many failed attempts"）が発生し、調査用アカウントの作成が完了できなかったため。
  - **Instagram**: 著者の調査環境（2025 年 10 月 8 日時点）において、設定画面にパスキー認証を設定する項目が表示されておらず、パスキー認証を利用できなかったため。
- **倫理的・内容的適合性による除外**:
  - **Chaturbate**: 同サービスは成人向けコンテンツを主とするプラットフォームであり、Web サイト内のスクリーンショット撮影を含む UI/UX 調査は、公序良俗の観点から不適切であると判断し除外した。
- **前提要件（決済情報）による除外**:
  - **AWS (Amazon Web Services)**: アカウント作成の最終段階においてクレジットカード情報の登録が必須であり、本研究の調査要件を満たさないため除外した。

4.1.5　調査対象サービス一覧

最終的に抽出された調査対象サービスは以下の通りである（表 1）。

表 1: 調査対象 Web サイト一覧

区分	サービス名	URL	調査日
国内	Hatena	hatena.ne.jp	2025 年 09 月 18 日
	MIXI M	account.mixi.com	2025 年 09 月 22 日
	Money Forward	id.moneyforward.com	2025 年 09 月 23 日
	My Sony	my.account.sony.com	2025 年 09 月 23 日
	NIKKEI ID	id.nikkei.com	2025 年 09 月 18 日
	ニンテンドーアカウント	accounts.nintendo.com	2025 年 09 月 25 日
	ヌーラボ	apps.nulab.com	2025 年 09 月 25 日
	pixiv	pixiv.net	2025 年 09 月 23 日
	RECRUIT ID	point.recruit.co.jp	2025 年 09 月 25 日
	Yahoo! JAPAN	yahoo.co.jp	2025 年 09 月 14 日
国外	Alibaba Cloud	alibabacloud.com	2025 年 10 月 27 日
	Amazon	amazon.co.jp	2025 年 10 月 08 日
	Bitbucket(Atlassian ID)	id.atlassian.com	2025 年 10 月 23 日
	Cloudflare	cloudflare.com	2025 年 10 月 05 日
	Dropbox	dropbox.com	2025 年 10 月 05 日
	GitHub	github.com	2025 年 10 月 04 日
	GitLab	gitlab.com	2025 年 10 月 26 日
	GoDaddy	jp.godaddy.com	2025 年 10 月 29 日
	Google	google.com	2025 年 10 月 04 日
	Mail.ru	account.mail.ru	2025 年 10 月 30 日
	Microsoft	account.microsoft.com	2025 年 10 月 04 日
	Namecheap	namecheap.com	2025 年 10 月 23 日
	Norton	jp.norton.com	2025 年 10 月 19 日
	NVIDIA	nvidia.com	2025 年 10 月 05 日
	Roblox	roblox.com	2025 年 10 月 27 日
	Stripe	stripe.com	2025 年 10 月 05 日
	WordPress	wordpress.com	2025 年 10 月 19 日
	X	x.com	2025 年 10 月 04 日
	Zoho	zoho.com	2025 年 10 月 30 日
	Yahoo	yahoo.com	2025 年 10 月 08 日

4.2　調査手法

本節では、調査の目的（第 3 章で整理した評価観点）、調査環境、記録項目と記録方法、および各サービスに対して実行した調査手順を述べる。

4.2.1　調査の目的と評価観点

本調査の目的は、第 3 章で整理した評価観点に照らし、主要 Web サービスにおけるパスキー認証の実装状況を観測し、比較可能な形で整理することである。具体的には、(i) Web Authentication API のパラメータ設定と、(ii) 周辺設計として外部から観測できる挙動（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー等）を中心に、各サービスの挙動を比較する。

4.2.2　調査環境の構築

調査の再現性を高め、閲覧履歴やキャッシュ等の影響を排除するため、以下の統制された環境で実施した。

- **OS**: Windows 11（アカウント作成・初期登録・初期認証）および macOS Tahoe 26（異なる OS 環境からのログイン、二つ目以降のパスキー登録）
- **ブラウザ**: Google Chrome
- **ブラウザ設定**:
  - 調査用の新規ユーザープロファイル（クリーンプロファイル）を作成し使用した。
  - DevTools で `Disable cache` を有効化し、キャッシュの影響を排除した。
  - `Preserve log` を有効化し、ページ遷移を跨いだ通信の追跡を可能にした。

4.2.3　記録項目と記録方法

評価観点に対応する根拠を残すため、(1) 構造化データ（API パラメータ等）と、(2) 観察記録（UI 遷移等）を併用して記録した。

■Web Authentication API パラメータの抽出　`navigator.credentials.create()` および `navigator.credentials.get()` が呼び出された際、引数として渡される `PublicKeyCredentialCreationOptions` および `PublicKeyCredentialRequestOptions` を JSON として保存した。

■UI の観察記録　ダイアログの表示、設定画面、エラーメッセージ等の各操作ステップにおいてスクリーンショットを保存し、登録・認証時の画面遷移、分岐条件、ユーザーへ提示される説明文言を記録した。

■通信ログの選択的保存　通信ログは常時監視しつつ、登録・認証・削除等の重要操作に関係する API 応答やエラー処理を確認できた場合に限り、当該ペイロードを選択的に保存した。これは、UI には現れないメタデータ（例：RP が返す認証器情報等）の有無を確認するためである。

4.2.4　調査項目と実行手順

各対象サービスに対し、アカウントの新規作成からパスキー登録、認証、管理に至るライフサイクルを同一方針で実行し、前項の方法で記録した。具体的な手順は次のとおりである。

1. **アカウント作成および初期パスキー登録**
   - 新規アカウントを作成し、第 3.2.1 節で述べた観点に基づき、アカウント作成手順内または作成直後にパスキー登録が明示的に案内されるかを記録する。
   - 初期パスキーとして Windows 環境のプラットフォーム認証器（Windows Hello）を用いて登録を行う。これにより、当該アカウントを「デバイス固定パスキーのみが登録された状態」とし、以降のクロスデバイス検証の前提条件を整える。
   - 登録セレモニーにおける `navigator.credentials.create()` の発行タイミングとパラメータ、画面遷移、登録後通知（メール等）の有無を記録する。
2. **初期パスキーによる認証 (Authentication)**
   - 登録したパスキー (Windows 11 で作成したもの) を用いて、当該デバイスでのログインを実施する。
   - 認証時の画面遷移および `navigator.credentials.get()` のパラメータを記録する。
3. **異種 OS 環境からのログインと追加登録**
   - ステップ 1 で使用したデバイス (Windows) とは異なる OS 環境 (macOS) のデバイスを用いてログインを試みる。既存のパスキー (Windows Hello) が利用できない環境において、パスワードやメール認証などの代替手段を用いてログインが可能か、そのフォールバック手順を記録する。
   - ログイン成功後、2 つ目のパスキーとして iCloud Keychain (同期パスキー) を用いた追加登録を試みる。この際、クロスプラットフォーム間でのパスキー追加の可否、パラメータ、および追加登録に伴うセキュリティ通知（メール等）の有無を確認する。
4. **パスキー管理機能およびメタデータの検証**
   - アカウント設定画面（管理画面）へアクセスし、登録済みパスキーの一覧表示項目を確認する。パスキーの名称（AAGUID に基づく自動命名か、ユーザーによる編集が可能か）、同期・非同期ステータスの表示、登録日および最終使用日の記録状況、削除機能の実装（確認ダイアログの有無）を調査する。
   - ブラウザの開発者ツール (Network パネル) を用いて通信ログを解析し、UI 上には表示されていないが RP からブラウザへ送信されているメタデータ（認証器の詳細情

報等）についても記録する。

5. **仮想認証器への対応状況の検証**
   - Google Chrome DevTools の WebAuthn Emulator 機能 [46] を利用し、仮想認証器 (Virtual Authenticator) を用いたパスキー登録、およびパスキー認証によるログインが可能か検証する。これにより、開発者向けツールやエミュレータ環境に対する制限の実装状況を確認する。
6. **アカウントリカバリー手段の確認**
   - パスキーを紛失した、あるいは利用できない状況を想定し、提供されているアカウントリカバリー手段（代替認証、リカバリーコード、公式ヘルプの案内等）を確認する。これはステップ 3 における別デバイスからのアクセス検証と併せて評価する。

なお、重要な情報の変更時などに求められる「再認証」については、各サービスの仕様により発生タイミングが異なるため、調査プロセス中に要求された場合に限り、その時点での `navigator.credentials.get()` パラメータおよび UI 挙動を追加的に記録した。

4.2.5　倫理面および安全性への配慮

本節では、調査の実施における倫理面の配慮（非侵襲性）と、偶発的にセキュリティ事象が観測された場合の対応方針について述べる。

■非侵襲的な調査手法への限定　他のユーザーやサービス運営への影響を避けるため、本調査における操作は、Web ブラウザを通じて正規のユーザーが実施可能な範囲に限定した。

- **正常系操作の模倣**: 本研究は、正規のユーザーが一般的に利用する状況での挙動を観測対象とする。そのため、調査では、各サービスで想定されている正規のユーザーとして振る舞い、意図的に形式を逸脱した不正なパラメータの送信や、認証ロジックの破壊を試みる攻撃的テスト（Fuzzing 等）は実施していない。
- **可用性への配慮**: 他のユーザーおよびサービス運営への影響を考慮し、すべての操作は手動で実行した。

■脆弱性発見時の対応プロトコル　本研究の主目的はパスキーの実装状況および UI/UX の調査であり、潜在的な脆弱性の発見を意図したものではない。しかしながら、RP の何らかの実装上の不備等により、認証回避や個人情報の漏洩につながる未公表の重大な脆弱性が偶発的に観測される可能性は否定できない。

そのような事態が発生した場合、本研究では以下の「責任ある開示」プロセスを採用する。

1. **調査の中断**: 脆弱性が疑われる挙動を確認した時点で、当該機能に対する詳細な深掘り調査は直ちに中止する。
2. **開発者への報告**: 発見された問題について、当該サービスのセキュリティ窓口、または

IPA（情報処理推進機構）等の公的な届出窓口を通じて速やかに報告を行う。

3. **論文記述の制限**: 修正が完了していない脆弱性に関しては、攻撃の再現が可能となる詳細な手順やパラメータ情報の論文への記載を控える。

以上の指針に基づき、本研究は学術的な調査としての公益性と、実社会への安全性の両立を図っている。

ここで、実際に偶発的に観測した事象について報告する。本研究の調査過程において、調査対象サービスの一つで、開発・デバッグ用の情報が第三者から参照可能な状態になっていることを偶発的に観測した。これはパスキー認証の挙動と直接関係するものではないが、攻撃者による脆弱性探索を容易にする可能性があると判断したため、以降の深掘り調査は行わず、当該サービスが公開するセキュリティ連絡窓口に 2025 年 11 月 6 日に連絡した。その後、約 3 週間後に修正を確認した。

5　調査結果

第 3 章で整理した設計指針（評価観点）に基づき、実際の Web サービスにおけるパスキー認証の実装状況を、Web Authentication API のパラメータ設定（`navigator.credentials.create()` / `navigator.credentials.get()` の指定値）と周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー等）の観点から整理する。5.1 節では主要パラメータの設定値を、5.2 節では周辺設計の実装状況を示す。各項目の調査結果は表にまとめ、本文では結果の要点と代表例を述べ、必要に応じて第 3 章の推奨との関係を補足する。

なお、本章で扱う観測対象はブラウザ上で確認できる API 呼び出し時の指定値および画面上の手順・文言であり、サーバー側がこれらをどのように検証・活用しているかは本研究の観測範囲外である。

5.1　Web Authentication API パラメータの実装状況

本節では、登録および認証セレモニーにおいて RP が指定する主要パラメータ（`navigator.credentials.create()` / `navigator.credentials.get()`）の設定値を整理する。

5.1.1　navigator.credentials.create()

本節では、登録セレモニーでブラウザ上の JavaScript から呼び出される `navigator.credentials.create()` メソッドの主要パラメータについて、その設定値の調査結果を報告する。

表 2、3 は、それぞれ国内/国外サービスでの `navigator.credentials.create()` の主要パラメータの調査結果である。`authenticatorAttachment` は、1 回目と 2 回目のパスキー登録で記録し両方の値を記載している。ここで「1 回目」は当該アカウントに対する初回のパスキー登録を、「2 回目以降」は同一アカウントに対する追加のパスキー登録を指す。また、`excludeCredentials` は 2 回目以降の登録でのみ意味のある値であるため、その値のみ記載した。Alibaba Cloud は複数のパスキーを登録できなかったため、2 回目以降での `authenticatorAttachment` と `excludeCredentials` は「不明」とした。さらに、ヌーラボ と Google では、通常のパスキー登録とは別にセキュリティキー登録用のボタンが別途用意されていたため、参考として当該ボタンから登録セレモニーを開始した場合で観測されたパラメータも表に併記した。ただし、以下の本文における集計（件数のカウント）では、特段の断りがない限り、通常の「パスキー登録」フローでの観測値を対象とする。

略語: Att=attestation, UV=userVerification, AA1=authenticatorAttachment(1 回目), AA2=authenticatorAttachment(2 回目以降), Ex=excludeCredentials(2 回目以降), Alg=pubKeyCredParams, RK=residentKey.　**記号:** Ex: ◯=Credential

ID が設定されている,×=設定されていない。

表 2: navigator.credentials.create() パラメータ：国内

RP	Att	UV	AA1	AA2	Ex	Alg	RK
Hatena	none	preferred	undefined	undefined	◯	RS256, RS384, RS512, ES384, ES512, PS256, ES256, Ed25519	preferred
MIXI M	none	required	undefined	undefined	◯	PS256, ES256	required
Money Forward	none	required	platform	platform	◯	RS256, PS256, ES256	required
My Sony	none	preferred	undefined	undefined	◯	RS256, PS256, ES256	preferred
NIKKEI ID	none	preferred	undefined	undefined	◯	RS256, ES256, Ed25519	required
ニンテンドーアカウント	none	required	platform	platform	◯	ES384, ES512, ES256, Ed25519	required
ヌーラボ	none	required	platform	platform	◯	RS256, RS384, RS512, ES384, ES512, ES256, Ed25519	required
ヌーラボ（セキュリティキー作成ボタン）	none	required	cross-platform	cross-platform	×	RS256, RS384, RS512, ES384, ES512, ES256, Ed25519	preferred

表 2: navigator.credentials.create() パラメータ：国内（続き）

RP	Att	UV	AA1	AA2	Ex	Alg	RK
pixiv	none	required	platform	platform	◯	RS256, ES256, Ed25519	required
RECRUIT ID	none	required	platform	platform	◯	RS256, ES256	required
Yahoo! JAPAN	none	required	platform	platform	×	RS256, ES256	preferred

表 3: navigator.credentials.create() パラメータ：国外

RP	Att	UV	AA1	AA2	Ex	Alg	RK
Alibaba Cloud	direct	discouraged	undefined	不明	不明	RS256, ES256	direct
Amazon	direct	preferred	platform	platform	◯	ES256	required
Bitbucket (Atlassian ID)	none	discouraged	undefined	undefined	◯	RS256, ES256, Ed25519	discouraged
Cloudflare	none	preferred	undefined	undefined	◯	RS256, ES256	discouraged
Dropbox	direct	discouraged	undefined	undefined	◯	RS256, ES256	preferred
GitHub	none	required	undefined	undefined	◯	RS256, ES256	required
GitLab	none	preferred	undefined	undefined	◯	RS256, PS256, ES256	discouraged
GoDaddy	none	discouraged	undefined	undefined	×	RS256, PS256, ES256, Ed25519	discouraged
Google	direct	preferred	platform	platform	◯	RS256, ES256	preferred

表 3: navigator.credentials.create() パラメータ：国外（続き）

RP	Att	UV	AA1	AA2	Ex	Alg	RK
Google（セキュリティキー作成ボタン）	direct	preferred	cross-platform	cross-platform	◯	RS256, ES256	preferred
Mail.ru	none	required	undefined	undefined	◯	RS256, RS384, RS512, ES384, ES512, PS256, ES256, Ed25519	preferred
Microsoft	direct	required	undefined	undefined	◯	RS256, ES256	required
Namecheap	direct	discouraged	undefined	undefined	◯	RS256, RS384, RS512, ES384, ES512, PS256, ES256, Ed25519	discouraged
Norton	none	preferred	undefined	undefined	×	RS256, RS384, RS512, ES384, ES512, ES256, Ed25519	discouraged
NVIDIA	none	required	undefined	undefined	◯	RS256, ES256	required

表 3: navigator.credentials.create() パラメータ：国外（続き）

RP	Att	UV	AA1	AA2	Ex	Alg	RK
Roblox	direct	preferred	undefined	undefined	◯	RS256, RS384, RS512, ES384, ES512, PS256, ES256, Ed25519	required
Stripe	none	required	undefined	undefined	◯	RS256, PS256, ES256	required
WordPress	none	preferred	undefined	undefined	◯	RS256, ES256	preferred
X	none	preferred	platform	platform	×	RS256, ES256	required
Yahoo	direct	preferred	undefined	undefined	◯	ES256	discouraged
Zoho	direct	required	undefined	undefined	◯	RS256, ES256	required

■Attestation (構成証明) 第 3 章 3.1.1 節で述べた通り、広く一般公開されるサービスでは認証器の排除を避ける観点から `attestation: none` が推奨されることが多い。

観測として、国内の調査対象 10 サービスは全て `none` を設定しており、この推奨と整合していた。国外サービスでも、20 サービス中 11 サービスが `none` を設定しており、`none` が過半であった。一方で、国外の残る 9 サービス（Amazon、 Dropbox、 Google、 Microsoft、 Yahoo、 Namecheap、 Alibaba Cloud、 Roblox、 Zoho）では `direct` が設定されていた。

`attestation: direct` の指定は、認証器に関する追加情報（例：認証器の種類やモデルを推定する手掛かり）を得て検証・記録したい場合に利用されうるため、セキュリティ要件が高いと想定されるサービス群で採用されやすい可能性がある。また、第 3 章で述べた通り、パスキー管理画面における認証器の識別性向上（例：認証器/パスキープロバイダ名の表示）に向けて、認証器に関する情報を保持したい意図が背景にある可能性もある。ただし、同様にインフラや決済を担う Cloudflare および Stripe が `none` を設定していたことから、事業領域のみで一意に説明できるわけではない。また、本研究の観測対象はブラウザ上で確認できる指定値であり、サーバー側が Attestation をどの程度検証・活用しているか、ならびに `direct` の指定が実運用でどのような運用方針につながっているかは、本研究の観測範囲外である。

■User Verification (ユーザー検証)　観測として、表 2、表 3 に示す通り、登録（navigator.credentials.create()）における userVerification の指定は、国内と国外で分布が異なっていた。国内サービスでは 10 サービス中 7 サービスが required を指定しており、preferred を指定していたのは、Hatena、NIKKEI ID、My Sony の 3 サービスであった。国外サービスでは preferred が 9 サービスと最も多く、次いで required が 6 サービス、discouraged が 5 サービス（Alibaba Cloud、Bitbucket、Dropbox、Namecheap、GoDaddy）であった。userVerification は、登録時にローカルでのユーザー検証（例：生体認証や PIN）をどの程度要求するかに関わるため、登録時の手間・失敗率・対応可能な認証環境の幅に影響する可能性がある。required を指定した場合、ローカルでのユーザー検証が常に要求されやすくなる一方で、ユーザー側の環境（センサー未設定、PIN 未設定、セキュリティ設定等の制約）によっては登録が成立しない可能性がある。preferred や discouraged は、認証器側の挙動に委ねる余地が大きくなり、登録の成立条件やユーザー体験の観点での選択肢となる。また、国外で discouraged を指定していたサービス群には、後述するように、パスワード認証後の二段階認証（2FA）の一方式としてパスキー認証を用いている例が含まれる。このような構成では、パスキー認証側でローカルのユーザー検証を一律に強制するのではなく、前段の要素や運用と組み合わせた設計になっている可能性がある。

ただし、本研究の調査は API 呼び出し時の指定値に基づくものであり、サーバー側がユーザー検証の結果（ブラウザから送られるフラグ）をどのように検証・利用しているかまでは観測できていない。

■Authenticator Attachment (認証器の要件)　authenticatorAttachment については、国内サービスの多くが platform を指定しているのに対し、国外サービスは未指定（undefined）とする傾向が見られた。具体的には、国内サービス 10 例中 6 例が platform を指定し、スマートフォンや PC 内蔵の認証器に限定していた。未指定（undefined）としたのは 4 サービスであった。ただし、ヌーラボについては、「セキュリティキー」と「パスキー」という名称でパスキー登録のボタンが別で用意されており、前者では cross-platform、後者では platform が設定されていた。一方、国外サービスでは 20 サービス中 17 サービスが undefined であり、プラットフォーム認証器とローミング認証器（セキュリティキー等）の双方を受け入れる設定となっていた。

3.1.1 節では、少なくとも一つ目のパスキー登録においては platform とすることが推奨されていることを述べたが、本調査結果によれば、一つ目と二つ目とで authenticatorAttachment の値を変えているサービスは無かった。例えば、一つ目を platform として同期パスキーの採用を期待する設定としつつ、二つ目以降では undefined としてセキュリティキー等の追加登録を受け入れる、といった段階的な設計も考えられるが、本調査対象では観測されなかった。

■Exclude Credentials (重複登録の防止)　`excludeCredentials` パラメータについては、最初のパスキー登録では意味をなさないため、2 回目以降の登録操作（既存のアカウントへのパスキー追加）について分析する。調査の結果、大半のサービスでは登録済みのパスキーの Credential ID リストが送信されていたが、国内の Yahoo! JAPAN、および国外の X、Norton、GoDaddy の計 4 サービスにおいては、このパラメータが空であることが確認された。これらのサービスでは、同一の認証器に対する追加登録を行った際、ブラウザ/OS 側でエラー等は表示されず、登録操作が完了した。また、管理画面上のパスキー一覧の件数が増加していることを確認した。ただし、追加された各エントリが暗号学的に異なる公開鍵であるかどうかは本調査の範囲では確定できていない。

`excludeCredentials` が空（未設定）の場合でも、少なくとも本調査で確認した範囲ではブラウザ/OS 側でエラーが表示されないまま登録が完了した。この場合、ユーザーは重複登録に気付きにくく、重複したパスキーが蓄積されると適切な管理画面が実装されていたとしても削除対象を判断しづらくなる恐れがある。また、第 3 章で述べた通り、重複登録は既存のパスキーを上書きする可能性がある。

■PubKeyCredParams (暗号アルゴリズム)　`pubKeyCredParams` で指定される公開鍵暗号アルゴリズムについては、サービスによって許容する種類数に幅が見られた。ヌーラボ では 7 種類、Hatena および Mail.ru では 10 種類以上のアルゴリズム候補が提示されていた。第 3 章で述べた通り、`pubKeyCredParams` は利用可能な認証器の範囲に影響するため、互換性を損なわない観点から、少なくとも Ed25519 (-8)、ES256 (-7)、RS256 (-257) を含めることが推奨される。観測として、調査した全サービスが ES256 (-7) を含んでいた。一方、RS256 (-257) については、国内の MIXI M および ニンテンドーアカウント、国外の Yahoo および Amazon においてリストに含まれておらず、ES256 等のみを受け入れる設定となっていた。また、Ed25519 (-8) は一部のサービス（国内：Hatena、NIKKEI ID、ヌーラボ、pixiv 等、国外：Namecheap、Roblox 等）で含まれていたが、全体としては混在していた。アルゴリズム候補が限定される場合、認証器やパスキープロバイダによっては登録が成立しない可能性がある。ただ、本調査ではアルゴリズム候補の限定によって登録や認証が失敗したケースは無かった。

■Resident Key（Discoverable Credential）　Discoverable Credential としての保存を要求する `residentKey` の指定は、国内外で傾向が異なっていた（表 2、表 3）。国内サービスでは `required` が 7 サービス、`preferred` が 3 サービスであった。ヌーラボ では認証器の種類に応じてパラメータを使い分けており、プラットフォーム認証器での登録時には `required` を、ローミング認証器での登録時には `preferred` を設定していることが確認された。

国外サービスでは `required` が 8 サービス、`preferred` が 4 サービス、`discouraged` が 7 サービスであった。一方で、Alibaba Cloud では `residentKey` に仕様上で定義されていない値（`direct`）が指定されている挙動が観測された。なお、この条件であってもパスキー登

録時にエラー等は表示されず、登録は成功した。

後述する通り国内サービスでは Conditional UI に対応しているサービスが多く、Conditional UI の利用には Discoverable Credential が前提となるため、国内で `required` が多い一因である可能性がある。ただし、国外サービスでは Conditional UI への対応が限定的であるにもかかわらず `required` が最多であったことから、Conditional UI 以外の要因（例：ユーザー名入力の有無、アカウント選択 UI の設計、既存の認証基盤との併用方針等）も影響している可能性がある。

国外サービスではパスキー認証を二段階認証の一環として（例えばパスワード認証と組み合わせて）利用するケースが少なくない。実際、そのようなサービスのうち、多くは `discouraged` を設定している（Bitbucket、Cloudflare、GitLab、GoDaddy、Namecheap、Norton）。例外として、Dropbox と WordPress は `preferred` を、Amazon は `required` を設定していた（Alibaba Cloud は仕様外の `direct` を指定）。Conditional UI の利用よりも、二段階認証の一つの選択肢としてパスキー認証を扱い、広く認証器を受け入れる設計を優先している可能性が考えられる。

5.1.2 navigator.credentials.get()

本節では、認証セレモニー中にブラウザ/OS で呼び出される `navigator.credentials.get()` メソッドの主要パラメータについて、その設定値の調査結果を報告する。認証セレモニーにおいては、コンテキスト（通常のログインか、再認証か）によって適切なパラメータ設定が異なるため、それぞれの状況に分けて分析を行う。

表 4、表 5 は、それぞれ国内/国外サービスでの `navigator.credentials.get()` の主要パラメータの調査結果である。ログインと再認証時それぞれを記載しているが、再認証時のパラメータを記録できたのは、国内ではヌーラボを除く 9 サービス、国外では Amazon、GoDaddy、Google、Microsoft、Roblox、Stripe、Zoho の 7 サービスであり、それ以外のサービスでは調査時に再認証が発生せず、確認できなかった。また、「補足」列では `allowCredentials` の解釈に必要な前提（例：ID 入力の有無、ボタン操作の有無）を記載している。

略語: AC=allowCredentials, UV=userVerification, **記号:** ◯=Credential ID が設定されている,×=設定されていない

表 4: navigator.credentials.get() パラメータ：国内

RP	補足	状況	AC	UV
Hatena	Conditional UI または「パスキーでログイン」ボタンをクリックで OS/ブラウザのダイアログからアカウント選択	ログイン	×	preferred
Hatena	「パスキーでログイン」ボタンをクリックで認証開始	再認証	◯	preferred
MIXI M	Conditional UI	ログイン	×	required
MIXI M	「認証する」ボタンをクリックで認証開始	再認証	◯	required

表 4: navigator.credentials.get() パラメータ：国内（続き）

RP	**補足**	**状況**	AC	UV
Money Forward	Conditional UI	ログイン	×	required
Money Forward	Conditional UI	再認証	◯	required
My Sony	メールアドレス 入力後、画面遷移し「パスキーでサインイン」をクリック。OS/ブラウザのダイアログからアカウントを選択。	ログイン	×	required
My Sony	Conditional UI だが allowCredentials が無い。	再認証	×	required
NIKKEI ID	Conditional UI または「パスキーでログイン」ボタンをクリックで OS/ブラウザのダイアログからアカウント選択。	ログイン	×	preferred
NIKKEI ID	Conditional UI または「パスキーでログイン」ボタンクリックで認証開始。	再認証	◯	preferred
ニンテンドーアカウント	Conditional UI または「パスキーでログイン」ボタンをクリックで OS/ブラウザのダイアログからアカウント選択。	ログイン	×	required
ニンテンドーアカウント	「パスキーでログイン」ボタンクリックで認証開始。	再認証	◯	required
ヌーラボ	Conditional UI または「パスキーでログイン」ボタンをクリックで OS/ブラウザのダイアログからアカウント選択。	ログイン	×	required
pixiv	Conditional UI または「パスキーでログイン」ボタンをクリックで OS/ブラウザのダイアログからアカウント選択。	ログイン	×	required
pixiv	「パスキーで続ける」ボタンをクリックして認証開始	再認証	◯	required
RECRUIT ID	Conditional UI	ログイン	×	required
RECRUIT ID	ユーザーの介入（ボタンクリック等）無しで自動で開始される。	再認証	◯	required
Yahoo! JAPAN	必要な ID（メールアドレス、電話番号等）を入力して画面遷移後、自動でパスキー認証開始。	ログイン	◯	required
Yahoo! JAPAN	再認証が要求された画面で、「ログイン」ボタンをクリックしてパスキー認証開始。	再認証	◯	required

表 5: navigator.credentials.get() パラメータ：国外

RP	**補足**	**状況**	AC	UV
Alibaba Cloud	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	discouraged
Amazon	メールアドレス入力後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	preferred
Amazon	自動で認証が開始	再認証	◯	preferred
Bitbucket (Atlassian ID)	メールアドレスとパスワードで認証後、「ログイン」ボタンをクリックしてパスキー認証開始。	ログイン	◯	discouraged
Cloudflare	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	preferred
Dropbox	メールアドレスとパスワードで認証後、画面遷移して二段階認証の画面でパスキー認証を選択。	ログイン	◯	discouraged
GitHub	Conditional UI または「signin with passkey」をクリックして OS/ブラウザのダイアログからアカウント選択。	ログイン	×	required
GitLab	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	discouraged
GoDaddy	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。複数のパスキーが登録されていても、「Default」に登録したパスキーの Credential ID のみが allowCredentials に設定されている。	ログイン	◯	preferred
GoDaddy	メールアドレスとパスワードで認証後、「Verification Method」として「Security Key」を選択で認証開始。複数のパスキーが登録されていても、「Default」に登録したパスキーの Credential ID のみが allowCredentials に設定されている。	再認証	◯	preferred
Google	Conditional UI	ログイン	×	preferred
Google	メールアドレス 入力後にパスキーでの認証を開始する旨の案内画面で「次へ」をクリックでパスキー認証開始	ログイン	◯	preferred
Google	メールアドレス 入力後にパスキーでの認証を開始する旨の案内画面で「次へ」をクリックでパスキー認証開始	再認証	◯	preferred
Mail.ru	メールアドレス 入力後、id.vk.ru に画面遷移し、そこで自動的にパスキー認証開始	ログイン	◯	required

表 5: navigator.credentials.get() パラメータ：国外

RP	**補足**	**状況**	AC	UV
Microsoft	メールアドレス 入力後、画面遷移して「顔、指紋、PIN、またはセキュリティキーを使用する」ボタンをクリックで認証開始。	ログイン	◯	required
Microsoft	Conditional UI	ログイン	×	required
Microsoft	「顔、指紋、PIN、またはセキュリティキーを使用する」ボタンをクリックで認証開始。	再認証	◯	required
Namecheap	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	discouraged
Norton	メールアドレスとパスワードで認証後、画面遷移してパスキー認証が自動で開始。	ログイン	◯	preferred
NVIDIA	「login with security device」をクリックし、OS/ブラウザのダイアログからアカウント選択。	ログイン	×	required
Roblox	Conditional UI	ログイン	×	preferred
Roblox	再認証要求の際に、「確認」ボタンクリックで認証開始。	再認証	◯	preferred
Stripe	メールアドレスを入力すると、UI が変化し、「このアカウントではパスキーが使用されています」」と表示される。そのまま「パスキーの使用を続ける」とクリックすると、パスキー認証に進む。	ログイン	◯	required
Stripe	再認証要求の際に、「パスキーを仕様」にチェックを入れたうえで、「続行」をクリックで認証開始。	再認証	◯	discouraged
WordPress	まずメールアドレスに届く認証リンクをクリックしてログイン、その後二段階認証の画面で「セキュリティキーを使用して続行」をクリックしてパスキー認証開始。	ログイン	◯	preferred
X	メールアドレス入力後、画面遷移してパスキー認証が開始するので、アカウントを OS/ブラウザのダイアログから選択。	ログイン	×	preferred
Yahoo	メールアドレスを入力して「Next」ボタンをクリックすると、自動でパスキー認証開始。	ログイン	◯	preferred
Zoho	メールアドレスを入力し、「次へ」をクリックすると、自動でパスキー認証が開始。	ログイン	◯	preferred
Zoho	再認証が要求される際に、自動で開始される。	再認証	◯	preferred

■Allow Credentials (認証器の絞り込み)　本パラグラフでは、`allowCredentials` パラメータの指定状況について分析する。

ログイン　国内サービスでは、調査対象とした 10 サービス中 9 サービスにおいて、`allowCredentials` が空（設定なし）の状態でリクエストされていた。表 4 の補足に示す通り、その多くは Conditional UI または「パスキーでログイン」ボタンを起点として OS/ブラウザダイアログでアカウント選択を行う設計であり、RP が事前にユーザーを特定しない状況では、`allowCredentials` を空とする指定と整合する。一方 My Sony では、ログイン手順の初めにメールアドレスを入力させているにもかかわらず、続く `navigator.credentials.get()` の呼び出し時に `allowCredentials` が空である挙動が確認された。その結果、ユーザーはメールアドレス入力後に表示されるブラウザのダイアログで、再度アカウントを選択し直す必要があり、ID 入力情報が有効に活用されていない実装となっていた。

国外でも同様に、`allowCredentials` に具体的な ID リストを設定するか、Conditional UI や「パスキーでログイン」ボタン等によりアカウント選択を行う設計が観測された。中でも X では、My Sony と同様に複数回のアカウント選択が必要となる挙動が確認されており、`allowCredentials` の指定と UI の整合が課題となり得る。また GoDaddy は特殊であり、設定から「Default」として指定したパスキーの `Credential ID` のみが `allowCredentials` として設定されていた。そのため、そのパスキーを利用できないデバイスからは、他のパスキーでログインするための操作が追加で必要となっていた。

再認証　再認証は、既に確立されたセッションに基づきユーザーが確定している状況で実行されることが多い。そのため、RP が `allowCredentials` を対象ユーザーに絞り込むことが基本となる。本調査で再認証が確認できた 16 サービス（国内 9、国外 7）のうち、15 サービスでは `allowCredentials` が設定されており、再認証時に無関係なアカウントが候補として提示されにくい設計となっていた。

ただ、My Sony においては、再認証時であっても `allowCredentials` が設定されていないことが判明した。この状態では、ユーザーが誤って別のアカウントのパスキーを選択して認証開始させてしまう可能性があり、ユーザーの混乱を招く恐れがある。また、RP が意図しないアカウントで認証が開始された場合、再認証が失敗する等のエラーにつながる可能性もある。

なお、RECRUIT ID 、Amazon 、Zoho など一部のサービスでは、適切な `allowCredentials` の指定に加え、ユーザーの明示的な操作（パスキーでログイン、ボタンのクリックなど）無しに自動的に再認証を開始する UX が実装されていた。

■User Verification (ユーザー検証)　認証（`navigator.credentials.get()`）における `userVerification` の指定状況を、ログイン時と再認証時に分けて整理する（表 4、表 5）。

まず、ログイン時の設定状況を見ると、国内サービスでは多くのサービスが `required` を指定しており、ローカルなユーザー検証を常に要求する構成となっていた。対して、より柔軟な `preferred` を採用していたのは、Hatena および NIKKEI ID のみであり、`discouraged` を指定するサービスは存在しなかった。一方、国外サービスでは、ログイン時に `preferred` を指定する事例が最も多く（10 件）、次いで `required`（5 件）、`discouraged`（5 件）が続く分

布となっていた。required を指定していたのは、Mail.ru、 NVIDIA、 Microsoft、 GitHub、Stripe など、相対的に高いセキュリティ要件を有すると考えられるサービス群に限られており、一般的なコンシューマ向けサービスでは preferred あるいは discouraged によって、ユーザー検証を緩やかに要求する運用が選択されていた。特に、パスワード認証と併用してパスキー認証を利用しているサービス（Alibaba Cloud、Bitbucket、Dropbox、GitLab、Namecheap）では、discouraged を設定していた。

再認証時についても、国内サービスではログイン時と同様に required が広く用いられていた。一方、国外サービスでは、Microsoft がログイン時・再認証時を通じて一貫して required を指定していた一方、Stripe ではログイン時に required を要求する一方、再認証では discouraged を設定していた。

本研究では認証時の userVerification について、セキュリティとユーザビリティのバランスを踏まえた現実的な解として preferred が推奨されていることを述べた。 その観点からは、国外サービスにおいて preferred が多数派である点は、ユーザー検証を一律に強制しない実装が多いことを示している。一方、国内サービスのようにログインおよび再認証の双方で一律に required を設定する運用は、ユーザーの環境（PIN 未設定等）によっては認証の摩擦や失敗につながる可能性がある。

5.2　周辺設計の実装状況

本節では、第 3 章で整理した周辺設計に関する評価観点に照らし、調査対象サービスにおける実装状況を整理する。以降の各節では、アカウント作成から登録、認証、管理、アカウントリカバリーまでの各フェーズについて、結果の要点と具体例を述べる。

5.2.1　アカウント作成とパスキー登録への誘導

3.2.1 節では、アカウント作成手順の中でパスキー登録を促す説明文言やボタン等が組み込むことが推奨されていることを述べた。調査の結果、アカウント作成時点でパスキー登録が明示的に案内される例は限定的であった。

国内サービスにおいて、アカウント作成時点でパスキー登録が案内されていたのは調査対象 10 サービス中 3 サービス（Hatena、Money Forward、RECRUIT ID）のみであった。Hatena と Money Forward では、アカウント作成後にパスキー登録を推奨する案内が表示され、その場でパスキー登録を開始可能であった。RECRUIT ID では、アカウント作成後に自動でマイページに遷移し、画面上部に「パスキー認証が利用可能である」旨が表示された。また、パスキー認証のみでアカウントを新規作成できるサービスは存在せず、依然としてパスワード認証や他の認証手段の登録が前提となっている。

国外サービスにおいても同様の傾向が見られ、20 サービス中 4 サービスのみがアカウント作成中または作成後にパスキーの登録を案内していた。これらの内、Yahoo、Microsoft、Amazon

などでは、アカウント登録手順が完了した後にパスキーの登録が案内される設計となっており、特に Amazon では登録手順の一環として自動的にパスキー登録プロセスが開始される挙動が確認された。また、NVIDIA ではサインアップの初期段階からパスキー認証を選択可能であった。

以上より、アカウント作成時においてパスキー登録を明示的に提示する設計は限定的で、ユーザーが自発的にパスキー認証を探しに行かなければ利用が開始されない現状が浮き彫りとなった。

5.2.2　用語の統一と誤解の防止

3.2.2 節で述べた通り、ユーザーがパスキー認証の挙動を誤解しないためには、ユーザー向けの説明で用いる用語を適切に選び、必要に応じて補足することが重要である。本節では、まず調査対象サービスにおける用語の観測結果を示し、続いて第 3 章で整理した評価観点に照らして、誤解につながり得る点を補足する。

表 6、表 7 は、それぞれ国内/国外のサービスで使われている用語の調査結果である。

表 6: パスキーに関する言葉遣い：国内

RP	**パスキーの呼称**	**作成時**	**削除時**
Hatena	パスキー	作成	削除
MIXI M	生体認証（パスキー認証）	端末を登録	削除
Money Forward	パスキー	登録	解除
My Sony	パスキー	作成	削除
NIKKEI ID	パスキー	作成	削除
ニンテンドーアカウント	パスキー	設定	解除
ヌーラボ	パスキー	作成	削除
pixiv	パスキー	作成	削除
RECRUIT ID	パスキー（生体認証）	登録	削除
Yahoo! JAPAN	パスキー	追加	削除

表 7: パスキーに関する言葉遣い：国外

RP	パスキーの呼称	作成時	削除時
Alibaba Cloud	U2F	Bind	Unbind
Amazon	パスキー	追加	削除
Bitbucket (Atlassian ID)	セキュリティキー	登録	削除
Cloudflare	Security Key Authentication	Add	Remove
Dropbox	セキュリティキー	追加	無効
GitHub	Passkeys	Add	Delete
GitLab	WebAuthn devices	Register	Delete
GoDaddy	Security Key	Add	Delete
Google	パスキー（とセキュリティキー）	作成	削除
Mail.ru	パスキー	追加	削除
Microsoft	パスキー	追加	削除
Namecheap	Device Authentication（U2F）	Add	Remove
Norton	セキュリティキー	追加	削除
NVIDIA	Hardware Security Device	Add	Remove
Roblox	パスキー	追加/作成	削除
Stripe	パスキー	追加	削除
WordPress	セキュリティキー	登録	削除
X	パスキー	作成	削除
Yahoo	Passkeys	Create	Remove
Zoho	パスキー（+ セキュリティキー）	設定/追加	削除

■「パスキー」の呼称　国内サービスにおいては、調査した全サービスが「パスキー」という呼称を採用しており、用語の標準化が進んでいることが確認された。一方、国外サービスでは 20 サービス中、「パスキー/Passkey」と明記しているのは 10 サービスに留まった。残りのサービスでは「Security Key (6 件)」、「U2F (2 件)」、「WebAuthn」、「Hardware Security Device」といった、ユーザーにとって技術的な差異が不明瞭な用語が混在しており、認知の妨げとなっている可能性がある。

特殊な例として、Google、Zoho、ヌーラボ では、通常のパスキー登録とは別に「セキュリティキー」として登録するボタンが用意されていた。これらのボタンは、技術的には na

`vigator.credentials.create()` のパラメータで `authenticatorAttachment` を `cross-platform` と設定される形で登録させることになるものである。同一サービス内で登録経路が分岐していることは、ユーザーが期待すべきログイン体験（パスキー認証単独での利用か、他の認証方式と組み合わせる前提での利用か）の違いを示唆する重要な実装例である。当該ボタンから登録された「セキュリティキー」は、単体では認証が完了できず、パスワード認証等と組み合わせる第二要素として扱われる運用が観測された。このように、用語に対するサービス側の位置づけによって、ユーザーが期待すべきログイン体験が異なり得る。

■「生体認証」表記　3.2.2 節で指摘した通り、「生体認証」という表現は、生体情報が RP に送信されるという誤解につながる可能性がある。本調査では、国内の MIXI M および RECRUIT ID において、「パスキー」という用語と併用する形で「生体認証」という言葉が用いられていた（表 6）。第 3 章で述べた評価観点に照らすと、このような表現を用いる場合には、生体情報が RP に送信されないこと等の前提を画面内で明示し、誤解を抑えるための補足を併記することが重要である。

■操作を表す動詞　パスキーのライフサイクル管理に関する動詞についても不統一が見られる。Google、X、Yahoo など一部のサービスでは「作成 (Create)」「削除 (Delete)」が用いられている一方で、多くのサービスでは「登録 (Register)」、「追加 (Add)」、「設定 (Set)」、「解除 (Unset)」といった用語が混在していた。国内では「登録（Money Forward、MIXI M、RECRUIT ID）」、「追加（Yahoo! JAPAN）」、「設定・解除（ニンテンドーアカウント）」といった表記揺れが見られた。

第 3 章で述べた通り、これらの動詞はユーザーに伝える表現であり、技術的な状態遷移を一意に保証するものではない。そのため、どの動詞を選択するにしても、その操作によって「何が起きるか」を補足することが重要である。

5.2.3　登録セレモニーのセキュリティと UX

本節では、登録セレモニー前後に関する調査結果を整理する。3.2.3 節で扱った観点（登録前の再認証、登録前後のメッセージング、登録完了後の通知）について、表 8、表 9 に国内/国外の結果を示す。

記号: ◯=メール通知が来る,×=来ない

表 8: 登録セレモニーのセキュリティと UX：国内

RP	**登録時再認証**	**メッセージ: 登録前**	**メッセージ: 登録後**	**メール通知**
Hatena	無し	生体情報がサーバーに送信されない旨、パスキーをパスワードに代わる、簡単で安全なログイン方法だと説明するメッセージと、およびパスキーに関するヘルプへの案内が「パスキーを作成する」ボタンが専用のページとして表示される。	画面上部に、パスキー名とともに登録された旨が表示される。	○
MIXI M	無し	管理画面上で、生体認証またはセキュリティキーの登録が求められるというメッセージが画面上部にある。	登録後に、画面下部に生体認証設定が完了した旨が表示される。	×
Money Forward	無し	ポップアップが（恐らく OS/ブラウザのダイアログのことを指しているものと思われる）表示されるという案内と、「パスキーを登録する」ボタンが専用のページとして表示される。	登録された旨が画面上部に表示される。	○
My Sony	登録前に再認証はなく、「登録後」に再認証が行われる。	指紋認証や顔認証などでログインできる旨と、パスワードが無効化されること、そしてパスキーに関するヘルプへの案内とともに「パスキーを作成」ボタンがモーダルで表示される。	「パスキーを使ってサインインできるようになりました」と表示	○
NIKKEI ID	毎回ではないが、再認証が求められる。 パスキーまたはパスワード。	パスワードよりも安全であること、生体認証を使用すること、そしてパスキーに関するヘルプへの案内とともに、「パスキーを作成する」ボタンが専用のページとして表示される。	完了した旨と、パスキーの名前・登録日時が表示される。	○

表 8: 登録セレモニーのセキュリティと UX：国内（続き）

RP	登録時再認証	登録前メッセージ	登録後メッセージ	メール通知
ニンテンドーアカウント	パスワード + メール宛に届く認証コード、またはパスキーを使った再認証が求められる。	生体情報がサーバーに送信されないこと、メールアドレスとパスワードは引き続き管理が必要なこと、およびパスキーに関するヘルプへの案内とともに「設定する」ボタンが専用のページとして表示される。	「OS + ブラウザ から設定」というパスキー名と登録日時が表示される。	×
ヌーラボ	無し	生体認証や PIN でログインできるようになること、パスワードマネージャーに保存されるため複数のデバイスで使用可能である旨とともに、「パスキーを作成」および「セキュリティキーにパスキーを作成」ボタンがモーダルで表示される。	画面右下にパスキーが登録された旨表示される	×
pixiv	メール宛に届く認証コードで再認証	顔認証・指紋認証・PIN などのデバイスの認証機能が使用できるようになること、2 段階認証が有効な場合でもパスキー認証だけでログインできるという説明と、パスキーに関するヘルプへの案内が「パスキーを作成」ボタンとともに専用のページとして表示される。	パスキーを登録した旨が表示され、パスキー名をその場で変更できる。	○
RECRUIT ID	無し	管理画面上で、対応デバイスの案内、パスキー登録でエラーが発生した際のヘルプとともに、「パスキーを登録する」ボタンが表示。	登録したパスキーの情報（OS、ブラウザ、登録日時）が表示される。	○
Yahoo! JAPAN	無し	パスキー管理画面上で、生体情報がサーバーに送信されない旨、およびパスキーに関するヘルプへの案内が「パスキーを追加」ボタンとともに表示。		○

表 9: 登録セレモニーのセキュリティと UX：国外

RP	登録時再認証	メッセージ: 登録前	メッセージ: 登録後	メール通知
Alibaba Cloud	メール宛に届く認証コードを入力	U2F キーを USB ポートに指し、キーのボタンを押すよう専用のページで指示される。	Bound に成功したこと、そして U2F 認証が今後必要なことが表示される。	○
Amazon	電話番号宛、またはメール宛に届く認証コード（6 桁）を入力	理画面からそのまま「パスキーを追加する」ボタンをクリックする。その画面の下部に、パスキーに関して「様々なデバイスでパスキーが使えること」「友人や家族とパスキーを共有する方法」「二段階認証が有効な場合パスキーを有効にしてもワンタイムコードが必要なこと」「パスキーに関するプライバシー考慮事項（生体情報はサーバーに送信されない旨など）」が詳しく文章で説明されている。	パスキーの登録が完了した旨が表示される。	○
Bitbucket (Atlassian ID)	パスワード	サポートされているモバイルデバイスまたは Web ブラウザー、ハードウェアキーが必要なことの説明と、そして複数のセキュリティキーを追跡できるように名前をつけることの要求が、「キーを登録」ボタンとともに専用ページで表示される。	セキュリティキーの登録に成功した旨が画面右上にポップアップで表示される。	○
Cloudflare	パスワードで再認証。（パスキーを登録していても、パスキーでの再認証はできない）	パスキーを追加する前に一度ダイアログが表示され、パスワード再入力が求められる。	セキュリティキーの登録が成功した旨が表示され、名前の入力が求められる。	×
Dropbox	パスワードでの再認証。 既にパスキーが登録されている場合、パスワード + 二段階認証（認証アプリ orSMS）	専用のプロンプトが何段階か表示される。 1. セキュリティキーを追加すると二段階認証ができる旨 2. パスワード認証 3. セキュリティキーを「差し込んだこと」のチェック	セキュリティキーの登録された旨が表示され、パスキー名の修正が求められる。	○

表 9: 登録セレモニーのセキュリティと UX：国外（続き）

RP	登録時再認証	登録前メッセージ	登録後メッセージ	メール通知
GitHub	無し	顔認証や指紋認証、デバイス PIN でログインできること、そしてパスワードと二要素認証を置き換えるものであることを説明する専用ページが、「Add Passkey」ボタンとともに表示される。	成功した旨の表示とパスキー名の入力が求められる。	◯
GitLab	OS/ブラウザ等でパスキーを登録「後」にパスワード入力	専用のページは無く、二要素認証の追加画面に、すべてのブラウザが WebAuthn をサポートしているわけではないこと、そしてリカバリコードを保存する必要があることが「Set up new device」というボタンとともに表示される。	登録に成功したことがポップアップで表示される。	◯
GoDaddy	メールアドレス + パスワード +Verification Method	ハードウェアセキュリティキーの操作方法の説明が専用ページで表示される。	成功した旨と、名前入力画面が表示される。	×
Google	無し	指紋認証・顔認証、画面ロックでログインできるようになること、ハードウェアセキュリティキー等も利用できること、そしてそれらの詳細についてのリンクが「パスキーを作成する」ボタンとともにモーダルで表示される。	パスキーが登録されたことと、ログイン時の本人確認で指紋認証、顔認証、画面ロック、ハードウェアのセキュリティキーを使用できるようになったことが表示される	◯
Mail.ru	電話番号宛に掛かってくる電話の末尾 6 桁を入力	FaceID や TouchID、USB/NFC/Bluetooth をサポートする外部デバイスを接続するよう案内するメッセージと、「追加する」「詳細」ボタンが専用ページで表示される。	パスキーが追加されたことと、当該デバイス/ブラウザでは指紋を使用してログインできること、外部デバイスを使用すればどのデバイスでもログインができるというメッセージが表示される。	×

表 9: 登録セレモニーのセキュリティと UX：国外（続き）

RP	登録時再認証	登録前メッセージ	登録後メッセージ	メール通知
Microsoft	登録時には再認証の必要はなく、初回登録後にメールに届く認証コードでの認証が必要。	専用のプロンプトではなく、追加するログイン方法の種類の一つとしてパスキーを追加するボタンが表示されている。また、パスキーに関するヘルプへのリンクも合わせて表示される。	名前の変更と、パスキーの登録が完了した旨モーダルで表示される。	○
Namecheap	無し	登録前に名前の入力が必要で、そこでユニークな名前をつけるよう推奨する文面と、「Register」というボタンとともに専用のページで説明される。	登録に成功した旨がポップアップで表示される。	×
Norton	無し	１つ目の登録時は特別なメッセージはなく、「セキュリティコードを受信する方法」として、「モバイル認証アプリ」か「セキュリティキー」を選択する。2 つ目以降の追加では、1 つ目の方法を利用できない場合のために別のデバイスの使用を推奨する旨が表示される。	パスキーの登録が完了した旨と、バックアップコードの保存を案内される。	○
NVIDIA	無し	ブラウザの指示に従って操作を完了させるよう専用ページで説明される。	登録に成功した旨がポップアップで表示される。	○
Roblox	パスワード （再認証が要求されない場合もある。） パスキー登録後はパスキーでも再認証可（パスワードでも可）。	パスキーが安全であることの強調、そしてセキュリティのために個人所有のデバイスを使うよう促すメッセージが、「続ける」というボタンとともにモーダルで表示される。パスキーのヘルプへの案内リンクも表示される。	画面上部に、パスキーの登録が完了した旨が表示される。	○

表 9: 登録セレモニーのセキュリティと UX：国外（続き）

RP	登録時再認証	登録前メッセージ	登録後メッセージ	メール通知
Stripe	メール宛に認証リンクが届くので、クリックして認証すると登録できる。	生体認証・PIN、ハードウェアセキュリティキーを利用してログインできる旨と、「続行する」ボタンがモーダルで表示。	パスキーが第二要素として必要になること、そしてログインする際はすべてのデバイスで当該パスキーにアクセスできなければならないことがモーダルで説明される。	○
WordPress	無し	専用のダイアログやページ等はないが、より堅牢な二段階認証を提供すること、セキュリティキーが物理的デバイスだけでなくブラウザに組み込まれていることもあると、「キーを登録」ボタンとともに、セキュリティの設定画面に表示される。また、パスキーに関するヘルプへのリンクも合わせて表示される。	登録後、画面右上に登録が完了した旨、ポップアップで表示される。	○
X	パスワードで再認証（パスキーが追加されていても）	漏洩やフィッシング詐欺に悪用されることが無いこと、二要素認証が有効になっていてもパスキーであれば追加の認証要素が必要ないことが「パスキーを作成する」ボタンとともにモーダルで表示される。	パスキーの登録が完了した旨がプロンプトで表示	○
Yahoo	メールアドレス + メール宛に届く認証コードでログイン。	指紋やデバイス解除に使うコードを使ってログインできることが、「Allow」ボタンとともに専用のページで表示される。	パスキーの登録が完了した旨専用の遷移画面で表示。	○
Zoho	毎回ではないが（恐らくログイン後時間が立ってから）、パスキーでの認証が要求される。（メールアドレスでの認証も可能）	対応する OS とブラウザについての案内と、パスキーが多要素認証とパスワードの組み合わせと同程度のセキュリティ対策ができること、そしてデバイス間で同期されることがモーダルで説明される。パスキー名の入力もセット。	パスキーが追加された旨が画面上部にポップアップで表示される。	×

■登録前の再認証　3.2.3 節では、パスキーの登録に先立ち、既にセッションが確立している場合でも再認証を要求することが推奨されていることを述べた。ここでは、再認証の強度を「登録開始前に追加の確認ステップが存在するか」と「その確認がパスワード以外（既存パスキー、メール/電話の認証コード、認証リンク等）を含むか」の 2 点で整理する。具体的には、(A) 登録前にパスワード以外を含む再認証を行う、(B) 登録前はパスワード再入力のみである、(C) 登録前の再認証がない、の 3 種で整理する。

国内では、(A) に該当する（登録前にパスワード以外を含む再認証を求める）サービスは 10 サービス中 3 件であり、NIKKEI ID、ニンテンドーアカウント、pixiv が該当した。一方、(C) のようにセッション確立後に登録前の再認証なしでパスキーの登録が可能なサービスは 6 件（Hatena、MIXI M、Money Forward、ヌーラボ、RECRUIT ID、Yahoo! JAPAN）であった。

My Sony は登録前の再認証はなく、登録後に追加したパスキーを用いた確認が求められていた。これは登録の正当性確認（登録前の再認証）ではなく、登録したパスキーで実際にサインイン可能な状態であることを検証する目的で実施されていると解釈できるため、本節の (A)–(C) とは別の特殊例として扱う。

国外では、(C) に該当するサービスは 20 サービス中 6 件（GitHub、Google、Namecheap、Norton、NVIDIA、WordPress）であった。また、(B) のように登録前の確認がパスワード再入力のみであるサービスは 5 件（Bitbucket、Cloudflare、Dropbox、Roblox、X）であった。また、Microsoft および GitLab では登録前に強い再認証が要求されない一方で、登録後に追加の確認ステップが存在することが確認された。

第 3 章で述べた観点に照らすと、(B) や (C) のように登録前の再認証が脆弱な（または存在しない）構成は、不正な登録のリスクを高め得るため、登録セレモニーの設計上の重要な課題である。登録プロセスにおける再認証の強制は、既存の認証基盤（メール OTP 等）を活用することで実装可能であり、セキュリティ向上の観点から改善が望まれる領域である。

■ハンドシェイクとプライバシーへの配慮　OS ネイティブの登録ダイアログが表示される前の「ハンドシェイク（説明画面）」の実装状況を分析する。

国内サービスでは、いずれも何らかのハンドシェイクが用意されていた。特に、Hatena、ニンテンドーアカウント、Yahoo! JAPAN では、生体情報がサーバーに送信されないことが明記されていた。3.2.3 で述べた通り、このような表記はユーザーの不安を軽減できる可能性がある。一方で、MIXI M、My Sony、NIKKEI ID、ヌーラボ、pixiv のように、生体認証を利用する旨を前面に出す表現が見られた。生体情報が RP に送信されない旨が併記されない場合、ユーザーの誤解につながる可能性がある。サイト内のヘルプ等で明記していても、ユーザーがそこまで参照するとは限らないため、ハンドシェイク内での説明が重要となる。これは、国外サービスの GitHub、Google、Mail.ru、Stripe、Yahoo にも当てはまる。

特に「生体認証」等の表現のみが強調される場合、ユーザーが「生体情報がサービス側へ送信される」あるいは「特定のセンサーを持つ端末が必須である」と誤解する可能性があるため、

3.2.3 節で述べた前提（生体情報が RP に送信されない等）を画面内で補足することが重要となる。

Amazon は特徴的であり、ヘルプ等に情報を集約するのではなく、パスキー登録を開始する画面内に、複数デバイスでの利用や二要素認証との関係、プライバシー配慮（生体情報がサーバーへ送信されない旨）等の説明をまとめて配置していた。ユーザーが全文を読むことは必ずしも期待できないが、登録を開始するのと同一画面で必要な補足情報へ到達しやすい設計例として位置づけられる。

■登録完了後の通知　3.2.3 節では、パスキーの登録はアカウントのセキュリティに関わる重要な変更であるため、登録完了時にメール等で通知することが推奨されていると述べた。この通知は、ユーザーが当該セッションに滞在していない場合でも（例えば不正な登録が行われた場合に）異変に気づく契機を提供する点で重要である。

調査の結果、国内では 10 サービス中 7 件、国外では 20 サービス中 15 件で通知メールが確認され、多くのサービスは推奨と整合していた。一方で、MIXI M、ニンテンドーアカウント、ヌーラボ、Cloudflare、GoDaddy、Mail.ru、Namecheap、Zoho では、パスキーを登録しても通知メールが送信されないことが確認された。

5.2.4　ログイン手順と Conditional UI

本節では、ログイン手順と Conditional UI の実装状況について分析する。調査結果は、表 4、表 5 に示す。

調査の結果、国内では Conditional UI に対応しているサービスが多かった（Hatena、MIXI M、Money Forward、NIKKEI ID、ニンテンドーアカウント、ヌーラボ、pixiv、RECRUIT ID）。一方、My Sony と Yahoo! JAPAN はログイン時の Conditional UI に対応しておらず、メールアドレスを入力して画面遷移した後にパスキー認証を行うフローとなっていた。ただし、My Sony は再認証手順では Conditional UI に対応していた。

一方、国外では Conditional UI への対応は限定的であった。調査対象 20 サービスのうち、Conditional UI に対応していたのは Google、Microsoft、GitHub、Roblox の 4 サービスのみであった。

国外サービスでは、パスキー認証を単体のログイン手段としてではなく、「二要素認証 (2FA) の一つ」として位置づける実装が複数確認された（Alibaba Cloud、Amazon、Bitbucket、Cloudflare、Dropbox、GitLab、GoDaddy、Namecheap、Norton、WordPress）。多くのサービスでは、ID とパスワードによる認証を通過した後に、追加でパスキー認証が要求されるフローとなっていた。特に Amazon では、二段階認証が有効な場合、パスキー認証の成功後に TOTP（ワンタイムパスワード）等の追加認証が毎回要求される挙動が確認された。パスキー認証は本来、フィッシング耐性を備えた強固な認証方式であり、単体で十分なセキュリティ強度を持つ。そのため、前段のパスワード認証の後にパスキー認証を要求する設計は、移行期の互換性確保としては合

理的である一方で、ユーザーの手順を増やしやすい。さらに、パスキー認証の成功後に追加でTOTP 等を毎回要求する構成は、パスキー認証導入のメリットであるユーザビリティの向上を損なう要因となり得るため、追加要素の要求条件（毎回かリスクベースか等）を慎重に設計することが望ましい。

5.2.5　パスキー管理機能の実装状況

本節では、管理画面における認証器/パスキープロバイダ情報の表示、編集機能、およびメタデータの扱いについて分析する。表 10、表 11 は、それぞれ国内/国外のパスキー管理画面の調査結果を表す表である。「パスキー名」の列では、管理画面におけるパスキー名がどのようにして決定されるか・できるかを説明している。「パスキー名変更」の列は、一度だけでなく何度も名前の変更ができる場合のみ ◯ にしている。例えば、Zoho のケースでは、登録直後に変更可能なもののその後は変更できないため、「パスキー名変更」の列は × となっている。「登録日」「使用日」は、パスキーの登録/使用についてどのような情報が表示されるかを表している。

また、同期されているかどうかの表示（表上の Sync の列）は管理画面上に同期可否（同期パスキー/デバイス固定パスキー等）を識別できる文言や説明が明示されている場合のみ ◯ としている。

略語: PasskeyName: UI 上のパスキー名がどのように設定されているか ReName: パスキー名を変更可能か RegiDate: パスキーを登録したときの情報 LastDate: パスキーを最後に利用したときの情報 Del: パスキーを削除可能か Sync: パスキーが同期されているかどうかが分かるか MSG: 同期パスキーの登録を促すメッセージがあるか CFM: 削除操作時に確認ダイアログが表示されるか **記号:** ◯=Yes,×=No

表 10: パスキー管理画面調査結果：国内

RP	PasskeyName	ReName	RegiDate	LastDate	Sync	MSG	Del	CFM
Hatena	自動で判別 使えない場合は "FIDO U2F セキュリティキー"。ブラウザの UserAgent も表示される。	◯	年・日付, 時間	年・日付, 時間	×	×	◯	◯
MIXI M	OS + ブラウザ名	◯	年・日付	年・日付	×	×	◯	◯
Money Forward	OS + ブラウザ名	×	年・日付	年・日付	×	×	◯	◯
My Sony	ブラウザの UserAgent	×	年・日付	無し	×	×	◯	◯

表 10: パスキー管理画面調査結果：国内（続き）

RP	PasskeyName	ReName	RegiDate	LastDate	Sync	MSG	Del	CFM
NIKKEI ID	自動で判別 使えない場合は、"YYYY/MM/DD に作成したパスキー"	×	年・日付, 時間	年・日付, 時間	○	×	○	○
ヌーラボ	自動で判別 使えない場合は "パスキー"	○	OS, ブラウザ, 年・日付	履歴, 時間	○	×	○	○
ニンテンドーアカウント	OS + ブラウザ名	×	年・日付, 時間	無し	×	×	○	○
pixiv	登録直後に名前が変更可能で、デフォルト値は自動判別。後から変更も可能。	○	年・日付	年・日付	○	×	○	○
RECRUIT ID	OS+ ブラウザ名	×	年・日付, 時間	無し	×	×	○	○
Yahoo! JAPAN	OS +「ブラウザー」	×	年・日付	年・日付	○	×	○	○

表 11: パスキー管理画面調査結果：国外

RP	PasskeyName	ReName	RegiDate	LastDate	Sync	MSG	Del	CFM
Alibaba Cloud	専用の管理画面がない	×	無し	無し	×	×	○	○
Amazon	自動で判別	○	年・日付	無し	×	×	○	○
Bitbucket (Atlassian ID)	登録前に手動入力。デフォルトは空欄	○	年・日付	時間	×	×	○	○
Cloudflare	登録直後に手動入力。デフォルト値は無し。	○	年・日付	年・日付, 時間	×	×	○	○
Dropbox	登録直後に手動入力。デフォルト値は自動で判別された値。	○	OS, ブラウザ, 年・日付, 時間	OS, ブラウザ, 年・日付, 時間	×	×	○	○

表 11: パスキー管理画面調査結果：国外（続き）

RP	PasskeyName	ReName	RegiDate	LastDate	Sync	MSG	Del	CFM
GitHub	登録直後に手動入力。デフォルト値は無し。	○	年・日付	年・日付	○	×	○	○
GitLab	登録直後に手動入力。デフォルト値は無し。	×	年・日付	無し	×	×	○	○
GoDaddy	登録直後に手動入力。デフォルト値は無し。	×	無し	無し	×	×	○	○
Google	自動で判別	○	年・日付, 時間	OS, ブラウザ, 場所, 年・日付, 時間	×	×	○	○
Mail.ru	"パスキー"	○	無し	無し	×	×	○	○
Microsoft	登録直後に手動入力。デフォルト値は自動で判別された値。	○	年・日付	IP アドレス, OS, ブラウザ, 場所, 履歴, 年・日付	○	×	○	○
Namecheap	名前は登録前に手動入力	×	無し	無し	×	×	○	○
Norton	"セキュリティキー"	×	年・日付	無し	×	×	○	○
NVIDIA	登録直後に手動入力。デフォルトでは、 "<Security Device Name>"	○	年・日付	年・日付	×	×	○	○
Stripe	登録直後に修正可能。デフォルトでは "自分のパスキー"	○	年・日付	年・日付, 時間	×	×	○	○
Roblox	自動で判別	×	無し	無し	×	×	○	○
WordPress	登録時にのみ、入力できる。デフォルト値は無し。	×	無し	無し	×	×	○	○

表 11: パスキー管理画面調査結果：国外（続き）

RP	PasskeyName	ReName	RegiDate	LastDate	Sync	MSG	Del	CFM
X	名前は自動入力 Web からだと "Web" iPhone のアプリからだと "iPhone"	×	年・日付, 時間	無し	×	×	○	○
Zoho	登録直後に手動入力。デフォルト値は無し。	×	時間	無し	×	×	○	○
Yahoo	自動で判別	×	年・日付, 時間	時間	×	×	○	○

■パスキー名の表示と編集機能　パスキー名の自動設定、編集機能の実装には国内外で顕著な差が見られた。

国外では、Amazon、Dropbox、Google、Microsoft、Roblox、Yahoo の 6 サービスで、認証器/パスキープロバイダの種類（例: "iCloud Keychain"、 "YubiKey 5 Series"など）を自動的に判別して表示する挙動が観測された。第 3 章 3.2.5 節で述べた通り、認証器が返す AAGUID 等の情報を用いることで、管理画面で認証器/パスキープロバイダ名を自動判別して表示することは可能である。しかし、本研究は管理画面上の表示を観測したものであり、当該サービスが実際に AAGUID を用いて判別しているか、あるいは別の情報（例：User-Agent 等）を用いているかは確定できない。また、これらのサービスはいずれもパスキー登録時に `attestation: direct` を設定していたが、これが管理画面での自動判別に必要であるか（あるいは偶然の相関か）についても、本研究の観測範囲では結論づけられない。

一方、国内サービスでは、pixiv、Hatena、NIKKEI ID、ヌーラボ のみがパスキー名の自動設定が行われていた。ただし、これらのサービスのうち、pixiv 以外は Windows Hello については 名称の自動設定ができていない。パスキー登録時、国内サービスはすべて `attestation: none` を設定しているのだが、Windows Hello はその際 AAGUID が得られないケースがある [49]。そのために、Windows Hello の場合は名称を自動で設定することができなかったものと推測される。pixiv では、Windows でパスキーが登録された場合に「Windows Hello」をデフォルトの名前にする設計 [49] のため、Windows Hello であっても名前を設定できている。

パスキー名の自動設定が行われないサービスでは、汎用的な名称（例: "FIDO U2F セキュリティキー"、"パスキー"）をデフォルト名前にしたり、「Chrome on Windows」のような OS とブラウザ名を利用した表示がなされている。これではユーザーが複数のパスキーを登録した際に識別することができなくなり、誤ったパスキーを削除してしまう可能性がある。

また、ユーザーによる「名称変更（リネーム）」機能の実装状況にも、国内外で差が見られた。国外サービスでは、任意のタイミングでユーザーが任意の名前に変更できる機能を 20 サービス中 10 サービスが提供している他、5 サービスは登録直前または登録直後にのみではあるが変更できる機能を提供している。対照的に、国内サービスでは 10 サービス中 6 サービスが名称の変更が一切不可能な仕様であった。AAGUID を取得できない、あるいは過去に AAGUID を保存しないまま登録されたパスキーに対しては、ユーザー自身によるラベリング機能の有無が重要となる。

■メタデータ（登録日・使用日）の可視化　3.2.5 で述べた通り、不正なパスキーの登録や、古い認証器/パスキープロバイダを発見するためには、登録日時および最終使用日時の表示が重要となる。調査の結果、国内では「登録日」の表示は全サービスで確認された。一方、国外では 20 サービス中 6 サービス（Alibaba Cloud、GoDaddy、Mail.ru、Namecheap、Roblox、WordPress）で「登録日」の情報が確認できなかった。また、「最終使用日」については、国内では 10 サービス中 3 サービス（ニンテンドーアカウント、My Sony、RECRUIT ID）で、国外では 20 サービス中 11 サービス（Alibaba Cloud、Amazon、GitLab、GoDaddy、Mail.ru、Namecheap、Norton、Roblox、WordPress、X、Zoho）で表示されていないことが確認された。最終使用日が不明な場合、ユーザーは現在使用しているアクティブなパスキーと、過去に登録して放置されたパスキーを区別しにくくなり、適切な棚卸し（削除）を行えない可能性がある。

また、Dropbox や Google、Microsoft では、日時に加えて OS やブラウザも表示される。さらに Microsoft では IP アドレスや利用された大まかな場所まで履歴として表示されることが確認された。

■同期ステータスの表示とバックアップへの配慮　同期パスキーとデバイス固定パスキーの違いを管理画面上でユーザーに明示しているサービスはごくわずかであり、国内の ヌーラボ、NIKKEI ID、pixiv、Yahoo! JAPAN、および国外の Microsoft、GitHub に留まった。

同期/非同期という性質は、認証器の紛失・故障時の可用性や、別デバイスからの利用可能性と関係するため、ユーザーが自身の状況を把握できるようにする観点で重要であることを、3.2.5 節で述べたが、そのような実装を行っているサービスは少数であった。

さらに、同期パスキーの登録を促すメッセージや警告機能は、調査した全サービスにおいて確認されなかった。ユーザーがデバイス固定のパスキーのみを登録している場合、デバイスの紛失や故障によってアカウントへのアクセス手段を失う可能性がある。一方で、サービスの要件によってはデバイス固定を前提とする設計もあり得るため、ユーザーがどの種類のパスキーを登録しているか、ならびに利用できなくなる可能性があることが UI 上で適切に伝達されているかが重要となる。

■削除機能と確認ダイアログ　調査対象の全サービスで管理画面からのパスキー削除機能および確認ダイアログの機能が実装されていた。

My Sony は特徴的で、パスキーを登録するとパスワードが無効化される仕様となっている。そのため、パスキーそのものを無効化しなければ、パスキーの最後の一つは削除できないようになっている。

■管理画面には反映されていないものの通信上では観測されるケース　管理画面上では対応していない部分についても、既にブラウザに対してデータが送信されており、将来的な改善が期待できるサービスが複数確認された。

- ニンテンドーアカウント：現時点で同期/非同期の区別はないが、デバイス上にあるかどうかを表すフラグが送られてきており、将来的には区別できるようになる可能性がある。
- NIKKEI ID：現時点で同期/非同期の区別はないが、`Backup Eligible` と考えられる値が送られてきており、将来的には表示される可能性がある。実際、NIKKEI ID による技術ブログの記事 [50] では、将来的にこの値を利用した同期/非同期の区別の機能を検討すると述べられている。ただし、同期/非同期の区別は `Backup Eligible` だけでなく `Backup State` フラグも確認する必要がある点に注意が必要である。
- My Sony：現時点で最終使用日は表示されていないが、最終使用日時に相当する情報と解釈される値が通信上で観測された。

5.2.6　仮想認証器と多様な認証環境への対応

本節では、開発者ツール等で利用できる「仮想認証器」の登録可否についての調査結果を分析する。3.3 節では、多様な認証環境への対応を一つの観点として整理した。

調査の結果、国内では 10 サービス中 4 サービス（RECRUIT ID、ニンテンドーアカウント、pixiv、Money Forward）において、仮想認証器を用いたパスキー登録が行えないことが確認された。具体的には、ブラウザの開発者ツールで仮想認証器を有効化すると、パスキー登録ボタンそのものがグレーアウトされ、操作がブロックされる挙動が見られた。これらのサービスは共通して Conditional UI に対応していることから、Conditional UI が利用できない環境をブロックしている可能性が考えられる。推測される技術的要因として、ブラウザが提供する `PublicKeyCredential.isConditionalMediationAvailable()`[*13] などの API を通じて認証器の環境を判定し、Conditional UI が想定通りに動作しない可能性のある仮想環境を意図的に排除している可能性が考えられる。ただ、実際にどのようにしてブロックをしているのか、なぜブロックをしているのかは、本研究の観測範囲では確定できない。

対照的に、国外では、20 サービス中 18 サービスで仮想認証器による登録および認証が可能であった。例外として、X では登録を試みた際に「問題が発生しました。しばらくしてからやり直してください」というエラーメッセージが表示され、仮想認証器による登録は成功しなかった。また Alibaba Cloud では、そもそも 2 つ目以降のパスキーが登録できない仕様であるため、

[*13] Conditional UI が利用可能かどうかを調べる静的メソッド [51]

仮想認証器による登録の可否を他サービスと同じ条件で比較できなかった。

さらに、表 3 の通り、パスキー登録時に `attestation: direct` を設定しているサービスでも仮想認証器で登録が成功した例が確認された。Kuchhal ら [15] が指摘する通り、Chrome の仮想認証器はあくまでテスト用であり、セキュリティ保証がない。少なくとも本調査条件では、`attestation: direct` の指定が仮想認証器の利用を排除する挙動に直結していなかった。なお、`attestation` パラメータの指定は Attestation の要求に関するものであり、サーバー側が Attestation をどの程度検証・活用しているかまでは本研究の観測範囲では確定できない。

3.3 節で整理した通り、RP が仮想認証器を含む多様な認証環境をどこまで受け入れるかは、利用可能性（新端末や特殊なブラウザ環境等でも利用できるか）と、テスト用途の認証器の悪用リスクの双方に関係する設計判断である。本調査結果からは、国内では 10 サービス中 4 サービスで仮想認証器がブロックされる一方、国外では 20 サービス中 18 サービスで登録・認証が可能であり、仮想認証器の受け入れ方針に差があることが示唆される。これは、受け入れる認証環境の範囲に関する設計判断がサービス間で異なる形で現れている可能性を示す。

5.2.7　アカウントリカバリーの実装状況

本節では、アカウントリカバリー戦略の調査結果を分析する。調査の結果、サービスごとにアカウントリカバリー戦略は大きく異なっており、セキュリティと可用性のバランスにおいて異なるアプローチが採られていることが明らかになった。

表 12: パスキー以外のログイン方法：国内

RP	**方法**
Hatena	引き続きパスワードでログイン可能
MIXI M	メールで届く認証コード
Money Forward	引き続きパスワードでログイン可能
My Sony	特殊な手順が必要。パスキーが利用できるデバイスが手元にある場合は、そのデバイスでの操作が必要。そうでない場合、電話番号とメール宛の両方で操作をしてアカウント回復をしなければならない。
NIKKEI ID	引き続きパスワードでログイン可能
ニンテンドーアカウント	パスワード + メールで届く認証コード（別途二段階認証を設定している場合、そちらでもよい）
ヌーラボ	パスワード + 二段階認証でログインが可能。
pixiv	パスワードと、メールで届く認証コード

表 12: パスキー以外のログイン方法：国内（続き）

RP	**方法**
RECRUIT ID	パスワード +（生年月日 or 氏名の入力）
Yahoo! JAPAN	SMS またはメールで届く認証コード

表 13: パスキー以外のログイン方法：国外

RP	**方法**
Alibaba Cloud	一度パスキーを登録すると、パスキーがないとログイン不可。メールアドレスや電話番号など含めた情報をフォームに入力しアカウント復旧を行う必要がある。
Amazon	引き続きパスワードでログイン可能
Bitbucket (Atlassian ID)	パスワード + バックアップコード
Cloudflare	パスキー登録時に保存したバックアップコードか、別の二段階認証の手段を使う。
Dropbox	パスワード + 二段階認証
GitHub	パスワード + 二段階認証（バックアップコード含む）
GitLab	パスワード + バックアップコード
GoDaddy	パスワードに加え、何らかの二段階認証が必要。あるいは、身分証明書等を使ってアカウント復旧を行う。
Google	引き続きパスワードでログイン可能
Mail.ru	SMS で届く認証コード
Microsoft	メールで届く認証コードでログイン可能。
Namecheap	パスワード＋バックアップコード
Norton	パスキー登録時に保存するバックアップコード
NVIDIA	追加のログイン認証を要求するよう明示的に設定しない限り、パスキー登録後もパスワードでのログインが可能。
Roblox	引き続きメールアドレス + パスワードでログイン可能。（メールアドレスや電話番号を登録していない場合、復旧できない可能性がある）
Stripe	パスワード + パスキー登録時に保存したバックアップコード。それに失敗した場合、身分証明書等を用いたアカウント回復が必要。
WordPress	メール宛に届く認証リンクで一段階ログイン + 二段階認証（アプリ、SMS、パスキー）

表 13: パスキー以外のログイン方法：国外（続き）

RP	方法
X	引き続きパスワードでログイン可能
Yahoo	メールで届く認証コード
Zoho	引き続きパスワードでログイン可能

表 12、表 13 は、それぞれ国内/国外サービスにおけるパスキー認証以外のログイン（アカウントリカバリー）手段の観測結果である。

まず、パスキー登録後もパスワードによるログイン手段を維持するサービスが確認された（国内：Hatena、Money Forward、NIKKEI ID、国外：Amazon、Google、X、Zoho、Roblox、NVIDIA）。なお、NVIDIA は「追加のログイン認証を要求するよう明示的に設定しない限り」パスワードでログイン可能であることが観測された。この構成では、パスキー認証を利用できない環境からも容易にログイン可能である一方で、アカウント全体のセキュリティがパスワード認証側の強度に依存し、パスキー認証導入によるフィッシング耐性向上の恩恵を享受できない可能性がある。

次に、パスキー認証以外のログインとして、SMS/メールの認証コード、バックアップコード、二段階認証等の追加要素を要求するケースも多かった。国内ではニンテンドーアカウント（パスワード + メール認証コード）、ヌーラボ（パスワード + 二段階認証）、pixiv（パスワード + メール認証コード）、RECRUIT ID（パスワード + 生年月日または氏名）などが該当した。また、MIXI M や Yahoo! JAPAN では、メール/SMS で届く認証コードによるログインが観測された。国外では Dropbox（パスワード + 二段階認証）、GitHub（パスワード + 二段階認証）、Bitbucket / GitLab / Namecheap（パスワード + バックアップコード）、Norton（バックアップコード）、Microsoft / Mail.ru / Yahoo（メール/SMS の認証コード）などが該当した。これらの方式はパスキー認証が提供するフィッシング耐性と同じ性質を持つとは限らないものの、「パスキー認証が使えない」という前提ではこのような二段階認証を用いた認証が現実的な解であると考えられる。

パスキー認証に完全に移行する例として、国内の My Sony の事例が挙げられる。同サービスでは、パスキー認証を有効化すると、これまで使用していたパスワードが無効化される仕様となっている。この構成では、完全にパスワードに依存しないログインを実現する一方で、パスキー認証が利用できない環境からのログインや、認証器紛失時のアカウントリカバリー手順の案内が不十分な場合、ユーザーがアカウントへアクセスできなくなる可能性がある。調査時点では、パスキー登録時に「パスワードが無効化される」旨の警告は確認された一方で、本調査で確認した範囲では、登録画面内で具体的なアカウントリカバリー手順（例：別の認証器の事前登録、バックアップコード等）へ到達するための明確な案内は確認できなかった。同様に、国外の Alibaba Cloud でも、一度パスキーを登録するとパスキー認証なしではログインできず、メー

ルアドレスや電話番号等の情報を入力してアカウント復旧を行う必要があることが観測された。このような挙動の場合、ユーザーが予期せずアカウントからロックアウトされるリスクがある。

また、Stripe ではパスキー認証が利用できない場合バックアップコードを利用する実装となっていたが、本研究の環境ではバックアップコードを利用してのアクセスには失敗した。その場合、身分証明書等を利用したアカウントリカバリーが求められた。

パスキー認証では、パスキー認証が利用できない状況に備えた代替のアカウントリカバリーを確保できているかが重要となる。特に「パスワード無効化」のようにパスキー認証へ強く依存する構成では、同期パスキーの利用を前提とする、あるいは本人確認に基づく代替のアカウントリカバリー手段を用意するなど、サービス側の設計判断がユーザーの可用性に直接影響する。

6　考察

本章では、第 3 章で整理した仕様・ガイドライン・先行研究による推奨と、第 5 章で観測した実装状況を対応づけ、推奨と異なる実装選択が確認された項目を中心に考察する。

まず、観測結果からアンチパターンを抽出し、さらに複数のアンチパターンを影響が現れる場面に沿ってまとめ、**アンチパターンの類型**として整理する。次に、パスキー認証に関連する新しい仕様が、これらの設計判断や推奨と異なる実装選択に与える影響を考察する。

6.1　アンチパターンの四類型

第 5 章では、Web Authentication API の主要パラメータと周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー等）の観点から、観測項目ごとに各サービスの実装状況を整理した。本節では、これらをユーザーが直面する場面に沿って横断的に整理し、アンチパターンを四つの類型として示す。各類型では、まず類型の概要を述べ、次に当該類型に該当するアンチパターン（実装例）を列挙する。さらに、観測例、影響、改善の方向性を述べる。

6.1.1　アンチパターンの類型 1：パスキー認証の利用開始に向けた案内の不足

この類型は、パスキー認証を導入していても、利用開始や利用時の案内が不足し、ユーザーが能動的に探索しない限りパスキー登録・パスキー認証の利用開始に至りにくい状態である。また、用語の選択や説明が不十分であるために、ユーザーが認証方式の性質を誤解する可能性のあるものもここに含める。

■アンチパターン

- パスキー登録・利用開始への案内を主要フロー（アカウント作成・ログイン等）から切り離し、設定画面内での探索に依存させる。
- UI 上で「パスキー/Passkey」という語を用いず、「生体認証」「Security Key」「U2F」等の語を混在させ、ユーザーが認証方式を理解しにくくさせる。
- 登録・追加・設定等、操作を表す用語・動詞の表記が不統一で、ユーザーが操作の意味を理解しにくい。

■観測例　5.2 節では、アカウント作成段階でパスキー登録が明示的に案内される例が限定的であることが確認された。国内では、調査対象 10 サービス中 3 サービスのみがアカウント作成時点でパスキー登録を案内していた。国外でも同様の傾向が見られ、NVIDIA ではサインアップの初期段階からパスキーを選択可能であり、Amazon ではアカウント登録手順の一環として自動的にパスキー登録プロセスが開始される挙動が確認された一方、多くのサービスではア

カウント作成後に設定画面から登録する構成であった。また、用語に関しては、例えば国内のMIXI M および RECRUIT ID で「生体認証」という語が併用されていたほか（表 6)、登録・追加・設定といった操作を表す動詞の表記揺れが観測された。加えて、国外サービスでは「パスキー/Passkey」と明記する例は半数に留まり、「Security Key」や「U2F」等の用語が混在していた（表 7)。

■影響　利用開始に向けた案内が不足している場合、パスキー認証の存在がユーザーに伝わりにくく、登録者が増えない。その結果、ログインや重要操作における主要な認証手段がパスワード認証等に残りやすく、パスキー認証の導入によって期待されるフィッシング耐性等の効果が限定される。さらに、登録数が伸びない状況では、管理画面の整備や再認証・通知といった周辺設計（後述の類型 3・4）も利用されにくく、「導入しているが使われない」状態が固定化されうる。また、用語や説明が不十分である場合、例えば「生体認証」という語から生体情報が RP に送信されると誤解され、ユーザーが登録や利用を避ける要因となり得る。さらに、パスキーが「Security Key」等の語で提示される場合、ユーザーがそれをパスキー認証として認知できず、利用開始に向けた案内が不足しているように見える要因となり得る。

■改善の方向性　パスキー登録・パスキー認証の利用開始は、アカウント作成・ログイン・設定・回復といった主要な画面遷移の中で、ユーザーが迷わない形で提示することが重要である。特に、用語は一貫させた上で、誤解が生じやすい語（例：「生体認証」）を用いる場合には補足を併記し、ユーザーの理解を支援する必要がある。

6.1.2　アンチパターンの類型 2：ユーザー操作支援の不足

この類型は、サービス側で対象ユーザーが既に定まっている状況（例：入力済みのユーザー ID、再認証の対象アカウント、パスキー登録の対象ユーザー）において、その文脈が Web Authentication API の呼び出しパラメータに反映されておらず、ユーザーに余計な選択や再入力を生じさせたり、意図しない状態変化（例：重複登録）を招いたりする状態である。

■アンチパターン

- ユーザー ID 等を入力済みにもかかわらず `navigator.credentials.get()` の引数で `allowCredentials` を設定せず、アカウント候補の再選択をユーザーに委ねる。
- 再認証など対象アカウントが既に定まっている場面でも `allowCredentials` を設定せず、誤選択・再試行を誘発する。
- 追加登録時に `excludeCredentials` を設定せず、重複登録を防がない。

■観測例　`allowCredentials` に関して、第 5 章では、ログイン手順の前半でメールアドレス等を入力させてユーザーを特定する文脈が存在するにもかかわらず、続く `navigator.credentials.get()` の呼び出し時に `allowCredentials` が設定されていない例として My

Sony が確認された（表 4）。この場合、OS/ブラウザは入力済みのメールアドレスに対応するパスキーのみに絞り込まず、利用可能なアカウント候補の選択をユーザーに委ねるため、ユーザーはメールアドレス入力後に再度アカウントを選択し直す必要がある。同様に X でも複数回のアカウント選択が必要となる挙動が確認されている（表 5）。

また、再認証においても My Sony では `allowCredentials` が設定されていないことが確認されており、ユーザーが別アカウントを誤って選択する、あるいは想定外のエラーとして扱われる可能性がある（第 5 章）。

国外の GoDaddy では、設定で「Default」として指定したパスキーの `Credential ID` のみが `allowCredentials` として設定されており、そのパスキーを利用できないデバイスからは、他のパスキーでログインするための操作が追加で必要となる挙動が確認された（第 5 章）。

一方で、ログイン開始時にユーザーにアカウント選択を委ねる設計（例：「パスキーでログイン」ボタン後の OS/ブラウザダイアログ、あるいは入力欄と統合された Conditional UI）では、`allowCredentials` を空とする指定自体は整合的である。問題となり得るのは、ユーザーを特定する情報を既に入力させている、または再認証として対象アカウントが既に定まっているにもかかわらず、アカウント選択が追加で発生する場合である。さらに、`excludeCredentials` が空である例として、国内の Yahoo! JAPAN、および国外の X、Norton、GoDaddy が確認されており、重複登録が成立する可能性のある状態であった（表 2、表 3）。

■影響　`allowCredentials` を空にしたまま運用すると、ブラウザ／ OS が提示する候補が（入力済み情報等により）想定されるユーザーに絞られないため、ユーザーは候補一覧から都度選択する必要がある。特に、入力済みのユーザー ID や再認証の対象アカウントが既に定まっている場面では、追加の選択が発生するだけでなく、誤って別アカウントのパスキーを選択し、サーバー側での検証（Credential ID の不一致やアカウントとの紐付け不一致等）により認証が失敗するといった、無駄な失敗が増える可能性がある。これにより、再試行や別方式へのフォールバックが増え、ログイン／再認証の所要時間や成功率が悪化する恐れがある。

`excludeCredentials` が空で重複登録が成立する場合も、管理画面上でどれを削除すべきかが分かりにくくなり、後述の類型 3（情報提示の不足）と結び付いて問題を増幅させる。

また、第 3 章で述べた通り、特に Discoverable Credential を用いる場合、同一 `rp.id` と `user.id` の組合せで既存の Discoverable Credential が上書きされる恐れがあるため [10]、`excludeCredentials` が空のままの追加登録は、管理上の混乱に加えて、RP 側が意図しない公開鍵で検証して認証に失敗する等の不整合を招く可能性がある。

■改善の方向性　ログインや再認証の場面でサービス側が対象ユーザーを特定できている場合は、その情報を `allowCredentials` に反映し、UI とパラメータの整合を保つことが有効である。追加登録においては `excludeCredentials` を設定し、重複登録が成立しないようにすることで、ユーザーの意図しない状態変化を抑えられる。

6.1.3　アンチパターンの類型 3：パスキー管理のための情報提示の不足

この類型は、ユーザーが自身のパスキーを長期的に管理するために必要な情報を提供しない状態である。具体的には、(i) どのパスキーがどのデバイス・環境に紐づくか識別できない、(ii) 不要・不審なパスキーを合理的に削除できない、(iii) 同期／デバイス固定の区別がつかず回復可能性を判断できない、といった形で現れる。

■アンチパターン

- 管理画面でパスキーの識別に必要な情報（例：登録日、最終使用日）を表示しない。
- 管理画面でパスキー名が汎用的であるにもかかわらず、名称変更等の識別性向上手段を提供しない。
- 同期パスキーかどうか等、回復可能性に関わる属性情報を管理画面上で提示しない。
- 登録が拒否される場合に理由や代替手段を明示せず、失敗の自己解決を困難にする。

■観測例　パスキー管理画面の調査結果（表 10、表 11）では、国内では名称変更ができないサービスが 10 サービス中 6 サービスで確認されたほか、パスキー名が汎用的である、あるいは OS/ブラウザ名のみが提示される例が確認された。また、最終使用日の表示が欠落している例として、国内のニンテンドーアカウント、My Sony、RECRUIT ID、および国外の Amazon、X が確認された。加えて、パスキーが同期パスキーかどうかに関する情報が管理画面上で提示されない例が多数であり、同期パスキーとデバイス固定パスキーの違いがユーザーに伝わりにくい状況が見られた。さらに、多様な認証環境の受け入れに関して、国内の RECRUIT ID、ニンテンドーアカウント、pixiv、Money Forward では仮想認証器による登録がブロックされる例が確認された。

■影響　管理情報が不十分であると、ユーザーは「使っていないパスキー」や「(ユーザーの見覚えのない）不審なパスキー」を識別できず、削除や棚卸しによるリスク低減が成立しにくい。特に最終使用日がない場合、現在使用中のパスキーと放置されたパスキーの区別が難しく、誤削除のリスクが高まる。また、同期可否が分からない場合、ユーザーは「追加のバックアップが必要か否か」を判断できない。認証器の登録がブロックされる場合に理由の説明が不足すると、ユーザーは失敗理由を理解できず、パスキー認証の利用自体を断念する可能性がある。

■改善の方向性　管理画面では、識別のための名称（自動設定・変更機能)、登録日・最終使用日等のメタデータ、同期可否に関する情報を提示し、ユーザーが自律的に棚卸しできる前提を整えることが重要である。加えて、登録が拒否される場合には、その理由と代替手段を示すことで、不要な断念や手戻りを抑えられる。

6.1.4　アンチパターンの類型 4：安全性を損なう周辺設計

この類型は、パスキー認証自体は適切に実装されていても、パスキー登録前の再認証やアカウントリカバリー等の周辺設計が不十分で、結果としてアカウント全体の安全性や可用性を損なう可能性のある状態である。

■アンチパターン

- パスキー登録・追加の直前に再認証を要求せず、セッション乗っ取り時の不正追加を抑止しにくい。
- パスキー登録完了後の通知（例：メール）を送らず、不正追加の早期検知機会を提供しない。
- パスキーに強く依存する構成（例：パスワード無効化）において、ユーザーが準備すべき回復手段や手順の案内が不十分である。
- アカウントリカバリーがフィッシング耐性の低い単一経路（例：パスワードのみ）に偏り、アカウント強度の下限を引き下げる。

■観測例　パスキー登録前の再認証に関して、国内では NIKKEI ID、ニンテンドーアカウント、pixiv のみが登録「前」に再認証を要求しており、My Sony では登録「後」に再認証が行われる特殊な手順が確認された（表 8）。国外では、NVIDIA、Namecheap、Norton、WordPress、Google、GitHub を含む複数サービスで、セッション確立後に追加の再認証を要求せずにパスキー登録が可能であった（表 9）。

また、登録完了後の通知に関して、国内の MIXI M、ニンテンドーアカウント、ヌーラボ、および国外の Cloudflare、GoDaddy、Mail.ru、Namecheap、Zoho では通知メールが送信されない例が確認された（表 8、表 9）。

アカウントリカバリー手段に関しては、パスワード以外の追加要素（メール・SMS の認証コード、バックアップコード等）が多くのサービスで採用されている一方（表 12、表 13）、パスキー登録後もパスワードでログイン可能な構成が残る例も確認された（例：Amazon、Google、X 等）。この構成は「パスキーが使えない」状況への耐性を高める一方で、アカウント全体の下限はパスワード経路の強度に依存し得る。また、My Sony のようにパスキー登録後にパスワードが無効化される構成や、国外の Alibaba Cloud のようにパスキーなしではログインできず復旧手順が必要となる構成も観測された。

■影響　パスキー登録前に再認証を要求しない、または再認証が不十分である場合、攻撃者がセッションを獲得したときに、新たなパスキーを登録される恐れがある。登録完了通知が無い場合、不正なパスキー登録が発生してもユーザーが気づく契機を欠き、対応が遅れる可能性がある。また、アカウントリカバリー経路がパスキー認証の性質（フィッシング耐性等）と異な

る方式に依存している場合、最終的なアカウント強度はアカウントリカバリー経路の性質に依存する。加えて、パスキーに強く依存する構成では、アカウントリカバリー手段の確保や案内が不十分だと可用性が低下し、ロックアウトが発生する可能性がある。

■改善の方向性　パスキー登録前の再認証に対しては、既存の認証基盤（メール OTP 等）も活用しつつ、セッション確立後に再認証を要求する設計が有効である。また、登録完了通知を送ることで、不正追加の早期検知を支援できる。アカウントリカバリー手段については、パスキーが利用できない状況を想定し、ユーザーが事前に準備すべき手段（別デバイスの追加登録、バックアップコード等）と、万一のアカウントリカバリー手順を明確に案内することが重要である。

6.1.5　まとめ

以上の類型は、第 5 章で観測された推奨と異なる実装選択から抽出されたアンチパターンを、影響が現れる場面の観点から横断的に整理したものである。また、これらは互いに独立ではなく、利用開始に向けた案内の不足（類型 1）が利用機会を減らし、ユーザー操作支援の不足（類型 2）が成功率を下げ、情報提示の不足（類型 3）が自己解決を困難にし、最終的に安全性を損なう周辺設計（類型 4）がアカウント全体の安全性・可用性を押し下げる、といった形で連鎖する可能性がある。したがって、個別項目の改善に加え、ユーザーがパスキー認証を継続利用できる前提（利用開始の経路、操作支援、管理、アカウントリカバリー）を一貫して設計することが重要である。

6.2　パスキーに関連する新たな仕様が与える影響

本節では、パスキー認証を取り巻く仕様進化のうち、第 3 章で整理した論点や、第 5 章で観測した実装状況に影響する可能性があるものを取り上げる。具体的には、パスキーやパスワードなど、認証に使われるクレデンシャル[*14]をプロバイダ間で移行するための Credential Exchange Specifications（Credential Exchange Format [53]、Credential Exchange Protocol [54]）を対象とし、RP の設計・運用にもたらす検討事項を述べる。

6.2.1　クレデンシャルエクスポート／インポート仕様（Credential Exchange Specifications）

FIDO Alliance は、パスキーを含む各種クレデンシャルを異なるプロバイダ間で安全に交換するための仕様群として、Credential Exchange Specifications を策定している。この仕様は、クレデンシャルのデータ構造を定義する Credential Exchange Format（CXF）[53] と、それらをアプリケーション間で安全に転送するためのプロトコルである Credential Exchange Protocol（CXP）[54] から構成される。

CXF は、パスキーやパスワード等のクレデンシャルを表現するための標準的なフォーマット

[*14] クレデンシャル は、ユーザー認証に用いられる情報（例：パスワード、パスキー等）を指す [52]。

を定義する。これにより、従来、CSV 等の非標準フォーマットや実装依存の形式で行われてきたクレデンシャルの移行を、暗号学的に保護された共通フォーマットに統一することを目的としている。一方、CXP は、二つのクレデンシャルプロバイダ（例えば、ブラウザ組み込みのパスワードマネージャとサードパーティ製パスワードマネージャ）間で、これら CXF 形式のデータをエンドツーエンドに暗号化して転送するためのプロトコルを定義する。

本論文執筆時点（2025 年）では、CXF v1.0 が FIDO Alliance による Proposed Standard として公表されており、CXP は Working Draft として標準化が進行中である。同仕様は、パスキーやパスワードなどのクレデンシャルを「ユーザーが選択した任意のプロバイダ間で安全に移行可能にする」ことを目指しており、パスキーを既存のパスワードと同等以上に移行しやすい認証手段とすることが企図されている。

これらの仕様が普及した場合、ユーザーはデバイスやプロバイダの変更に伴ってパスキーを移行しやすくなる。したがって、RP 側では「どの環境のパスキーが登録されているか」をユーザーが理解できるようにし、削除時の誤解（どこから何が消えるか）を避ける設計の重要性が高まる。

■エクスポート／インポートが RP 実装に与える影響　クレデンシャルのエクスポート／インポートが一般化すると、「鍵はデバイス/パスキープロバイダから出ない」という単純な説明が成り立ちにくくなり、「どのプロバイダ上に保存されているパスキーか」が変化する可能性がある。したがって RP 側の認証器/パスキープロバイダの受け入れ方針とパスキーの管理画面に対して、次のような重要な影響が生じる。

第一に、アカウント管理画面におけるパスキーの表示・命名規則が、エクスポート／インポートの導入によって一層重要になる。第 3 章では、ユーザーが自身の認証状態を理解しやすくするために、パスキーごとに名称を付与したり、登録日時を表示することの重要性を述べた。しかし、CXF/CXP によりクレデンシャルが他プロバイダに移行した場合、登録時点でのパスキープロバイダ名が実際の保存先と一致しなくなる可能性がある。例えば、もともとブラウザ組み込みのパスワードマネージャに格納されていたパスキーが、ユーザーの操作により別のパスワードマネージャへ移行した場合、RP が「このパスキーはブラウザ X 上に存在する」といった前提に基づいて付与したラベルは、現実と乖離する。

第二に、プロバイダ変更によるセキュリティ特性の変化に対して、どのように対応するのかという問題が生じる。CXP は、クレデンシャルの転送を暗号学的に保護することを目的としているが、クレデンシャルプロバイダ自体のセキュリティ方針や実装は、ベンダごとに異なりうる。例えば、ある Web サービスが特定の種類の認証器/パスキープロバイダを想定してリスク評価を行っていた場合、ユーザーがパスキーを別の認証器/パスキープロバイダに移行することにより、当初想定していなかったリスクを持つ場所でパスキーが管理される可能性がある。RP が全ての認証器/パスキープロバイダごとに厳格な許可・禁止の方針を設定することは、現実的には難しく、ユーザー選好とセキュリティ要件のバランスをどのようにとるかは今後の課題である。

もっとも、CXF/CXP の主たる目的は、ユーザーによるクレデンシャルプロバイダの選択権拡大およびロックインの緩和にあり、パスキー採用の阻害要因となっていた「プロバイダ変更時の移行困難さ」を低減する点に価値がある。したがって、RP に求められるのは、特定のプロバイダを一方的に排除することではなく、クレデンシャルの種類や利用シナリオに応じて、アカウント保護レベルとユーザーの選択自由度の折り合いをつけるための方針設計である。

7　結論

7.1　貢献と示唆

本研究の目的は、RP が導入・運用時に陥りやすい実装上の落とし穴となり得る点を、主要 Web サービスの実装実態に基づいて整理して提示することである。この目的のために、本研究は、仕様・公開ガイドライン・先行研究を基に評価観点を整理し、国内外の主要 Web サービス（国内 10 件・国外 20 件、計 30 件）における実装の実態を外部から観測可能な範囲で調査した。そして、観測結果からアンチパターンを抽出し、それらをアンチパターンの四類型として整理したうえで、その影響と改善の方向性を考察した。

本研究の主要な貢献は、次の三点に要約される。

1. **設計指針（評価観点）の整理**：仕様・ガイドライン・先行研究を参照し、Web Authentication API のパラメータ設定と、周辺設計（登録・認証手順における UI/UX、多様な認証環境への対応、アカウントリカバリー等）の観点から、サービスが実装時に検討すべき事項を整理した。
2. **主要な Web サービスに対する実装実態の整理**：同一の観測手順に基づき、各サービスが採用している設計選択を項目別に整理した。パスキー認証の利用開始に向けた案内、ログイン時の操作手順、パスキーの管理画面で提示される情報、追加登録時の確認手順や通知、アカウントリカバリーの設計に差があり、ユーザーの迷いやすさや誤操作の起きやすさが大きく変わり得ることを明らかにした。また、一部には運用上の保護を弱める可能性がある設計が見られ、安全性に影響する可能性が示唆された。
3. **アンチパターンの抽出と類型化**：第 5 章で得られた観測結果から、仕様や公開ガイドライン等が示す推奨と異なる実装のうち、安全性または利便性を損ない得る実装をアンチパターンとして抽出した。さらに、複数のアンチパターンを、影響が現れる場面（利用開始、認証操作、管理、回復等）に沿ってまとめ、四つの類型として整理した。これにより、RP は一つのアンチパターンを手掛かりとして、同種のアンチパターンが他にも存在しないかを横断的に点検でき、改修の優先順位付けを検討するための足がかりを得られる。

以上より、本研究は、仕様や公開ガイドライン等が示す推奨事項を導入・運用の点検に利用しやすい形で補完するために、主要 Web サービスの実装実態に基づくアンチパターンとその四類型を提示した。

7.2　今後の展望

最後に、本研究の限界を踏まえ、今後の研究課題を述べる。

■金融（銀行・証券）分野を含む調査対象の拡張　：本研究では、口座開設時に厳格な本人確認等を要する金融分野サービスは、アカウント作成の容易性に関する選定要件を満たさないため除外した。一方で、国内では金融分野におけるパスキー導入が急速に進展していることが確認されており [48]、当該分野を含めた実装実態の把握は重要である。今後は、調査設計（選定要件、調査環境、評価可能な範囲）を見直した上で、金融分野における導入実態を把握する必要がある。例えば、既に当該サービスのアカウントを保有するユーザーに協力を仰いで観測を行う、あるいは当該サービス事業者と連携し、観測環境や観測範囲を合意した上で調査を実施することが一つの案である。また、本研究は外部から観測可能な範囲を主な対象としているが、観測だけでは判別が難しい運用上の判断（例：パスキー登録前の再認証方針、通知方針、アカウントリカバリー手段の選定理由等）については、事業者に対するアンケート等により補完的に把握することも考えられる。

■Web Authentication API Level 3 の追加仕様を含む仕様進化の影響評価　：

本研究では、Web Authentication API 仕様のうち、Level 3 から新たに加わった仕様の一部（Conditional UI や Backup State/Backup Eligible フラグ）や、パスキーに関連する新たな仕様の一例（Credential Exchange Format（CXF）[53]、Credential Exchange Protocol（CXP）[54]）を取り上げたが、Web Authentication API Level 3 には他にも複数の拡張が含まれる。例えば、Signal API や Related Origin Requests 等 [10] は、認証の手順設計、パスキーの管理、プライバシー配慮といった観点に影響する可能性がある。今後は、これらの追加仕様が本研究の評価観点や観測手順にどのような変更を要するかを整理した上で、同様の観測枠組みにより実装状況の把握を継続する必要がある。

謝辞

本稿の執筆にあたり，日頃より様々な形でご指導，助言をいただきました佐古和恵教授に深く感謝申し上げます．また，日頃から研究を支えていただいた佐古研究室の方々にも感謝申し上げます．

参考文献

[1] Google, “Passkeys demo,” https://passkeys-demo.appspot.com/, accessed: 2025-12-28.

[2] E. Kitamura, “Help users manage passkeys effectively,” https://web.dev/articles/passkey-management, May 2025, accessed: 2025-11-25.

[3] D. Temoshok, Y.-Y. Choong, R. Galluzzo, M. LaSalle, A. Regenscheid, D. Proud-Madruga, S. Gupta, and N. Lefkovitz, “Nist sp 800-63-4: Digital identity guidelines,” 2025-08-01 04:08:00 2025, accessed: 2025-12-21. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=959880

[4] S. Komanduri, R. Shay, P. G. Kelley, M. L. Mazurek, L. Bauer, N. Christin, L. F. Cranor, and S. Egelman, “Of passwords and people: Measuring the effect of password-composition policies,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’11)*, 2011, pp. 2595–2604.

[5] Y. Zhang, F. Monrose, and M. K. Reiter, “The security of modern password expiration: An algorithmic framework and empirical analysis,” in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS ’10)*, 2010, pp. 176–186.

[6] FIDO Alliance, “Fido user authentication specifications | fido alliance,” https://fidoalliance.org/specifications, Dec. 2025, accessed: 2025-12-21.

[7] F. Alliance, “Fido alliance overview | fido alliance,” https://fidoalliance.org/overview/, accessed: 2025-12-30.

[8] W. W. W. Consortium, “W3c,” https://www.w3.org/, accessed: 2025-12-30.

[9] W3C, “Web authentication: An api for accessing public key credentials level 2,” World Wide Web Consortium (W3C), W3C Recommendation, Apr. 2021, accessed: 2025-12-21. [Online]. Available: https://www.w3.org/TR/webauthn-2

[10] ——, “Web authentication: An api for accessing public key credentials level 3,” World Wide Web Consortium (W3C), W3C Working Draft, Jan. 2025, accessed: 2025-12-21. [Online]. Available: https://www.w3.org/TR/webauthn-3

[11] F. Alliance, “Client to authenticator protocol (ctap),” FIDO Alliance, Proposed Standard, June 2022, accessed: 2025-12-30. [Online]. Available: https://fidoalliance.org/specs/fido-v2.1-ps-20210615/fido-client-to-authenticator-protocol-v2.1-ps-errata-20220621.html

[12] FIDO Alliance, “Passkeys,” https://fidoalliance.org/passkeys/, accessed: 2025-11-25.

[13] FIDO Alliance，“パスキーエクスペリエンス”，accessed: 2025-12-21.

[14] ——，“Fido アライアンス、2025 年世界パスキーデーにパスキーの普及とパスワードレスの未来を支持”，https://fidoalliance.org/fido-alliance-champions-widespread-passkey-adoption-and-a-passwordless-future-on-world-passkey-day-2025/?lang=ja，May 2025，accessed

2025-12-17.

[15] D. Kuchhal, M. Saad, A. Oest, and F. Li, “Evaluating the security posture of real-world fido2 deployments,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2381 – 2395. [Online]. Available: https://doi.org/10.1145/3576915.3623063

[16] P. Chen, “Vulnerability testing for webauthn,” Thesis, University of Twente, Enschede, Mar. 2024. [Online]. Available: https://purl.utwente.nl/essays/98532

[17] L. Lassak, A. Hildebrandt, M. Golla, and B. Ur, “"it’s stored, hopefully, on an encrypted server’’: Mitigating users’ misconceptions about FIDO2 biometric WebAuthn,” in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 91–108. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/lassak

[18] World Wide Web Consortium (W3C), “Web authentication: An api for accessing public key credentials level 1 is a w3c recommendation,” https://www.w3.org/news/2019/web-authentication-an-api-for-accessing-public-key-credentials-level-1-is-a-w3c-recommendation/, Mar. 2019, published: 5 March 2019.

[19] ——, “Web authentication: An api for accessing public key credentials level 2 is a w3c recommendation,” https://www.w3.org/news/2021/web-authentication-an-api-for-accessing-public-key-credentials-level-2-is-a-w3c-recommendation/, Apr. 2021, published: 8 April 2021.

[20] WHATWG, “Html standard,” Dec. 2025, section 7.1.1 Origins. Accessed: 2026-01-08. [Online]. Available: https://html.spec.whatwg.org/multipage/browsers.html

[21] えーじ，倉林 雅，小岩井 航介，パスキーのすべて: 導入・UX 設計・実装　技術評論社，Jan. 2025.

[22] K. Zaky, M. Wiseman, S. Miller, and E. L. Saint, “Fido attestation: Enhancing trust, privacy, and interoperability in passwordless authentication,” FIDO Alliance, White Paper, May 2024, accessed: 2025-12-18. [Online]. Available: https://fidoalliance.org/wp-content/uploads/2024/06/EDWG_Attestation-White-Paper_2024-1.pdf

[23] R. Lindemann, “Fido metadata service,” FIDO Alliance, Implementation Draft fido-metadata-service-v2.0-id-20180227, Feb. 2018, accessed: 2026-01-10. [Online]. Available: https://fidoalliance.org/specs/fido-v2.0-id-20180227/fido-metadata-service-v2.0-id-20180227.html

[24] E. Kitamura, “Passwordless sign-in on forms with webauthn passkey autofill,” https://developer.chrome.com/docs/identity/webauthn-conditional-ui, Nov. 2022, accessed: 2025-11-25.

[25] Google, “Introduction to server-side passkey implementation | google identity,” https://deve

lopers.google.com/identity/passkeys/developer-guides/server-introduction, May 2025, accessed: 2025-12-27.

[26] H. Furukawa, and T. Karino, "Synced Passkey Deployment: Emerging Practices for Consumer Use Cases," FIDO Alliance, White Paper, Jan. 2024, accessed: 2025-12-18. [Online]. Available: https://fidoalliance.org/wp-content/uploads/2024/05/Synced-Passkey-Deployment_-Emerging-Practices-for-Consumer-Use-Cases_2024-Final.pdf

[27] F. Alliance, "Passkey central," https://www.passkeycentral.org, accessed: 2025-12-21.

[28] FIDO Alliance, "原則 | passkey central", https://www.passkeycentral.org/ja/design-guidelines/principles, accessed: 2025-12-21.

[29] ——, "パスキーを使用したサインイン", https://www.passkeycentral.org/ja/design-guidelines/required-patterns/sign-in-with-a-passkey, May 2023, accessed: 2025-11-25.

[30] W3C, "Webauthn adoption community group," https://www.w3.org/community/webauthn-adoption/, accessed: 2026-01-11.

[31] W. W. C. A. Group, and the FIDO Alliance, "passkeys.dev," https://passkeys.dev, accessed: 2025-11-21.

[32] Google, "Passkeys | google identity," https://developers.google.com/identity/passkeys, Nov. 2024, accessed: 2025-12-21.

[33] ——, "web.dev," https://web.dev, accessed: 2025-12-21.

[34] M. Foundation, "Web authentication api," https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API, accessed: 2025-11-21.

[35] Yubico, "Passkey relying party implementation guidance," https://developers.yubico.com/Passkeys/Passkey_relying_party_implementation_guidance, accessed: 2025-11-21.

[36] M. Miller, "Simplewebauthn," https://simplewebauthn.dev/, accessed: 2025-11-21.

[37] W. Brown, "How hype will turn your security key into junk," https://fy.blackhats.net.au/blog/2023-02-02-how-hype-will-turn-your-security-key-into-junk/, Feb. 2023, accessed: 2025-11-21.

[38] A. Yuriy, "Webauthn/fido2: Demystifying attestation and mds," https://medium.com/webauthnworks/webauthn-fido2-demystifying-attestation-and-mds-efc3b3cb3651, May 2021, accessed: 2025-02-21.

[39] R. Lindemann, "Fido security reference," FIDO Alliance, Proposed Standard fido-security-ref-v2.1-ps-20220523, May 2022, accessed: 2026-01-10. [Online]. Available: https://fidoalliance.org/specs/common-specs/fido-security-ref-v2.1-ps-20220523.html

[40] Vincent, "WebAuthn pubKeyCredParams & credentialPublicKey: CBOR & COSE," https://www.corbado.com/blog/webauthn-pubkeycredparams-credentialpublickey, Corbado, Dec. 2025, accessed: 2025-12-21.

[41] E. Kitamura, and M. Nalpas, "Discoverable credentials deep dive," https://web.dev/articles/w

ebauthn-discoverable-credentials, Dec. 2023, accessed: 2025-12-23.

[42] Apple Inc., "Passkeys," https://developer.apple.com/passkeys/, Apple, accessed: 2026-01-11.

[43] Google, "Passkeys user journeys," https://developers.google.com/identity/passkeys/ux/user-journeys, Sep. 2025, accessed: 2025-11-25.

[44] E. Kitamura, "Secure and seamless passkeys: A deployment checklist," web.dev, June 2022, accessed: 2025-11-25.

[45] ——, "Determine the passkey provider with AAGUID," https://web.dev/articles/webauthn-aaguid, Mar. 2024, accessed: 2025-12-12.

[46] Fawaz Mohammad, Jecelyn Yeen, Sofia Emelianova, "WebAuthn: Emulate authenticators," https://developer.chrome.com/docs/devtools/webauthn, Oct. 2024, accessed: 2025-12-18.

[47] J. Kunke, S. Wiefling, M. Ullmann, and L. L. Iacono, "Evaluation of account recovery strategies with fido2-based passwordless authentication," 2021. [Online]. Available: https://arxiv.org/abs/2105.12477

[48] 大賀智子．(2025, 12) 証券口座の不正アクセス対策「パスキー認証」など必須に．日本経済新聞．Accessed: 2025-12-17．[Online]. Available: https://www.nikkei.com/article/DGXZQOUB128TL0S5A211C2000000/

[49] abcang，"パスキーにデフォルトの名前をつける方法について ～AAGUID の活用とフォールバックの工夫"，pixiv inside, https://inside.pixiv.blog/2024/08/15/170000，Aug. 2024，accessed: 2025-12-12.

[50] 奥田和史，"日経 ID で予定しているパスキー関連機能"，Hack Nikkei (Nikkei Engineering Blog), https://hack.nikkei.com/blog/advent20241221/，Dec. 2024，accessed: 2025-12-12.

[51] M. Foundation, "Publickeycredential: isconditionalmediationavailable() static method," https://developer.mozilla.org/en-US/docs/Web/API/PublicKeyCredential/isConditionalMediationAvailable_static, accessed: 2026-01-15.

[52] W. W. W. Consortium, "Credential management level 1," https://www.w3.org/TR/credential-management-1/, accessed: 2026-01-09.

[53] FIDO Alliance, "Credential exchange format," FIDO Alliance, Proposed Standard FIDO-CXF-v1.0-ps-20250814, Aug. 2025, accessed: 2025-12-07. [Online]. Available: https://fidoalliance.org/specs/cx/cxf-v1.0-ps-20250814.html

[54] ——, "Credential exchange protocol," FIDO Alliance, Working Draft FIDO-CXP-v1.0-wd-20240522, May 2024, accessed: 2025-12-07. [Online]. Available: https://fidoalliance.org/specs/cx/cxp-v1.0-wd-20240522.html